Escola de Engenharia Elétrica, Mecânica e de Computação

CERISE – Centro de Excelência em Redes Inteligentes Sem Fio e Serviços Avançados

Projeto de Implantação do CERISE-Centro de Excelência em Redes Inteligentes Sem fio e Serviços Avançados com Alcance Nacional e Internacional

Relatório Técnico-Científico

Título do Plano de Trabalho Individual: Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG

Bolsista: Eduardo Martins de Carvalho Caixeta

Grupo / Eixo Temático: Gêmeos Digitais / Navegação Autônoma / Realidade Aumentada

Coordenadores dos Grupos / Eixos Temáticos: Prof. Dr. Alisson Assis Cardoso (Navegação Autônoma e Gêmeos Digitais); Prof. Dr. Carlos Galvão Pinheiro Junior (Realidade Aumentada)

Versão: 26/10/2026

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

Neste relatório, são apresentadas as atividades e os resultados obtidos como bolsista do CERISE entre janeiro e outubro de 2026, referentes ao plano de trabalho individual intitulado “Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG”.

Primeiramente, apresenta-se uma introdução com a contextualização do trabalho. Em seguida, são expostos os objetivos constantes no plano de trabalho, as atividades desenvolvidas, os resultados obtidos e uma comparação entre as atividades previstas e aquelas efetivamente executadas, incluindo as pendências.

A realidade aumentada (RA) vem se destacando como uma tecnologia emergente com potencial para transformar diversas áreas da sociedade. Ao mesclar elementos virtuais ao ambiente físico em tempo real, a RA possibilita interações mais dinâmicas e informativas, com impacto direto em setores como saúde, educação, engenharia e entretenimento [1]. Um dos avanços mais notáveis está relacionado à sua crescente acessibilidade, impulsionada por dispositivos móveis: mapeamento tridimensional, reconhecimento de objetos e processamento em tempo real tornaram a RA mais precisa e adaptável a diferentes contextos [2]. Em aplicações industriais, a RA tem sido empregada em manutenção, treinamento técnico e assistência remota, enquanto no campo educacional enriquece experiências de aprendizado por meio de simulações imersivas [3], e na área da saúde apoia procedimentos e treinamentos médicos [4].

Um gargalo recorrente dessas aplicações é a obtenção de conteúdo tridimensional. Em 2025, foi desenvolvido um aplicativo capaz de capturar fotografias de um objeto e enviá-las a um servidor para a construção do modelo 3D correspondente, etapa que, naquela versão, era apenas simulada pelo servidor. Em 2026, a evolução de redes neurais generativas de imagem para 3D, capazes de produzir malhas texturizadas a partir de uma única fotografia em segundos, motivou a integração de um modelo dessa classe ao aplicativo, com otimização de desempenho e com um servidor capaz de atender múltiplos usuários simultaneamente.

Paralelamente, o conceito de gêmeo digital, uma representação virtual sincronizada com um sistema físico, depende de mecanismos de percepção capazes de estimar o estado do sistema real. No caso de braços robóticos, a

estimação de pose a partir de imagens, sem acesso aos estados das juntas, permite construir gêmeos digitais de robôs sem instrumentação adicional. Em 2025, foi treinado um modelo de estimação de pose de braço robótico baseado em visão computacional e profundidade monocular; em 2026, o trabalho avançou para a geração de um novo conjunto de dados com um braço robótico virtual, a validação do modelo nesse novo cenário e a construção do gêmeo digital do braço.

Por fim, no eixo de Navegação Autônoma, o uso cooperativo de múltiplos drones para monitoramento de áreas, por exemplo para detecção e contagem de pessoas, exige a divisão da área de atuação entre as aeronaves, o planejamento de rotas de cobertura e um enlace de comunicação de longo alcance e baixo consumo, papel para o qual a tecnologia LoRa é candidata natural.

2. OBJETIVOS DO PLANO DE TRABALHO

O objetivo geral deste plano de trabalho é realizar pesquisa, desenvolvimento e inovação em gêmeos digitais, tecnologias imersivas, robótica e navegação autônoma no escopo do CERISE/EMC/UFG. O plano está organizado em dois eixos:

- Desenvolvimento, aprimoramento e validação de soluções baseadas em inteligência artificial e visão computacional, visando à geração de objetos tridimensionais com textura e melhor desempenho no aplicativo desenvolvido em 2025; à utilização de um braço robótico, físico ou virtual, para a geração de novos conjuntos de dados; à validação do modelo de detecção de pose desenvolvido no ano anterior; ao desenvolvimento de um gêmeo digital do braço robótico; e ao desenvolvimento de software para segmentação de áreas em cenários com múltiplos drones.
- Desenvolvimento e consolidação da infraestrutura computacional, dos sistemas embarcados e dos mecanismos de comunicação do projeto, incluindo a otimização do servidor do aplicativo para múltiplas solicitações simultâneas, a publicação do aplicativo na Play Store e a continuidade do projeto de comunicação baseado em LoRa.

Os objetivos específicos para a consecução do plano de trabalho são:

- Objetivo 1 — Inteligência artificial aplicada à geração de modelos tridimensionais: integrar ao aplicativo um modelo de rede neural capaz de gerar objetos 3D com textura, otimizar o processamento e validar a aplicação em uso real.
- Objetivo 2 — Infraestrutura computacional e implantação do aplicativo:

otimizar o servidor para múltiplas solicitações, adequar a arquitetura para escalabilidade e confiabilidade e publicar o aplicativo na Play Store.

- Objetivo 3 — Geração de dados e validação de modelos de detecção de pose: utilizar um braço robótico (físico ou virtual) para gerar um novo conjunto de dados, validar o modelo de detecção de pose do ano anterior e analisar seu desempenho e robustez.
- Objetivo 4 — Desenvolvimento de gêmeo digital do braço robótico: modelar o braço em ambiente virtual, integrar o modelo de detecção de pose ao ambiente digital e reproduzir o comportamento do braço com base nos dados capturados.
- Objetivo 5 — Planejamento e adequação de sistema com drones para detecção e contagem de pessoas: planejar o sistema, desenvolver software de segmentação da área de atuação entre os drones, construir os módulos de comunicação definitivos e adequar os drones para recebê-los.

3. ATIVIDADES REALIZADAS

As atividades foram organizadas em frentes de trabalho (geração 3D, servidor e aplicativo, dataset e gêmeo digital do braço robótico, estimação de pose, comunicação LoRa e drones). Cada frente produziu código versionado, resultados medidos e guardados em arquivos (JSON e CSV) e evidências na forma de gráficos, capturas de tela e saídas de terminal. Nesta etapa, foram realizados:

(1) análise comparativa de 10 modelos abertos de geração 3D texturizada a partir de imagem e medição de 6 modelos ou variantes na estação NVIDIA DGX Spark (GB10) com 13 objetos do Google Scanned Objects, que resultou na seleção do Hunyuan3D-2 Turbo;

(2) otimização do modelo selecionado, com definição de três configurações de qualidade (*presets*) e empacotamento do pipeline de geração numa classe Python e numa imagem Docker;

(3) validação dos objetos 3D gerados, com testes automatizados, carregamento dos arquivos GLB nos visualizadores usados pelo aplicativo e análise qualitativa com fotografias reais de licença CC0;

(4) análise da arquitetura do servidor de 2025, reprodução dos seus gargalos com o modelo real e implantação de uma arquitetura assíncrona com fila persistente, cache, prioridade e controle de admissão, avaliada por testes de carga;

(5) integração do aplicativo CERISE AR 1.1.0 (React Native com Expo) ao

servidor real de geração 3D, com validação ponta a ponta no emulador Android e compilação para o Simulador iOS;

(6) adequação técnica e normativa do aplicativo para a Play Store e preparação do pacote de publicação (AAB assinado, política de privacidade, formulários e ficha da loja), sem a publicação propriamente dita;

(7) geração, verificação integral e documentação de um dataset sintético do braço robótico virtual servo6dof, com 5.000 poses e 30.000 imagens, em Gazebo e ROS 2;

(8) porte do modelo de estimação de pose de 2025 para a estação DGX Spark, adaptação ao braço servo6dof por transferência de aprendizado e avaliação de precisão, generalização entre câmeras, robustez e latência;

(9) caracterização e validação do modelo digital do braço (URDF, cinemática direta, espaço de trabalho, projeção sobre as imagens do dataset e planejamento com MoveIt 2);

(10) integração do modelo de pose a um gêmeo digital do braço, com visualizador web e nó ROS 2, avaliado em simulação;

(11) desenvolvimento e verificação, sem hardware de rádio, do software do protótipo de comunicação LoRa (protocolo, firmware, testes, simulação de acesso ao canal e dashboard) e projeto do módulo de comunicação dos drones;

(12) desenvolvimento de um pacote de software para segmentação da área de atuação entre drones heterogêneos e de algoritmos de realocação dinâmica, avaliados em simulação 2D;

(13) ajuste fino e avaliação de um detector YOLO11 para detecção e contagem de pessoas em imagens aéreas do conjunto VisDrone e validação, por simulação, da agregação das contagens entre drones;

(14) criação de cinco repositórios privados no GitHub, um por frente: cerise-2026-gen3d, cerise-2026-pose-servo6dof, cerise-2026-servo6dof-gemeo-digital, cerise-2026-drones-segmentacao e cerise-2026-lora-prototipo; as alterações de 2026 do aplicativo estão no repositório cerise-ar, no pull request #1, aberto como rascunho;

(15) uso da infraestrutura computacional do CERISE e de apoio: a estação NVIDIA DGX Spark, com GPU GB10 (arquitetura Blackwell), CPU aarch64 e 128 GB de memória unificada, compartilhada com serviços de IA em produção, para os experimentos com GPU, o servidor de geração 3D e as simulações do braço; um Mac mini M4 com Xcode para a compilação iOS; e um notebook com Windows e o Android SDK para as compilações Android, o emulador e a manutenção dos repositórios;

(16) elaboração deste relatório técnico-científico, com os resultados

rastreáveis aos arquivos de medição guardados nos repositórios.

4. RESULTADOS OBTIDOS

Nesta seção, é apresentado um resumo dos resultados obtidos:

1) Análise comparativa e seleção do modelo de geração 3D: o Hunyuan3D-2 Turbo foi selecionado, com a melhor geometria medida (distância de Chamfer de 0,032 e F-score@0,05 de 0,951);

2) Otimização do modelo integrado e desempenho: redução de cerca de 38 % no tempo de geração e três *presets* com tempos medianos de 23,9 s, 30,4 s e 82,1 s;

3) Validação dos objetos 3D gerados: 58 de 58 testes automatizados aprovados e 7 de 7 arquivos GLB carregados nos visualizadores;

4) Servidor de geração 3D: taxa de erro reduzida de 90,9 % para zero e latência p95 até 22 % menor com 16 clientes simultâneos;

5) Aplicativo CERISE AR com geração de objetos 3D texturizados: fluxo da foto ao modelo texturizado validado no emulador contra o servidor real, em 13,5 s a 31,5 s sem fila;

6) Adequação do aplicativo para a Play Store: pacote 1.1.0 pronto e verificado, com publicação pendente;

7) Dataset do braço robótico: 5.000 poses e 30.000 imagens, verificadas integralmente e documentadas;

8) Aplicação do modelo de pose de 2025 ao novo dataset: acurácia ADD@60 mm de 68,0 % com transferência de aprendizado, contra 32,8 % do baseline;

9) Avaliação de precisão e robustez do modelo de pose: robustez a brilho e contraste, sensibilidade a ruído, desfoque e compressão e latência de 162,5 ms por imagem;

10) Modelo digital do braço robótico: cinemática validada contra o ROS 2 e IoU de silhueta de 0,950 e 0,938 nas duas câmeras;

11) Gêmeo digital do braço robótico: erro médio de 5,41° nas juntas 1 a 4 e latência mediana de 99,6 ms, em simulação;

12) Protótipo de comunicação LoRa: software verificado sem hardware (51 testes automatizados e 1.130 execuções de simulação), com o ensaio físico pendente;

13) Projeto dos módulos de comunicação para os drones: requisitos, gabinete e estimativas de enlace e de energia, sem fabricação;

14) Segmentação da área de atuação entre drones: menor makespan em 31 de 40 casos com o DARP com refinamento por tempo;

15) Colaboração entre drones e realocação dinâmica: cobertura de pelo menos 99,7 % após o abandono de um drone, com aumento médio de 23,1 % no makespan;

16) Detecção e contagem de pessoas em imagens aéreas: mAP50 de pessoa de 0,528 (validação) e 0,316 (test-dev) após ajuste fino.

Serão apresentadas a seguir seções abordando com mais detalhes cada um dos tópicos acima.

4.1. Análise comparativa e seleção do modelo de geração 3D

Esta etapa corresponde à Atividade 1 do Objetivo 1 do plano de trabalho, que prevê a análise comparativa de redes neurais de geração de objetos 3D com textura. O aplicativo CERISE AR de 2025 enviava as fotografias capturadas a um servidor que apenas simulava a reconstrução e devolvia sempre o mesmo arquivo GLB, conforme detalhado na análise do servidor apresentada adiante. O objetivo da atividade foi levantar os modelos abertos de imagem para 3D texturizado e compará-los experimentalmente na infraestrutura do laboratório, a fim de selecionar a rede a ser integrada ao aplicativo.

4.1.1. Metodologia

Os experimentos foram executados na estação NVIDIA DGX Spark (GB10) do laboratório, com GPU de arquitetura Blackwell (sm_121), CPU aarch64 de 20 núcleos e 128 GB de memória unificada. Foi usado um contêiner derivado de `nvcr.io/nvidia/pytorch:26.05-py3` (PyTorch 2.12, CUDA 13.2), com as extensões CUDA dos modelos compiladas para sm_121 e o alocador do PyTorch limitado a 24 GB, teto definido para esta frente. A estação é de produção e foi compartilhada durante todas as medições, com a geração de um conjunto de dados de outra frente, treinamentos e serviços de inferência (vLLM). Por isso, os tempos absolutos devem ser lidos como limites superiores, dependentes da ocupação da máquina.

O conjunto de avaliação reuniu 13 objetos do Google Scanned Objects [5] (licença CC-BY 4.0), que fornecem a malha escaneada de referência. Cada malha foi normalizada e renderizada com nvdiffrast [6] numa vista de entrada de 1024×1024 px com fundo transparente (azimute 30°, elevação 15°) e em três vistas de referência não observadas pelos modelos (azimutes 120°, 210° e 300°). Na geometria, foram calculados a distância de Chamfer e o F-score a τ = 0,05 e 0,1, após normalização pela caixa envolvente e alinhamento por ICP com escala uniforme [7], iniciado em 12 guinadas e duas convenções de eixo. Na aparência, foi usada a similaridade de cosseno CLIP ViT-L/14 [8], calculada entre a entrada

e o render do resultado na mesma vista (CLIP entrada) e entre os renders do resultado e da referência nas três vistas novas (CLIP vistas novas). No desempenho, foram medidos a mediana do tempo por objeto em carga quente (modelos residentes e primeira geração descartada) e o pico de memória do alocador. Um autoteste do alinhamento, que aplica rotação, escala e translação arbitrárias à própria malha de referência, indicou piso de erro com Chamfer médio de 0,0042 e F-score@0,05 mínimo de 0,976. A saída desse autoteste permanece na estação de trabalho e não foi copiada para o repositório.

4.1.2. Levantamento da literatura

A tabela a seguir resume os modelos considerados e o resultado da tentativa de instalação no GB10, que combina CPU aarch64, arquitetura sm_121 e CUDA 13.

Tabela 1. Modelos abertos de geração 3D texturizada a partir de imagem e situação da instalação no GB10

Modelo (organização, ano)	Licença	Saída	Situação no GB10
Hunyuan3D-2.0 [9] (Tencent, 2025)	Tencent Hunyuan 3D 2.0 Community	malha e textura UV RGB	instalado e medido
Hunyuan3D-2mini Turbo (Tencent, 2025)	idem	malha e textura UV RGB	instalado e medido
Hunyuan3D-2 Turbo e FlashVDM [10] (Tencent, 2025)	idem	malha e textura UV RGB	instalado e medido
Hunyuan3D-2.1 [11] (Tencent, 2025)	Tencent Hunyuan 3D 2.1 Community	malha e materiais PBR	instalado com adaptações e medido
TRELLIS [12] (Microsoft, 2024)	MIT	gaussianas, campos de radiância ou malha	bloqueado: dependências sem versão aarch64
TRELLIS.2 (Microsoft, 2025)	MIT	malha O-Voxel e PBR	bloqueado: componente de acesso restrito
TripoSR [13] (Stability AI e Tripo AI, 2024)	MIT	malha com cor por vértice	instalado e medido
Stable Fast 3D [14] (Stability AI, 2024)	Stability AI Community	malha com UV, textura e material	bloqueado: pesos de acesso restrito
SPAR3D [15] (Stability AI, 2025)	Stability AI Community	nuvem de pontos e malha texturizada	bloqueado: pesos de acesso restrito
InstantMesh [16] (TencentARC, 2024)	Apache-2.0	malha com cor por vértice ou textura	instalado e medido

Os bloqueios são concretos. O TRELLIS depende de `spconv` e `cumm`, que não têm distribuição para linux-aarch64 no PyPI: nenhum dos 10 arquivos de cada distribuição consultada é aarch64 (`resultados/trellis_bloqueio_pypi.txt`). O TRELLIS.2 usa extensões CUDA próprias e o codificador DINOv3, distribuído em repositório de acesso restrito. Os pesos do Stable Fast 3D e do SPAR3D também exigem login no Hugging Face e aceite da licença pelo bolsista, etapa ainda pendente.

4.1.3. Benchmark prático

Foram instalados e medidos seis modelos ou variantes. Executar o Hunyuan3D-2.1 em aarch64 exigiu duas adaptações: o GLB com materiais PBR passou a ser escrito com trimesh, porque o Blender (`bpy`) não tem pacote para essa plataforma, e o rasterizador da versão 2.1 foi compilado à parte. Pelo custo de cerca de 4 min por objeto, o Hunyuan3D-2.0 na configuração padrão e o 2.1 foram medidos em subconjuntos de 4 e 2 objetos, respectivamente.

Tabela 2. Benchmark no GB10 em carga quente (n = objetos avaliados; tempo mediano por objeto)

Modelo	n	Tempo (s)	Pico GPU (GB)	Chamfer ↓ / F@0,05 ↑	CLIP entrada / vistas novas ↑
TripoSR	13	0,9	4,2	0,042 / 0,904	0,849 / 0,809
InstantMesh	13	16,5	23,2	0,046 / 0,893	0,844 / 0,836
Hunyuan3D-2 mini-Turbo	13	65,8	13,6	0,035 / 0,930	0,893 / 0,859
Hunyuan3D-2-Turbo	13	68,8	14,6	**0,032 / 0,951**	0,907 / 0,854
Hunyuan3D-2 .0 (padrão)	4	229,5	14,5	0,039 / 0,918	**0,933 / 0,884**
Hunyuan3D-2 .1 (PBR)	2	242,9	13,8	0,032 / 0,949	0,883 / 0,878

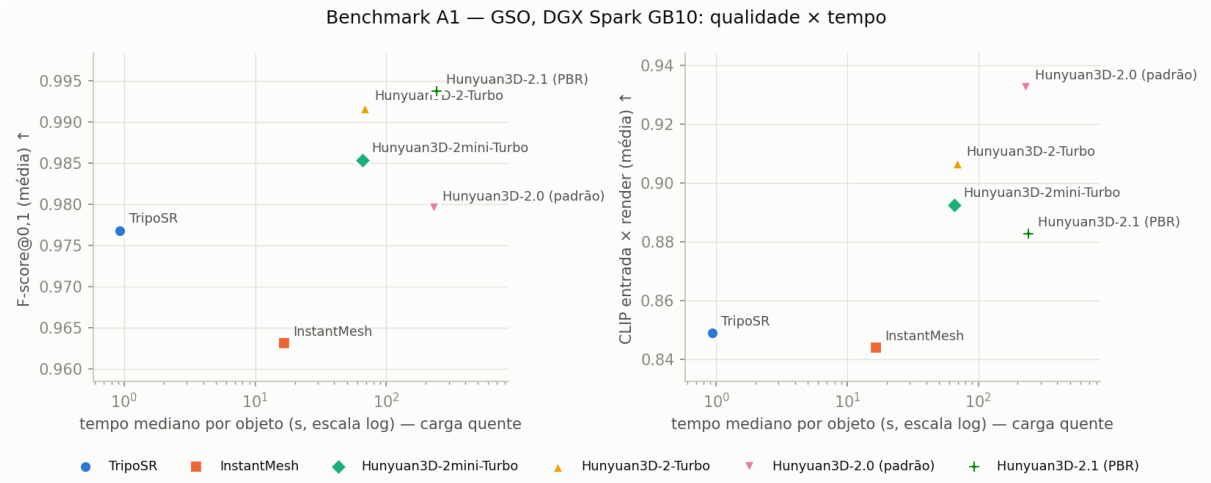


Figura 1. Qualidade em função do tempo mediano por objeto (escala logarítmica) no benchmark com objetos do Google Scanned Objects: à esquerda, F-score@0,1 da geometria; à direita, similaridade CLIP entre a entrada e o render do resultado



Figura 2. Comparação qualitativa numa vista não observada pelos modelos (azimute 210°): entrada, referência escaneada e resultado de cada modelo alinhado à referência; "não medido" indica objeto fora do subconjunto daquele modelo

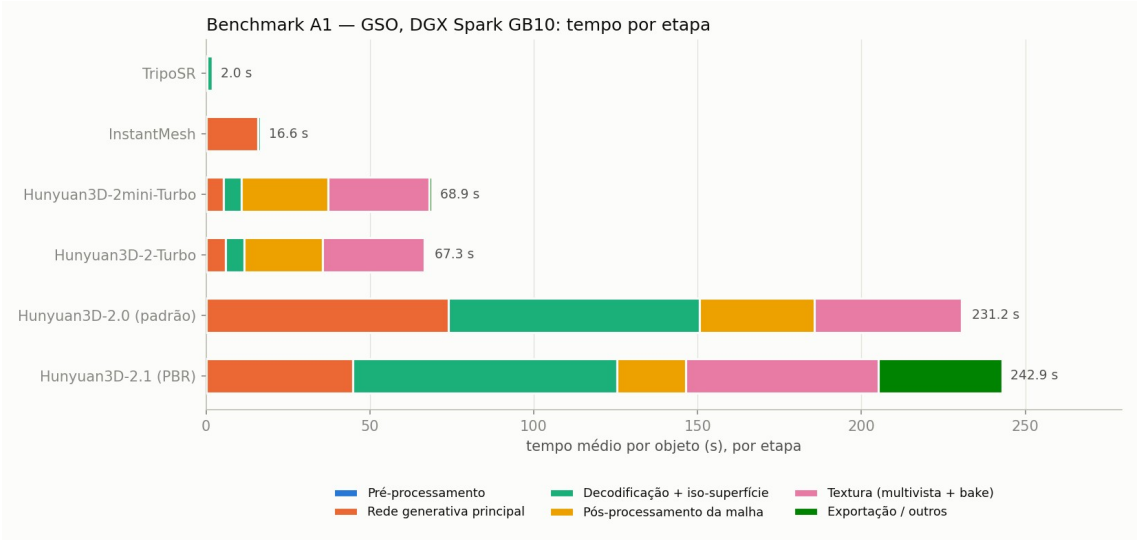


Figura 3. Tempo médio por objeto decomposto por etapa para cada modelo avaliado

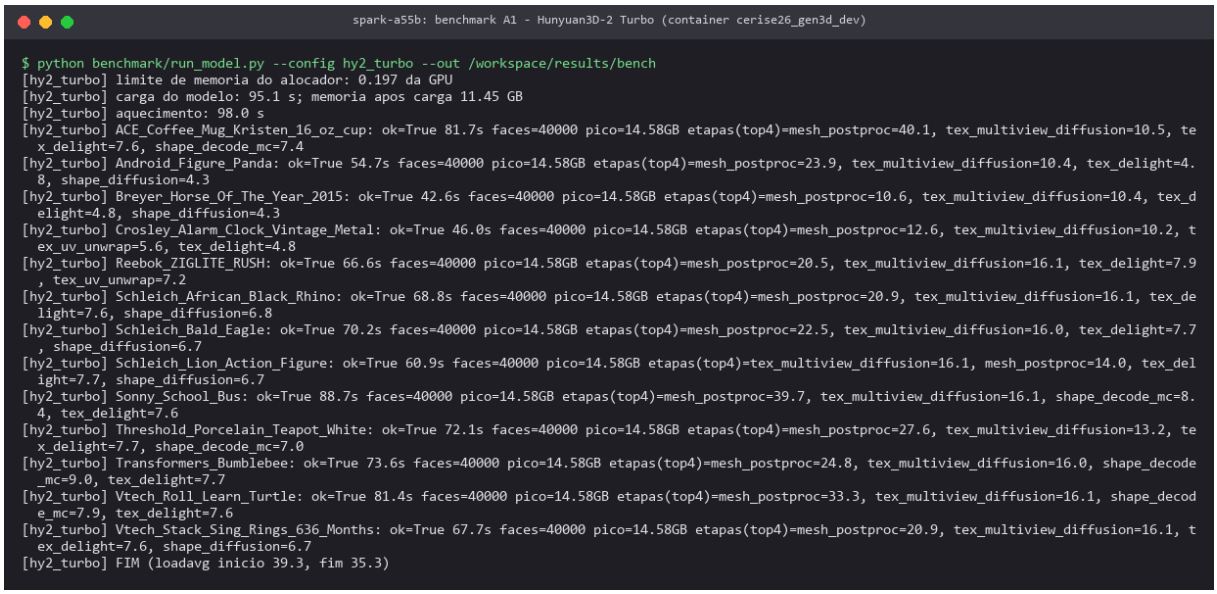


Figura 4. Saída do terminal do benchmark do Hunyuan3D-2 Turbo no contêiner da estação DGX Spark: carga do modelo, aquecimento e, por objeto, tempo total, faces, pico de memória e etapas mais longas

4.1.4. Discussão e seleção

O Hunyuan3D-2-Turbo obteve a melhor geometria no conjunto completo (Chamfer 0,032; F-score@0,05 de 0,951 e F-score@0,1 de 0,992). O Hunyuan3D-2.1 alcançou valores equivalentes nos dois objetos em que foi medido. O TripoSR foi duas ordens de grandeza mais rápido (0,9 s), mas apresentou geometria inferior e aparência limitada à cor por vértice, com as faces ocultas escuras e borradas. O InstantMesh também produz apenas cor por vértice e atingiu 23,2 GB de pico, no limite do teto da frente. Seu F-score@0,05 médio caiu para 0,766 no subconjunto que inclui a águia e a tartaruga, contra 0,913 apenas no ônibus e na caneca. Segundo as métricas por objeto mantidas na estação, essa queda se concentra na águia, cujas asas finas colapsam.

Na aparência, o Hunyuan3D-2.0 com o módulo de textura completo obteve os maiores valores de CLIP (0,933 na entrada e 0,884 nas vistas novas), à custa de 229,5 s por objeto. O CLIP do Hunyuan3D-2.1 ficou subestimado, porque o renderizador da avaliação usa apenas o albedo e não aplica sombreamento PBR, o que escurece o resultado. A comparação de tempos entre modelos é apenas indicativa: os modelos turbo foram medidos com média de carga da CPU próxima de 39, enquanto o 2.0 e o 2.1 rodaram com carga entre 3 e 4. A mesma configuração Turbo levou 28,7 s por objeto (mediana) na série sequencial de otimização, executada em outro horário, com a máquina parcialmente livre. A decomposição por etapa mostra que, nos modelos turbo, a rede de forma (difusão e decodificação) consome em média de 10,8 a 11,7 s, e o restante do tempo cabe à limpeza da malha com pymeshlab e à textura, que se tornaram os alvos da otimização. Já no 2.0 e no 2.1 predominam os 50 passos

de difusão (45 a 74 s) e a decodificação volumétrica sem FlashVDM (77 a 81 s).

Foi selecionado o **Hunyuan3D-2 Turbo** (forma destilada com FlashVDM, textura Paint-v2-0-Turbo e remoção de iluminação), pelos seguintes motivos: melhor geometria medida no conjunto completo; textura UV real, requisito para boa aparência em realidade aumentada; pico de memória de cerca de 14,5 GB, compatível com o teto da frente e com a coexistência com os serviços de produção; e licença que permite o uso no Brasil, embora exclua a União Europeia, o Reino Unido e a Coreia do Sul e exija licença própria para uso comercial acima de 1 milhão de usuários mensais. O TripoSR (MIT) fica documentado como alternativa de licença permissiva, e o Hunyuan3D-2.1 (PBR), como evolução futura quando houver GPU dedicada.

Código e evidências: repositório cerise-2026-gen3d (privado)

4.2. Otimização do modelo integrado e desempenho

Esta etapa corresponde à Atividade 3 do Objetivo 1 do plano de trabalho (aplicativo otimizado e relatório de desempenho), na parte relativa à otimização do modelo de geração. A integração com o servidor e com o aplicativo é tratada nas subseções próprias. O objetivo foi reduzir o tempo de geração do Hunyuan3D-2 Turbo sem perda mensurável de qualidade, definir configurações (*presets*) para o aplicativo e empacotar o pipeline para uso pelo servidor.

4.2.1. Variantes avaliadas

Partiu-se da configuração base: 5 passos de difusão, FlashVDM, octree 384, limpeza e decimação para 40 mil faces com pymeshlab, textura de 2048 px e precisão fp16. Cada fator foi alterado isoladamente: pós-processamento rápido da malha (descarte de componentes com menos de 1 % da área do maior e decimação quádrica com `fast-simplification`), octree 256, 20 mil e 10 mil faces, textura de 1024 px, 10 passos de difusão, módulo de textura completo (não destilado), forma em bf16 e FlashVDM desligado. Foram medidas duas séries. Na série intercalada, com 4 objetos, os modelos foram carregados uma única vez e todas as variantes de parâmetros foram executadas em sequência para cada objeto, o que as submete às mesmas condições de ocupação. Na série sequencial, com 7 objetos, cada variante foi executada numa rodada própria, o que permitiu incluir as variantes que exigem outros pesos, com tempos mais sensíveis à ocupação da máquina.

Tabela 3. Série intercalada (4 objetos; razão = mediana, por objeto, do tempo da variante dividido pelo tempo da base)

Variante	Tempo mediano (s)	Razão vs. base	Pós-proc. da malha (s)	F@0,05 ↑	CLIP entrada / vistas novas ↑
base	63,9	1,00	33,1	0,949	0,917 / 0,837
pós-	**37,5**	**0,62**	**6,2**	0,950	0,908 / 0,833

processamento rápido					
octree 256	**41,0**	**0,64**	13,1	0,951	0,914 / 0,844
20 mil faces	63,8	0,99	33,3	0,948	0,915 / 0,843
10 mil faces	63,6	0,98	33,6	0,949	0,914 / 0,845
textura 1024 px	63,0	0,99	33,0	0,949	0,912 / 0,843
10 passos de difusão	67,1	1,10	32,9	0,948	0,913 / 0,864

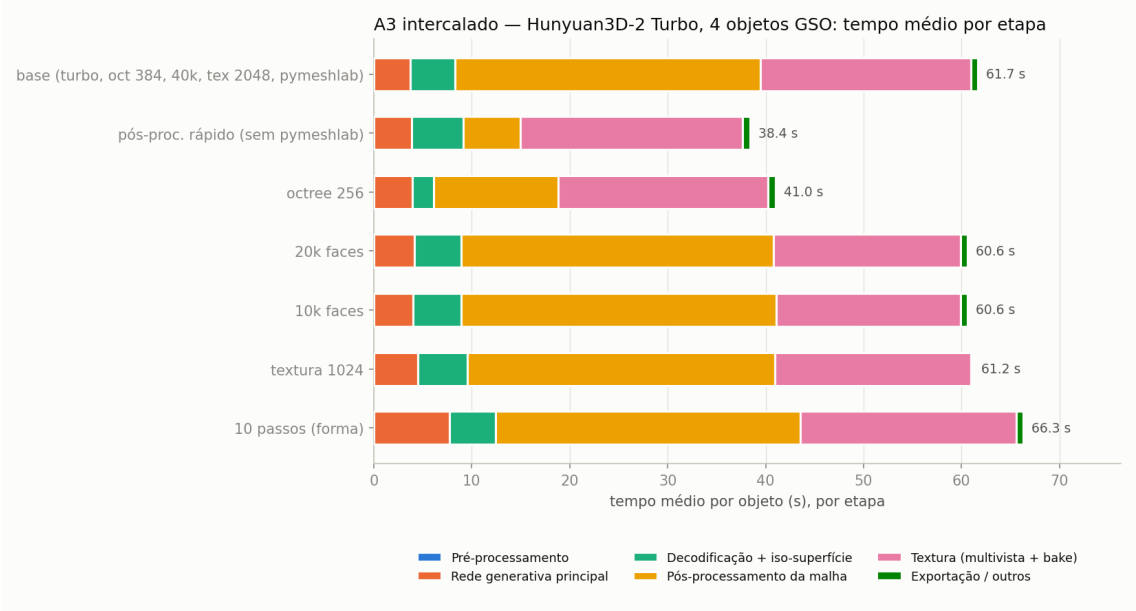


Figura 5. Tempo médio por objeto e por etapa de cada variante do Hunyuan3D-2 Turbo na série intercalada (4 objetos)

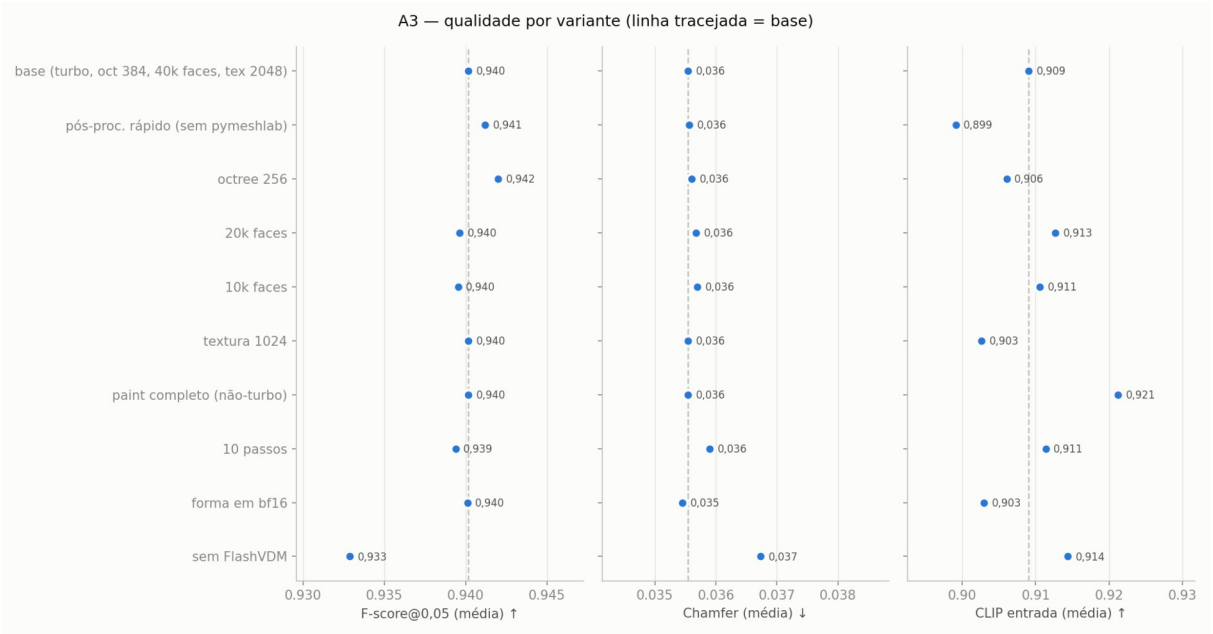


Figura 6. Qualidade de cada variante na série sequencial (7 objetos): F-score@0,05, Chamfer e CLIP de entrada; a linha tracejada indica a configuração base

Os principais resultados foram os seguintes:

- **FlashVDM** mostrou-se indispensável: sem ele, a etapa de forma passou de 3,5 s para 115,7 s (cerca de 33 vezes, medido em 3 objetos), sem ganho de geometria (F-score@0,05 de 0,933 contra 0,940).
- O **pós-processamento rápido** trouxe a maior economia: o tempo da limpeza caiu de 33,1 s para 6,2 s e o total foi reduzido em cerca de 38 % (razão mediana de 0,62 na série intercalada e de 0,55 na sequencial), com geometria inalterada e CLIP de entrada 0,009 menor.
- A **octree 256** reduziu o total em cerca de 36 % sem perda mensurável de qualidade.
- O orçamento de faces e a textura de 1024 px praticamente não alteraram o tempo, mas reduziram o GLB bruto (3,7 MB para 2,8 MB com 10 mil faces e para 1,9 MB com textura de 1024 px, ambos com textura PNG).
- **10 passos** custaram 10 % a mais de tempo, sem ganho geométrico, e elevaram o CLIP das vistas novas em 0,027.
- O **módulo de textura completo** elevou o tempo da textura de 12,0 s para 51,7 s na série sequencial, mas foi a única variante que melhorou claramente as partes ocultas (CLIP vistas novas de 0,846 para 0,893).
- A **forma em bf16** teve qualidade equivalente após uma correção de tipo no *marching cubes* e não trouxe ganho de tempo, de modo que o fp16 foi mantido. A compilação com `torch.compile` não foi avaliada.

4.2.2. Presets e pipeline empacotado

Com base nesses resultados, foram definidos três presets sobre um único modelo de forma (Turbo com FlashVDM e 5 passos) e com o pós-processamento rápido, o que permite manter os modelos carregados no servidor. O preset **rápido** usa octree 256, 10 mil faces e textura de 1024 px com o Paint Turbo. O **padrão** usa octree 384, 20 mil faces e textura gerada em 2048 px e entregue em 1024 px, também com o Paint Turbo. O **alto** usa octree 384, 40 mil faces, textura de 2048 px e o módulo de textura completo. Os presets foram medidos de ponta a ponta com o pipeline empacotado (remoção de fundo, geração, pós-processamento para o celular e miniatura) nos 13 objetos do GSO e em 6 fotografias reais, com os serviços de produção ativos na mesma máquina.

Tabela 4. Presets medidos no pipeline empacotado (19 entradas; GLB com textura PNG, anterior à correção descrita adiante)

Preset	Tempo mediano GSO (s)	Tempo mediano fotos reais (s)	Faixa (s)	Pico GPU (GB)	GLB mediano (MB)

rápido	23,9	24,0	21,9–27,3	14,5	1,65
padrão	30,4	29,8	26,8–76,6	14,5	2,01
alto	82,1	56,7	53,5–104,5	14,5	6,14

A geometria foi a mesma nos três presets (F-score@0,05 de 0,951 nos 13 objetos do GSO). O CLIP de entrada foi de 0,890, 0,894 e 0,922 e o CLIP das vistas novas, de 0,840, 0,851 e 0,876, nos presets rápido, padrão e alto. A carga a frio do pipeline levou 115 s, com 11,4 GB alocados, e o aquecimento, 79 s. No preset alto, esses tempos foram de 107 s e 118 s. Esses valores justificam manter os modelos residentes no servidor. A dispersão dos tempos (até 76,6 s no padrão) e a diferença do preset alto entre GSO e fotos reais refletem a variação de ocupação da GPU durante as rodadas.

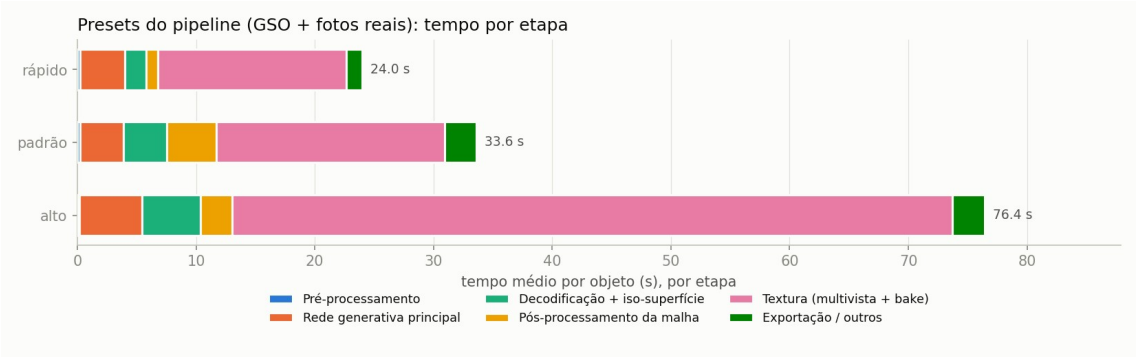


Figura 7. Tempo médio por objeto e por etapa dos três presets no pipeline empacotado; no preset alto predomina a difusão multivista do módulo de textura completo

O pós-processamento para o celular (`pipeline/postprocess.py`) assenta o objeto em y = 0 e o centraliza em x e z, corrigindo o script de ajuste de 2025, que deslocava o eixo Z pelo centro do eixo Y. Em seguida, escala a maior dimensão para 0,30 m, reduz a textura com glTF-Transform e valida o arquivo com o Khronos glTF-Validator. Por requisito do aplicativo, cujo `<model-viewer>` funciona sem conexão e cujo ViroReact não tem decodificadores, o GLB entregue não usa Draco, meshopt nem KTX2. O pipeline foi empacotado na classe `Gen3DPipeline`, que carrega os modelos uma única vez e oferece o método `generate(imagem, saída, preset)`, e na imagem Docker `cerise26_gen3d:latest` (38 GB), conforme ilustrado abaixo.

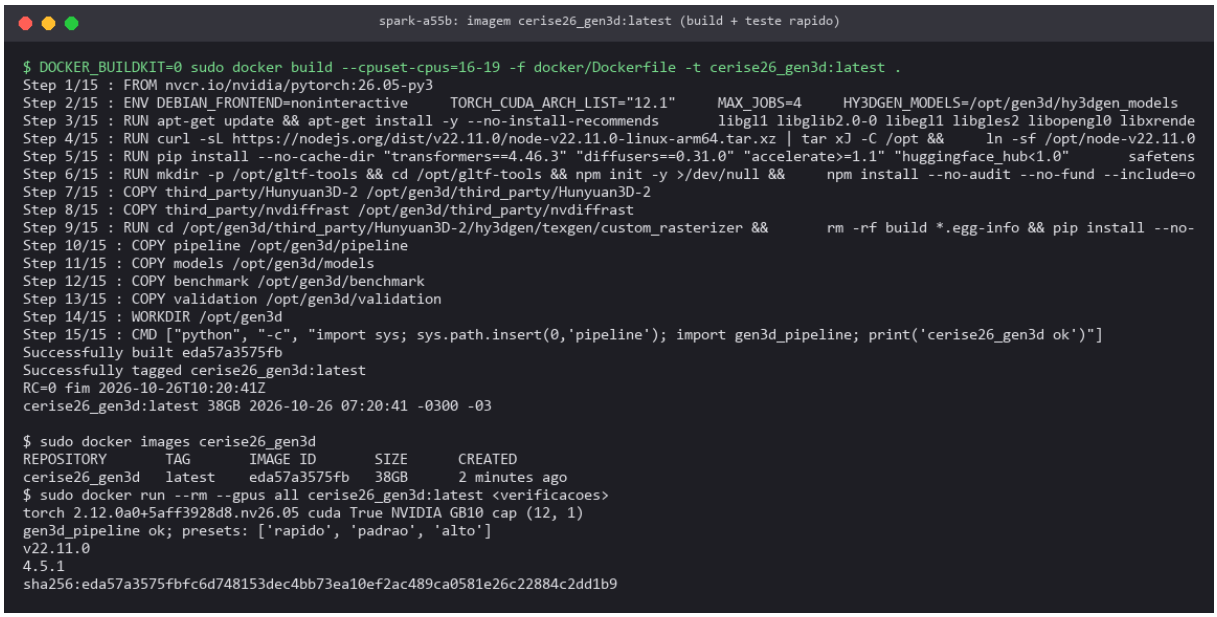


Figura 8. Construção da imagem Docker cerise26_gen3d:latest na estação DGX Spark e teste rápido dentro dela: PyTorch com CUDA no GB10, módulo do pipeline, presets, Node 22 e glTF-Transform

4.2.3. Errata da textura e limitações

O teste ponta a ponta do servidor, realizado em 26/10/2026, revelou que a recodificação da textura em JPEG não ocorreu nas rodadas desta seção. O pós-processamento chamava `gltf-transform jpeg` sem a opção `--formats`, e, na versão 4.5 da ferramenta, o padrão recomprime apenas texturas que já são JPEG. Assim, a textura PNG gerada pelo modelo passava intacta. Os tamanhos de GLB da tabela de presets estão, portanto, superestimados, e a comparação feita com essas rodadas, segundo a qual a compressão geométrica economizaria apenas cerca de 21 %, não é mais válida. As conclusões de tempo e de geometria não se alteram.

A correção (`--formats "*"`) foi aplicada no motor do servidor e em `pipeline/postprocess.py` no repositório. Com ela, a mesma fotografia de teste passou de 2,97 MB para 0,72 MB no preset padrão e de 2,60 MB para 0,44 MB no rápido, usado pela rota legada do servidor. No GLB corrigido, a textura JPEG ocupa 0,23 MB dos 0,72 MB, de modo que a geometria passa a responder pela maior parte do arquivo. Ainda falta reconstruir a imagem Docker com a correção e refazer a rodada de presets para obter as medianas definitivas. Todos os tempos foram medidos com a máquina compartilhada, e recomenda-se repeti-los numa janela ociosa da GPU.

Código e evidências: repositório cerise-2026-gen3d (privado)

4.3. Validação dos objetos 3D gerados

Esta etapa corresponde à Atividade 4 do Objetivo 1 do plano de trabalho,

que prevê o relatório de validação e os resultados experimentais. As métricas quantitativas de geometria e de aparência foram apresentadas nas subseções anteriores. Aqui, o pipeline selecionado e otimizado foi validado de três formas: testes funcionais automatizados sobre a mesma classe usada pelo servidor, carregamento dos arquivos nos visualizadores empregados pelo aplicativo e análise qualitativa, incluindo os casos de falha.

4.3.1. Testes funcionais automatizados

A suíte `validation/test_pipeline.py` (pytest) gera modelos a partir de três entradas em cada preset: duas imagens RGBA do GSO (águia e caneca) e uma fotografia real com fundo (bule), que força a etapa de remoção de fundo. A suíte verifica, para cada GLB final, os critérios resumidos na tabela a seguir e inclui ainda dois testes sem GPU, do assentamento e escala e da redução de textura.

Tabela 5. Verificações automatizadas do pipeline empacotado (9 GLBs: 3 entradas em 3 presets)

Verificação	Critério	Resultado
Validade do arquivo	Khronos glTF-Validator sem erros	aprovado
Material	material PBR com `baseColorTexture`	aprovado
Compatibilidade com o app	sem Draco, meshopt ou KTX2; textura JPEG ou PNG	aprovado
Orçamento de faces	até 10 mil, 20 mil e 40 mil faces conforme o preset	aprovado
Escala e posição	maior dimensão de 0,30 m, base em y = 0, centrado em x e z	aprovado
Tempo e tamanho	até 60, 90 e 180 s; até 3, 4 e 10 MB	aprovado
Total	rápido e padrão: 38 testes em 189,7 s; alto: 20 testes em 135,2 s	58 de 58 aprovados

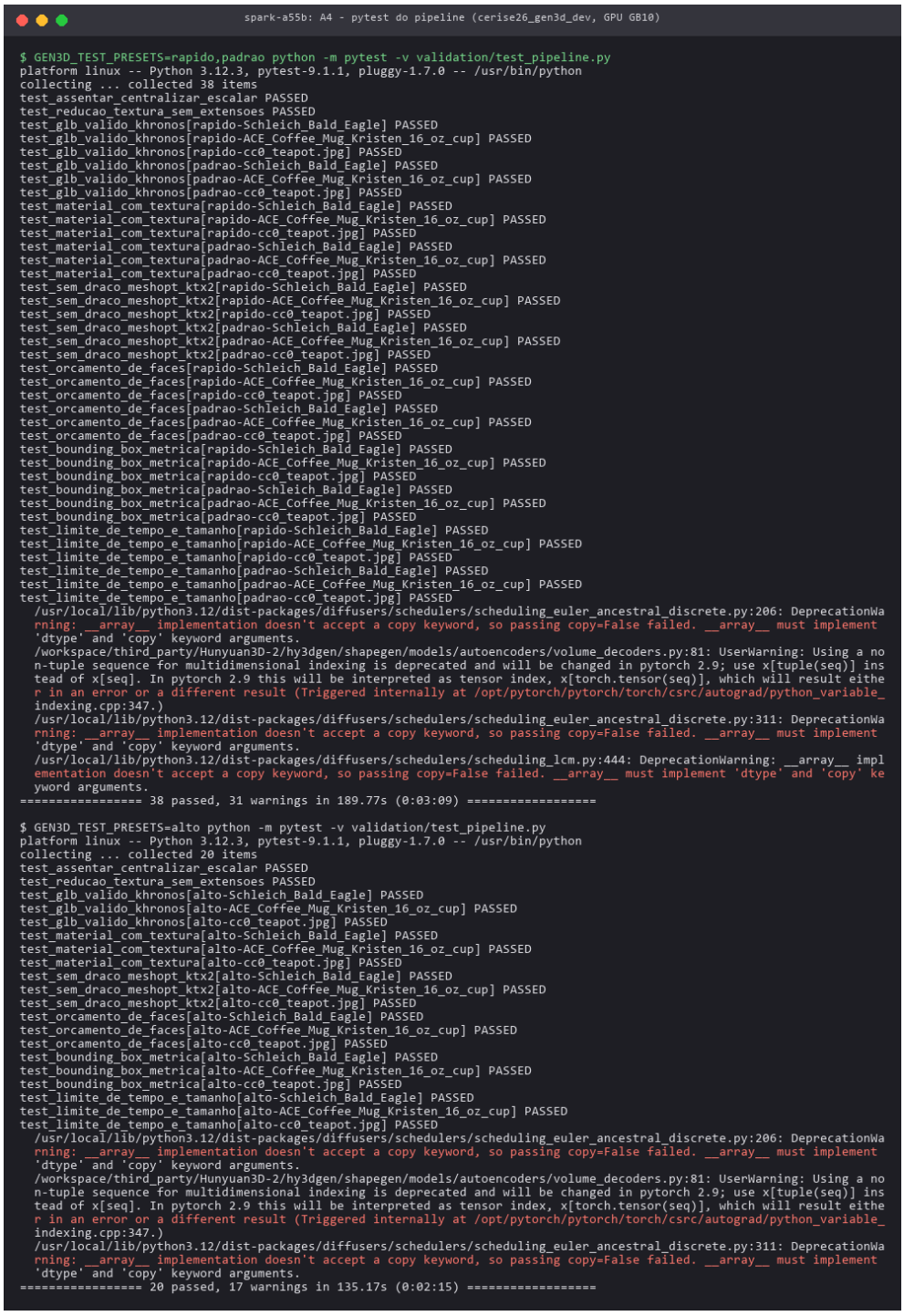


Figura 9. Saída do terminal da suíte pytest do pipeline no contêiner da estação DGX Spark: 38 testes aprovados nos presets rápido e padrão e 20 no preset alto

A suíte aceitava textura JPEG ou PNG e teve os limites de tamanho calibrados com a rodada de presets. Por isso, não detectou o defeito de recodificação da textura descrito na subseção anterior: os GLBs do teste tinham textura PNG. Recomenda-se exigir textura JPEG e recalibrar os limites de tamanho após a nova rodada de presets.

4.3.2. Carregamento nos visualizadores

Sete GLBs finais (cinco do preset padrão, um do rápido e um do alto, com objetos do GSO e fotografias reais) foram abertos em Chromium sem interface (Playwright, com WebGL por software) numa página com o `<model-viewer>` e, em paralelo, com o `GLTFLoader` do three.js. Os sete carregaram nos dois visualizadores, sem erros no console e sem extensões glTF, com uma malha, um material com mapa de cor, a contagem de triângulos do preset, maior dimensão de 0,30 m e base em y = 0 (`resultados/viewer/viewer_report.json`). Esses arquivos são anteriores à correção da textura. O GLB corrigido, com textura JPEG, foi validado da mesma forma no teste ponta a ponta do servidor, apresentado na subseção correspondente.

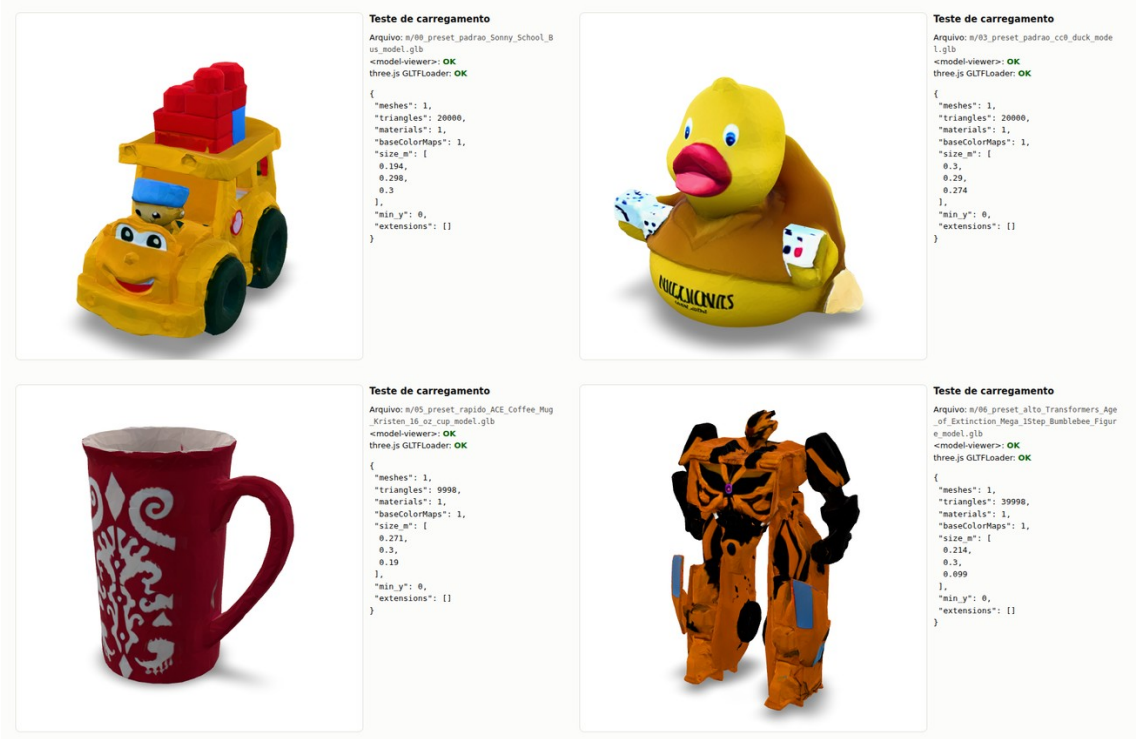


Figura 10. GLBs finais carregados no model-viewer e no three.js em Chromium sem interface, com as estatísticas lidas pelo GLTFLoader: ônibus e pato no preset padrão, caneca no rápido e robô no alto

4.3.3. Análise qualitativa e casos de falha

As figuras a seguir mostram, no preset padrão, a entrada, os renders frontal e de três quartos e um detalhe da textura para oito objetos do GSO e para as seis fotografias reais. Essas fotografias, com licença CC0, foram obtidas no Wikimedia Commons: quatro peças do acervo aberto do Metropolitan Museum (bule de prata e estatuetas de burro, galo e gato), um bule fotografado no Victoria and Albert Museum e um pato de borracha sobre fundo complexo. Nas vistas observadas, a textura reproduz detalhes finos, como o rosto e o boné do

motorista do ônibus, os cadarços e o logotipo do tênis e o padrão da caneca. A geometria preserva estruturas finas, como as pernas do cavalo e as asas da águia. Nas fotografias reais, a remoção de fundo funcionou inclusive sobre a grade metálica atrás do pato, e o pipeline produziu formas plausíveis para estatuetas cerâmicas antigas, bem diferentes dos objetos domésticos do GSO.

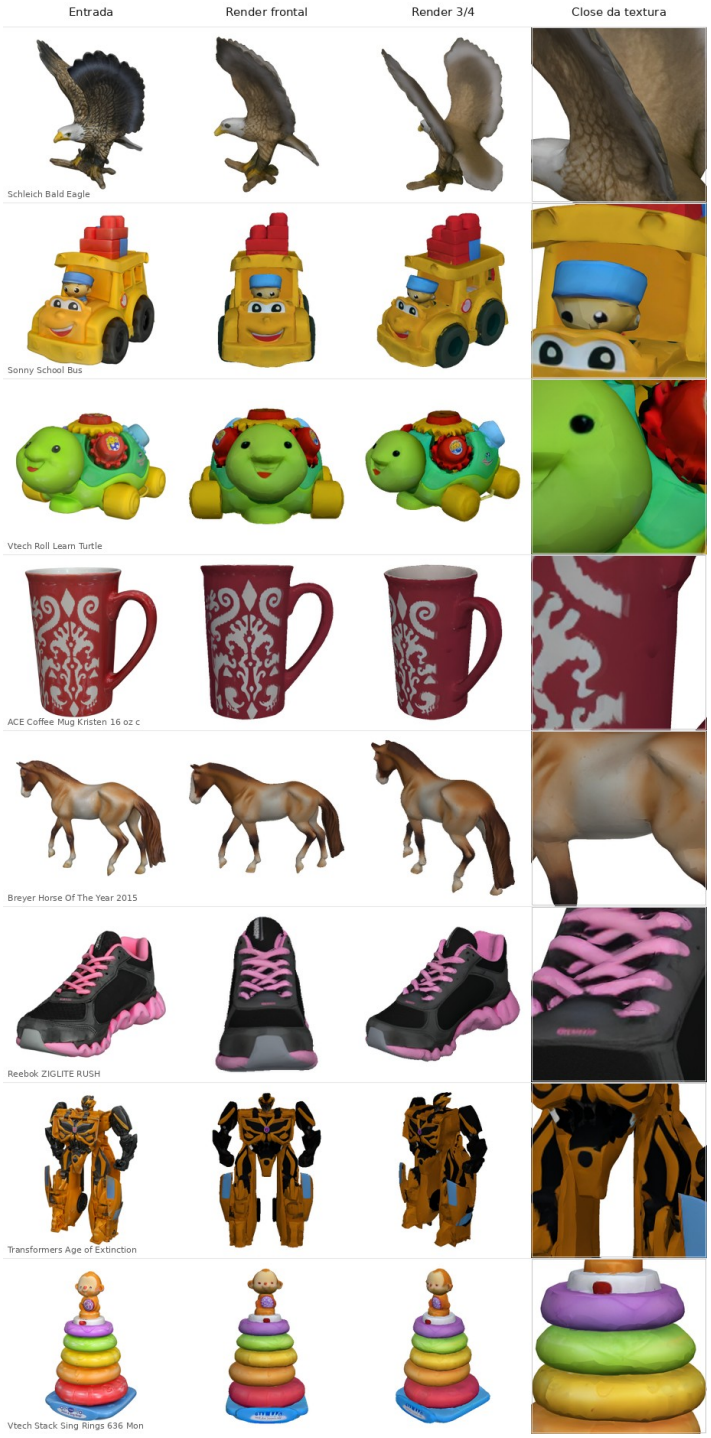


Figura 11. Grade qualitativa do preset padrão em oito objetos do Google Scanned Objects: entrada, render frontal, render de três quartos e detalhe da textura

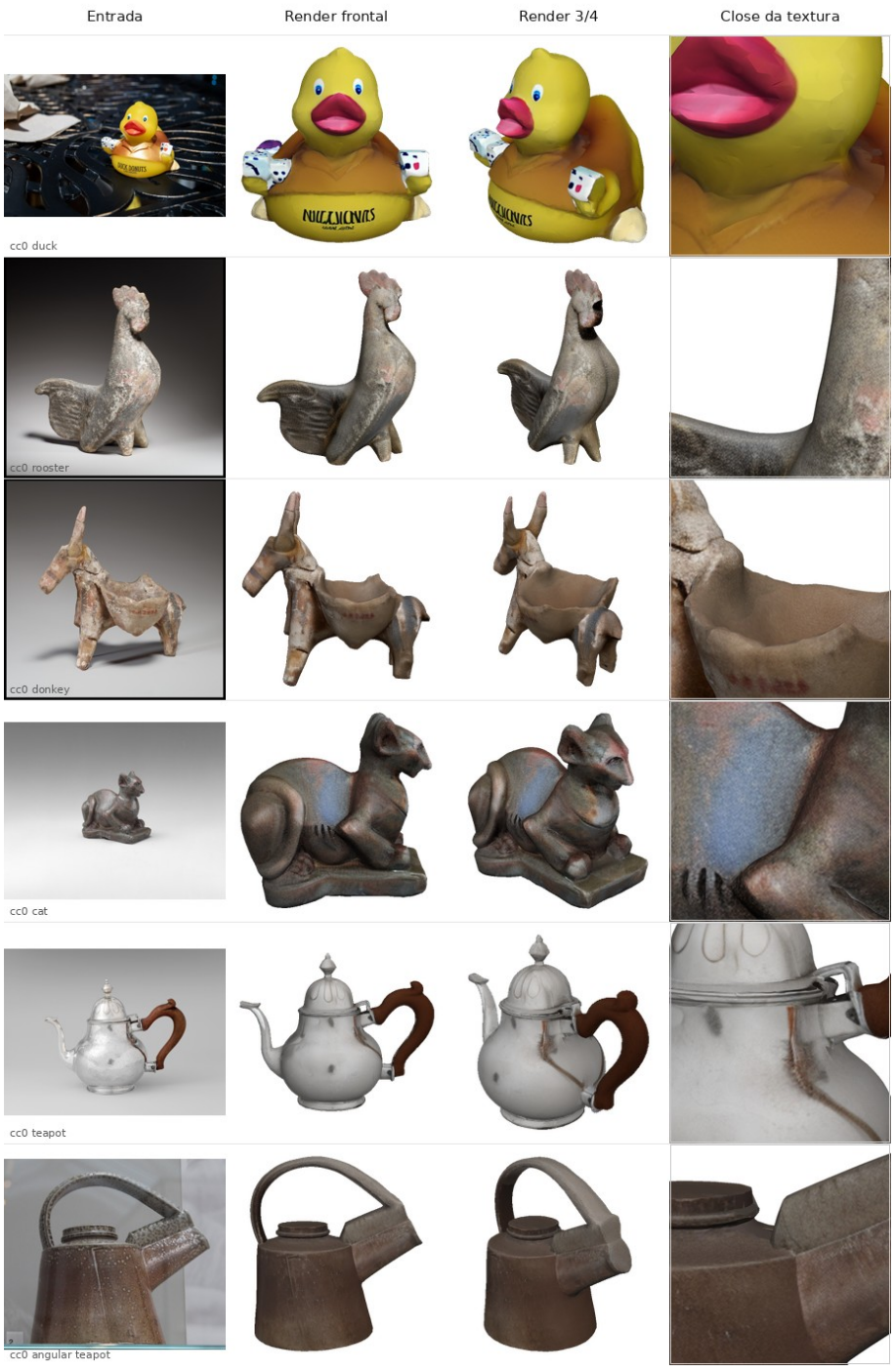


Figura 12. Grade qualitativa do preset padrão nas seis fotografias reais com licença CC0: entrada com fundo, render frontal, render de três quartos e detalhe da textura

Foram identificados os seguintes casos de falha e limitações:

- **Partes ocultas**: as faces não visíveis na foto são inferidas pelo modelo e tendem a sair mais lisas e acinzentadas, como nas costas das asas da águia. Regiões escuras ou finas podem sair pretas ou com lacunas, como nas costas do robô. O módulo de textura completo do preset alto atenua o problema (CLIP vistas novas de 0,876, contra 0,851 no

padrão).

- **Materiais metálicos ou especulares**: no bule de prata, os reflexos ficam incorporados ao albedo e o aspecto metálico se perde, porque o Hunyuan3D-2 não gera materiais PBR. O Hunyuan3D-2.1 trataria esse caso com mapas de metal e rugosidade.
- **Objetos compostos**: os dados seguros pelo pato saem como peças parcialmente destacadas, e a limpeza de componentes pequenos pode remover partes legítimas com menos de 1 % da área.
- **Estruturas finas nos concorrentes**: o InstantMesh colapsou as asas da águia, o que responde pela maior parte da queda do seu F-score@0,05 no subconjunto que a inclui; no TripoSR, as faces ocultas saem escuras e borradas.
- **Limitações da avaliação**: o CLIP mede semelhança visual, e não exatidão métrica; o renderizador sem sombreamento PBR subestima o Hunyuan3D-2.1; e não houve ainda teste com fotografias capturadas pelo próprio aplicativo no celular, etapa que depende da integração com o aplicativo, tratada adiante.

Código e evidências: repositório cerise-2026-gen3d (privado)

4.4. Servidor de geração 3D: arquitetura, gargalos e múltiplas requisições

Esta etapa corresponde à Atividade 5 do Objetivo 2 do plano de trabalho, que prevê a análise da arquitetura do servidor do aplicativo e a identificação de gargalos no atendimento a múltiplas requisições, e abrange também as metas associadas de otimização do servidor para solicitações simultâneas e de adequação da arquitetura para maior escalabilidade e confiabilidade. O servidor executa o modelo de geração 3D selecionado e otimizado nas subseções anteriores (Hunyuan3D-2 Turbo [9]) e está hospedado na mesma estação DGX Spark, compartilhada com os serviços de IA em produção.

4.4.1. Arquitetura de 2025 e gargalos identificados

O servidor de 2025 era um aplicativo Flask executado com o servidor de desenvolvimento, em modo de depuração. A rota `POST /` recebia as fotografias, executava `time.sleep(67)` para simular o processamento e devolvia sempre o mesmo arquivo GLB. A leitura do código evidenciou os seguintes gargalos:

- processamento dentro da requisição, com a conexão HTTP presa durante toda a geração;
- ausência de fila e de limite de concorrência, de modo que pedidos simultâneos resultariam em chamadas concorrentes à mesma GPU e aos mesmos objetos do modelo;

- nenhum estado persistente (identificador de pedido, progresso, cancelamento ou retomada) e arquivos com nomes fixos, sujeitos a sobrescrita entre pedidos;
- ausência de validação de entrada, autenticação, TLS, métricas e registros estruturados.

Como a espera simulada não exercita a GPU, a arquitetura de 2025 foi reproduzida com o modelo real, substituindo o `sleep` pela chamada ao pipeline de geração. Foram medidos dois modos: sem trava, que corresponde ao código original ao receber o modelo, e com trava global, a correção mínima que serializa o uso da GPU, mas mantém as conexões presas. Sem trava, a partir de dois clientes simultâneos todos os pedidos falharam com HTTP 500, porque as chamadas concorrentes compartilham o *scheduler* da difusão, cujo índice de passo é estado mutável, conforme ilustrado abaixo.

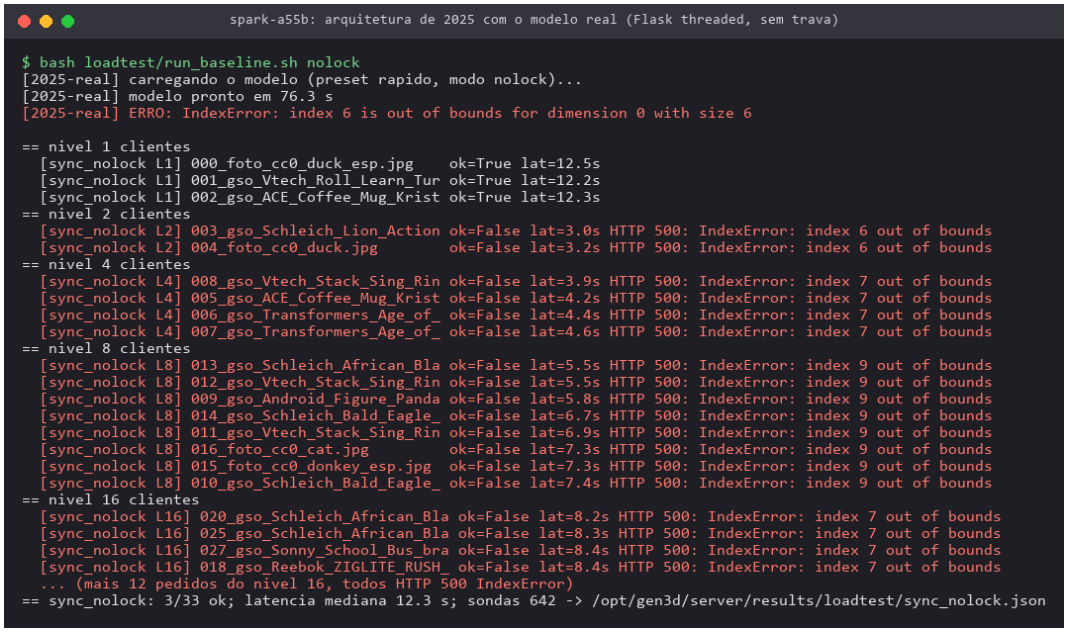


Figura 13. Execução da arquitetura de 2025 com o modelo real e sem trava: a partir de dois clientes simultâneos todos os pedidos falham com HTTP 500 devido ao estado compartilhado do scheduler de difusão

4.4.2. Arquitetura implementada em 2026

A nova arquitetura separa a recepção dos pedidos do processamento na GPU, conforme a figura a seguir. A API, em FastAPI [17], nunca acessa a GPU: valida e normaliza a foto (até 15 MB, orientação EXIF aplicada, metadados e GPS removidos), registra o job e responde `202` em milissegundos, com identificador, posição e estimativa de tempo. Os jobs ficam numa fila persistente em SQLite no modo WAL [18], com reserva atômica e ordem FIFO com prioridade por *preset* e envelhecimento (deslocamentos de 0, 30 e 90 s para os *presets* rápido, padrão e alto). Um processo de GPU carrega o modelo uma vez

e processa um job por vez (configuração de produção) ou dois *slots* com pesos compartilhados e travas por componente; aplica tempo limite, devolve à fila o job interrompido por reinício ou falta de memória e só aceita novo job com ao menos 10 GB livres no hospedeiro. Completam a arquitetura o cache por *hash* SHA-256 da imagem, das opções e da versão do pipeline, a coalescência de pedidos idênticos, os limites de fila (32 jobs, com resposta 429), de pedidos por cliente e de taxa, a retenção máxima de 7 dias e a implantação em Docker Compose com reinício automático e porta publicada apenas na interface de *loopback*.

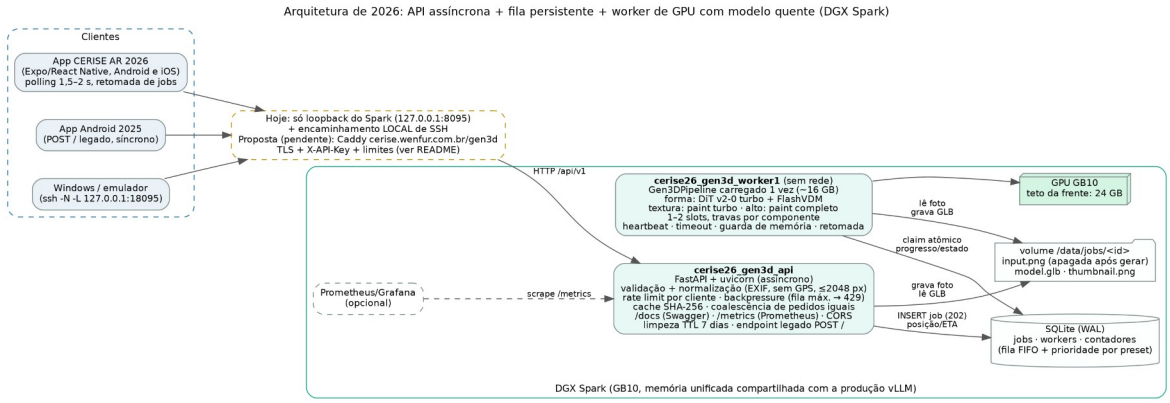


Figura 14. Arquitetura de 2026: API assíncrona, fila persistente em SQLite e processo de GPU com o modelo carregado; acesso restrito ao loopback da estação e a encaminhamento local por SSH

4.4.3. Testes de carga e resultados

Os testes foram executados em 26/10/2026 com os serviços de produção de IA ativos na mesma GPU, de modo que os valores refletem uma máquina compartilhada. O cliente de carga (asyncio e httpx) rodou no próprio hospedeiro, excluindo a latência de rede, e utilizou 64 imagens (renderizações do Google Scanned Objects e fotografias CC0) sem repetição dentro de cada cenário. O cenário principal consistiu em rajadas de 1, 2, 4, 8 e 16 clientes simultâneos, totalizando 33 jobs do *preset* rápido por configuração, com sondas de leitura paralelas para medir a responsividade.

Tabela 6. Resumo por configuração (33 jobs do preset rápido em rajadas de 1 a 16 clientes; latência do envio ao GLB baixado)

Configuração	Atendidos	Latência p50 / p95 (s)	p95 com 16 clientes (s)	Vazão com 16 clientes (jobs/min)	Leituras com GPU ocupada, p95 / p99 (ms)
2025, sem trava	3/33	12,3 / 12,5 (só 1 cliente)	sem sucesso	0	102,4 / 213,0
2025, com trava	33/33	52,1 / 199,0	210,2	4,37	123,4 / 228,6
2026, fila e 1 *worker*	33/33	49,6 / 174,1	184,1	4,99	6,6–6,7 / 8,3–8,6

2026, fila e 2 *slots*	33/33	46,5 / 155,6	164,4	5,63	6,6–6,7 / 9,0–9,2

A taxa de erro caiu de 90,9 % (30 de 33 pedidos na arquitetura de 2025 sem trava) para zero nas configurações novas. Com 16 clientes, a fila com 1 *worker* reduziu a latência p95 em 12 % e elevou a vazão em 14 % em relação à versão síncrona com trava; com 2 *slots*, os ganhos foram de 22 % e 29 %. A espera passou a ocorrer na fila, sem conexão presa, e as rotas de leitura responderam de 15 a 19 vezes mais rápido que o Flask no p95, inclusive sob *polling* de cerca de 50 req/s (9.569 sondas, sem erros).

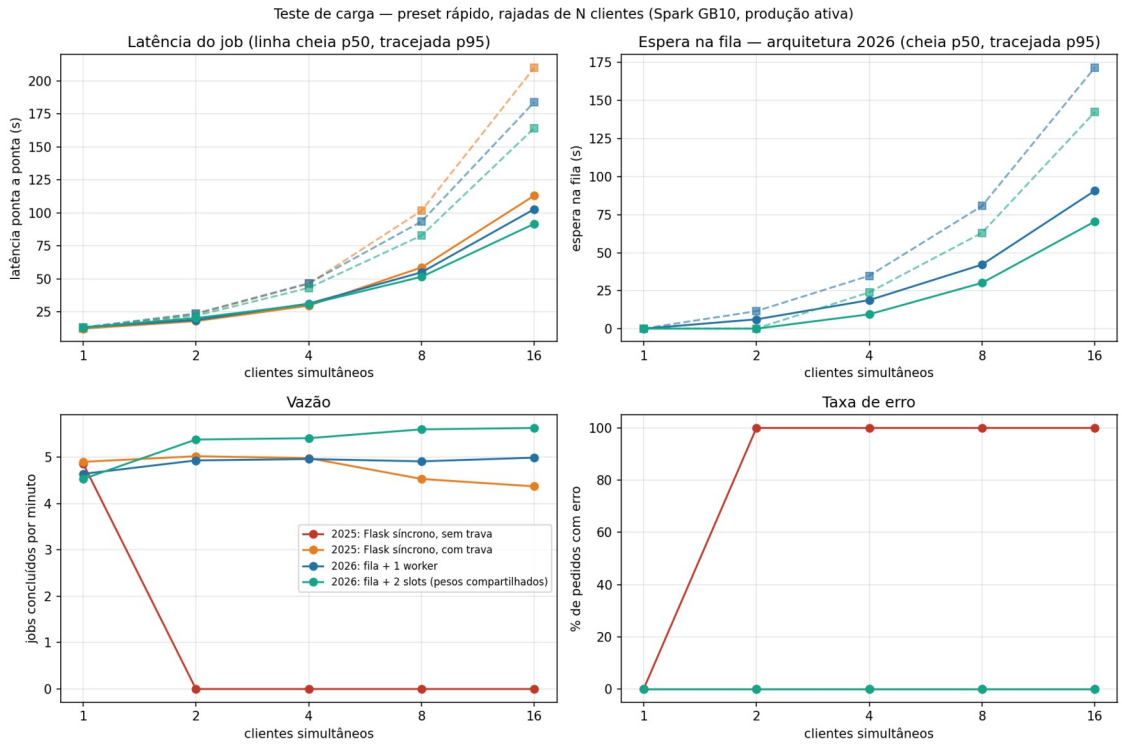


Figura 15. Latência ponta a ponta (p50 e p95), espera na fila, vazão e taxa de erro em função do número de clientes simultâneos para as quatro configurações

Tabela 7. Cenários adicionais na configuração de produção (1 worker)

Cenário	Pedidos	Atendidos	Execuções na GPU	Resultado
Reenvio de imagens já processadas (cache)	10	10	0	0,02 s por pedido
8 clientes com a mesma imagem nova (coalescência)	8	8	1	12,31 s (p50)
Misto com prioridade por *preset* (12 rápidos, 4 padrão)	16	16	16	p50: rápido 66,0 s; padrão 131,9 s

Misto em FIFO pura (mesmos pedidos)	16	16	16	p50: rápido 99,7 s; padrão 85,1 s
16 clientes com fila máxima de 8	16	9	9	7 recusados com 429 em 0,32 s

A prioridade reduziu em 34 % a latência mediana dos jobs rápidos, com média geral praticamente inalterada (86,0 s para 82,9 s); a espera máxima de um job padrão aumentou 29,9 s, dentro do envelhecimento de 30 s configurado, sem inanição. O limite de fila converteu esperas de até 192 s em recusas imediatas que o aplicativo pode exibir ao usuário.

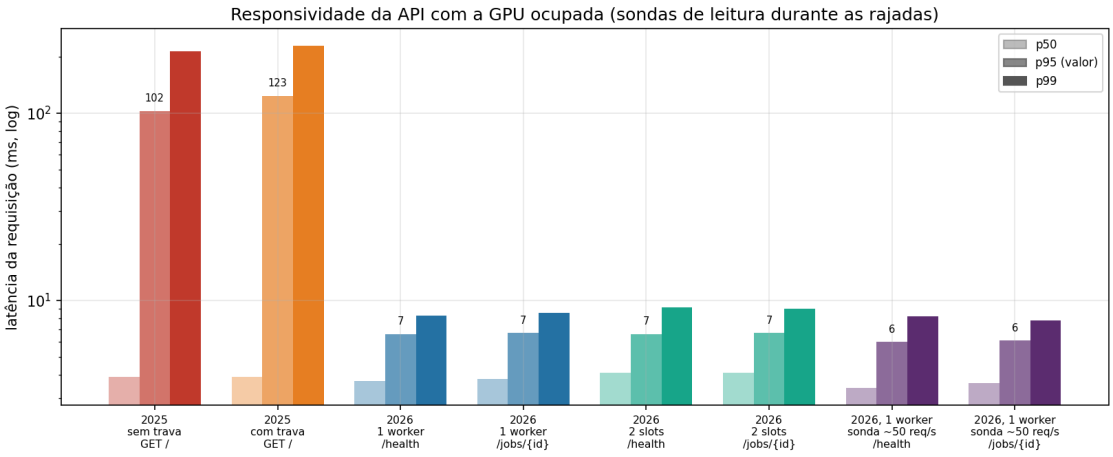


Figura 16. Responsividade das rotas de leitura com a GPU ocupada, em escala logarítmica: Flask de 2025 contra a API de 2026

4.4.4. Discussão e limitações

O gargalo dominante é a própria GPU: com 2 *slots*, a execução mediana de cada job subiu de 11,9 s para 21,0 s, enquanto a espera média nas travas foi de 0,05 a 0,2 s por componente, razão pela qual dois *slots* rendem só 13 % mais vazão que um. A capacidade sustentada medida é de cerca de 5 jobs/min no *preset* rápido; um objetivo de p95 de 1 min comporta rajadas de 4 a 5 pedidos (p95 de 46,7 s com 4 clientes e de 93,5 s com 8). O pico por job foi de 18,9 GiB alocados (22,9 GiB do processo), abaixo do teto de 24 GiB da frente. Os 18 testes automatizados do servidor, com motor simulado, foram aprovados, incluindo retomada após reinício, retenção e *backpressure*. Em dois desligamentos não planejados da estação, os contêineres retornaram automaticamente com a fila preservada; uma corrupção do cache de compilação do numba causada pelo primeiro foi corrigida passando esse cache para memória temporária.

Como limitações, as configurações foram medidas em sequência numa máquina compartilhada, sem controle da carga de fundo; o cenário misto não incluiu jobs do *preset* alto; a escala para várias GPUs ou máquinas foi apenas

projetada; o monitoramento sobre as métricas não foi ligado; e o servidor não foi exposto publicamente, sendo acessível apenas por encaminhamento local. Uma proposta de exposição, com opções e riscos, está documentada no repositório e aguarda decisão.

Código e evidências: repositório cerise-2026-gen3d, pasta server (privado)

4.5. Aplicativo CERISE AR com geração de objetos 3D texturizados

Esta etapa corresponde à Atividade 2 do Objetivo 1 do plano de trabalho, que prevê a integração do modelo de rede neural selecionado ao aplicativo e a entrega de uma versão funcional com suporte à geração de objetos 3D texturizados. O aplicativo CERISE AR, reescrito em 2026 em React Native com Expo [19] para Android e iOS, passou a permitir que o usuário fotografe um objeto, envie a imagem ao servidor descrito na subseção anterior, acompanhe a fila e o processamento, baixe o modelo GLB texturizado e o visualize em 3D e em realidade aumentada.

4.5.1. Implementação

Aplicativo e servidor foram desenvolvidos em paralelo sobre um contrato REST fixado previamente (`/api/v1`: verificação de estado, envio da foto com as opções, consulta do job, *download* do GLB e da miniatura e exclusão). Enquanto o servidor não estava disponível, um servidor simulado (*mock*) reproduziu o contrato, devolvendo GLBs de exemplo identificados na tela como tal. No aplicativo, destacam-se:

- cliente da API com envio multipart com progresso, tempo limite, cancelamento e mensagens de erro em português;
- gerenciador de jobs com *polling* (2 s na fila e 1,5 s em processamento), recuo exponencial em falhas de rede, *download* do GLB e da miniatura, medição do tempo de cada etapa, persistência local e retomada de jobs ao reabrir o aplicativo;
- leitura do cabeçalho do GLB para obter a caixa envolvente com as transformações da cena, usada para normalizar a escala na RA;
- visualizador 3D sem rede, com o `<model-viewer>` [20] embutido numa WebView;
- RA com ViroReact [21] (posicionamento por toque, escala e rotação), aberta somente após a confirmação de suporte do ARCore/ARKit, o que eliminou o encerramento do processo observado quando a sessão não podia ser criada.

No servidor, o motor de geração reporta o progresso por etapa e entrega o GLB no formato exigido pelo aplicativo: glTF 2.0 [22], apoiado em y = 0,

centralizado, maior dimensão de 0,30 m, textura JPEG e sem compressões Draco, meshopt ou KTX2; a rota legada de 2025 passou a devolver um modelo efetivamente gerado. Num teste a partir de um computador externo, uma foto CC0 no *preset* padrão produziu o GLB em 16,1 s (15,3 s de GPU), aprovado no Khronos glTF-Validator sem erros nem avisos. Esse teste revelou que a textura era gravada em PNG, defeito descrito na errata da subseção de otimização do modelo; corrigido o pós-processamento, o GLB caiu de 2,97 MB para 0,72 MB (−76 %) no *preset* padrão e de 2,60 MB para 0,44 MB (−83 %) na rota legada, que usa o *preset* rápido, sem alteração mensurável no tempo de geração.

4.5.2. Validação ponta a ponta com o servidor real

A integração foi validada no emulador Android 16 (API 36, x86_64 com tradução ARM), que alcançou o servidor por encaminhamento SSH local restrito ao *loopback*, sem exposição pública. Foram usadas imagens reais CC0 de objetos (pato de borracha, bule de prata e estatuetas de gato e de burro), escolhidas pelo seletor de fotos do sistema; não se trata de fotografias tiradas pela câmera de um celular. Para observar a fila, um segundo cliente enviou um job do *preset* alto imediatamente antes de um pedido do aplicativo, e a foto do pato foi reenviada para exercitar o cache. Os tempos foram registrados pelo aplicativo e conferidos com o estado dos jobs na API.

Tabela 8. Tempos medidos pelo aplicativo no emulador contra o servidor real (com textura e remoção de fundo; tamanho do GLB em KiB, como exibido no aplicativo)

Objeto e *preset*	Envio (s)	Fila: app / servidor (s)	Geração no app / execução no servidor (s)	Download (s)	Total (s)
Pato, rápido (340 KiB)	0,41	1,25 / 0,24	11,19 / 11,20	0,67	13,5
Burro, rápido (398 KiB)	0,49	1,35 / 0,20	11,24 / 11,26	0,45	13,5
Bule, padrão (622 KiB)	0,66	1,36 / 0,09	14,42 / 14,73	0,61	17,1
Gato, alto (1,44 MiB)	0,39	1,21 / 0,22	29,04 / 30,17	0,90	31,5
Gato, padrão, atrás de 1 job alto	0,40	28,25 / 27,85	15,81 / 15,88	0,46	44,9
Pato, rápido, reenvio (cache)	0,29	0,28 / 0	0 / 0	0,43	1,0

Sem concorrência, o tempo percebido variou de 13,5 s (rápido) a 31,5 s (alto), dominado pela execução na GPU (83 % a 96 % do total); envio e *download* somaram no máximo 1,3 s. A fila de 1,2 a 1,4 s registrada no aplicativo é artefato da resolução do *polling*, pois o servidor registrou de 0,09 a 0,24 s. Com um job alto à frente, a espera (27,85 s no servidor) praticamente igualou a

execução desse job (28,2 s), como esperado para um único processo de GPU. Todos os sete GLBs gerados nesse teste tinham textura JPEG, maior dimensão de 0,30 m e nenhuma extensão de compressão.

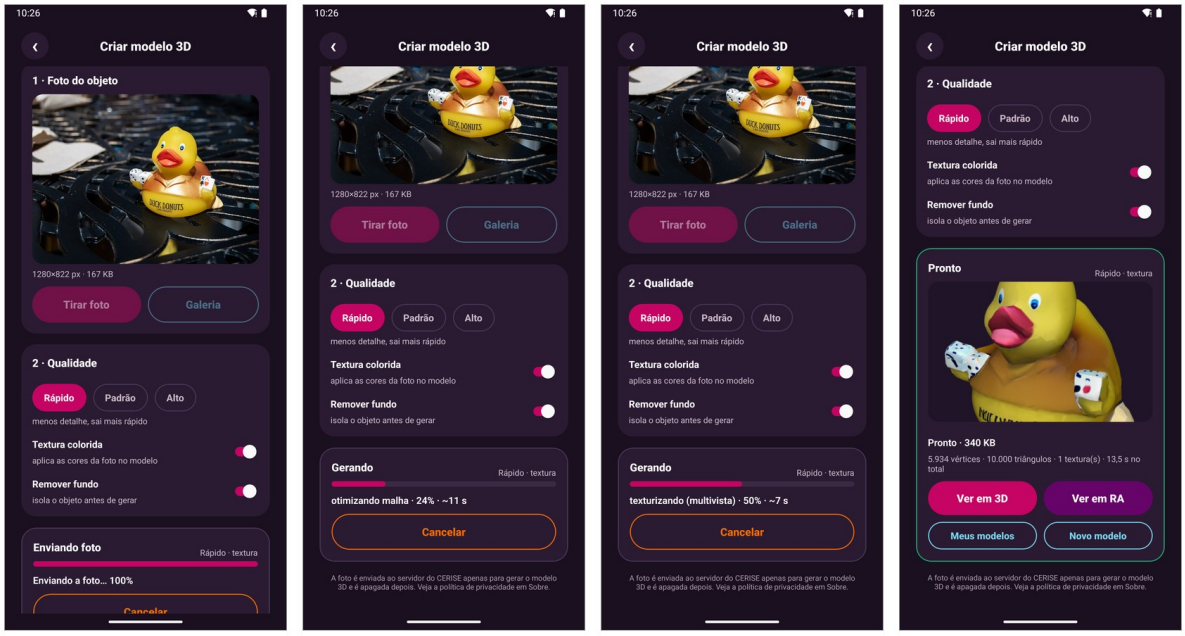


Figura 17. Fluxo de geração no Android com o servidor real: envio da foto, geração da malha, texturização multivista e resultado com o GLB texturizado baixado em 13,5 s no total

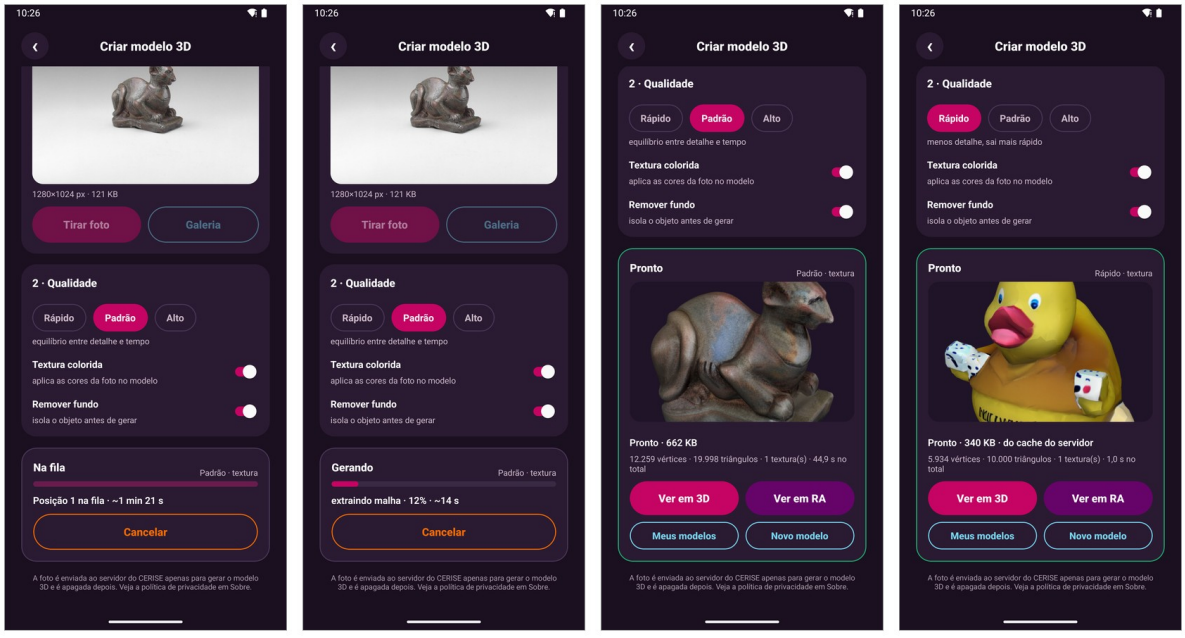


Figura 18. Fila e cache reais: pedido em posição 1 atrás de um job de outro cliente, geração após a fila, resultado em 44,9 s e reenvio da mesma foto atendido pelo cache do servidor em 1,0 s

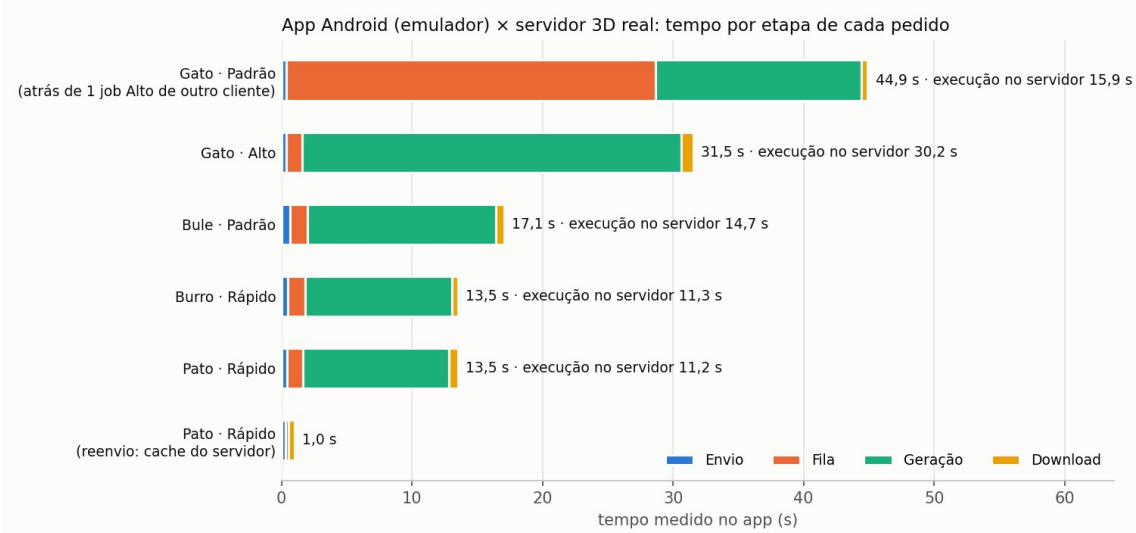
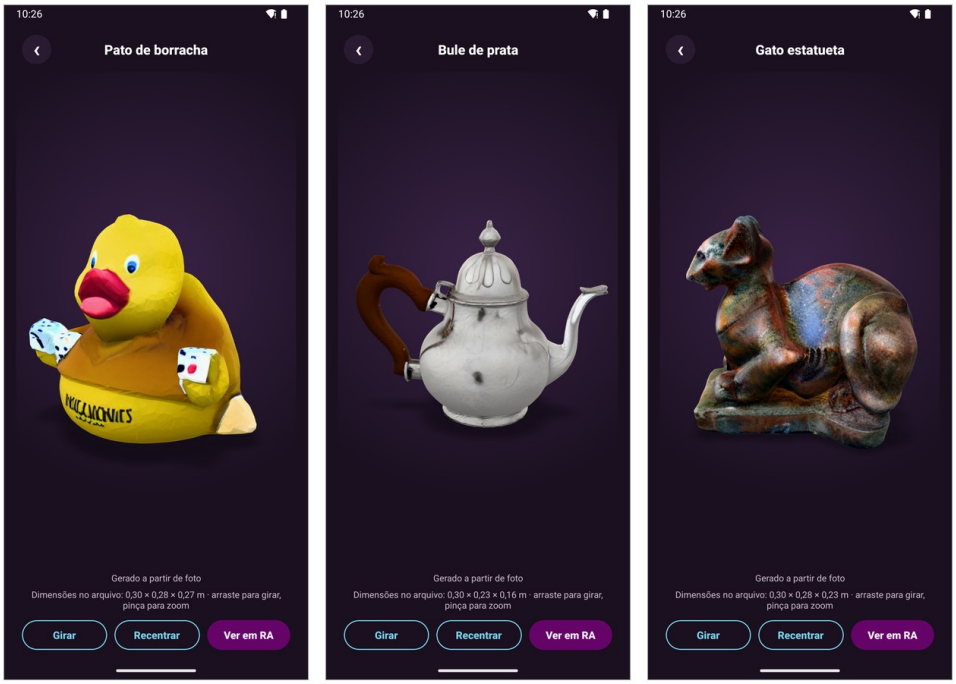


Figura 19. Tempo por etapa de cada pedido medido no aplicativo, com o tempo de execução informado pelo servidor

O teste com o servidor real motivou ajustes no aplicativo, todos em JavaScript. O *download* de um GLB de 398 KiB levava 5,0 s, embora o servidor respondesse em cerca de 1 ms; após deixar de gravar a lista de modelos a cada evento de progresso e baixar a miniatura em paralelo, o *download* caiu para 0,45 s e o total, de 18,0 s para 13,5 s (as mudanças foram aplicadas juntas, sem isolar a contribuição de cada uma). Passaram também a ser tratados os estados de servidor iniciando ou em pausa e as respostas do cache, e a exclusão de um modelo no aplicativo passou a apagar a cópia no servidor, verificado pela consulta do job, que passou de 200 para 404.

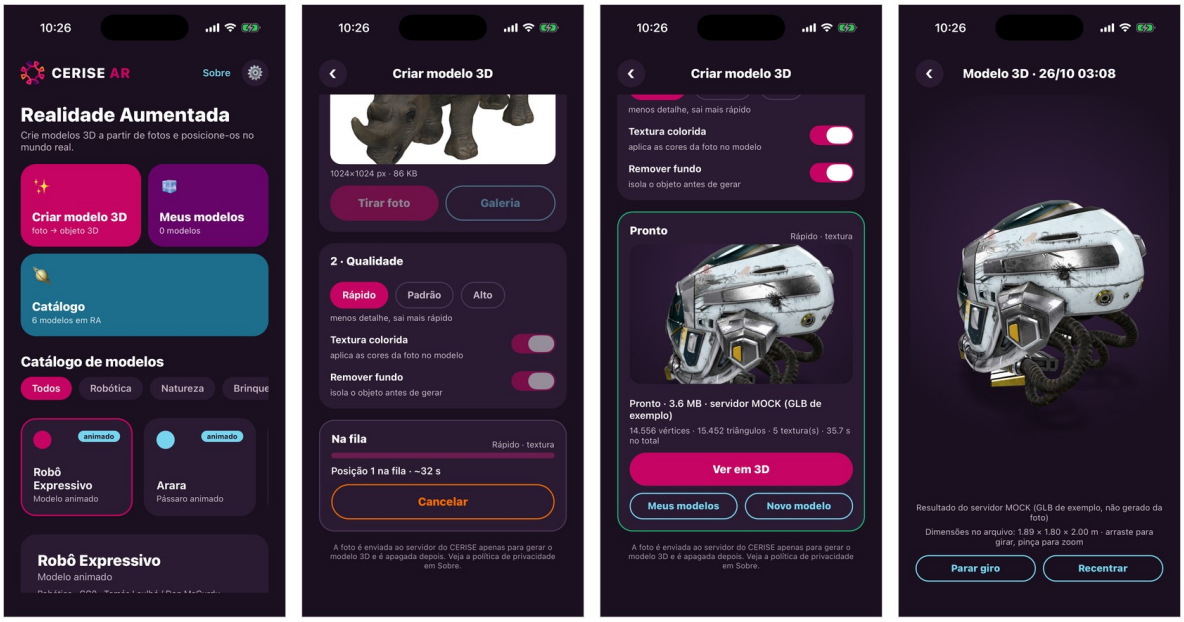


(a) Pato (Rápido) (b) Bule (Padrão) (c) Gato (Alto)

Figura 20. Visualizador 3D do aplicativo com modelos gerados pelo servidor a partir das fotos: pato no preset rápido, bule no padrão e gato no alto, com maior dimensão de 0,30 m lida do GLB

4.5.3. iOS

O *build* Release para o Simulador iOS foi gerado num Mac mini M4 (Xcode 26.6) em 72 s de compilação, após a instalação de 100 dependências nativas, resultando num pacote de 41 MB (iOS mínimo 17.6), executado no simulador do iPhone 17 Pro. Como a biblioteca nativa de RA não possui versão para simulador, esse *build* exclui o módulo de RA. O fluxo de geração, a lista de modelos e o visualizador 3D foram validados apenas contra o servidor simulado, como ilustrado abaixo; o modelo exibido é um GLB de exemplo, não gerado a partir da foto. O iOS não foi retestado contra o servidor real, pois o Mac utilizado não alcança a estação; as mudanças da integração são apenas de JavaScript e valem para as duas plataformas.



(a) Home (b) Na fila (c) Pronto (d) Visualizador 3D

Figura 21. Simulador iOS no iPhone 17 Pro: tela inicial, pedido na fila, resultado e visualizador 3D com o GLB de exemplo devolvido pelo servidor simulado

4.5.4. Limitações

A validação com o servidor real ocorreu apenas no emulador e pela rede interna, sem teste em aparelho físico, em rede móvel ou com fotografias da câmera. A RA dos modelos gerados não foi validada nesta versão, porque o ARCore não carrega no emulador x86_64 com bibliotecas ARM traduzidas e o simulador iOS não executa a biblioteca de RA. A estimativa de espera exibida na fila foi superestimada para o *preset* alto (1 min 21 s para 28 s reais), por depender de um valor inicial de 82 s ainda não recalibrado no servidor. Por fim, o uso fora da rede interna depende da exposição pública do servidor, ainda não decidida.

Código e evidências: repositório cerise-ar, branch feat/2026-gen3d-playstore, pull request em rascunho (privado)

4.6. Adequação do aplicativo para publicação na Play Store

Esta etapa corresponde à Atividade 6 do Objetivo 2 do plano de trabalho, que prevê a publicação do aplicativo na Play Store. O trabalho concentrou-se na adequação técnica e normativa do aplicativo e na preparação e verificação do pacote de publicação. A publicação propriamente dita não foi realizada: as etapas no Play Console exigem autenticação na conta de desenvolvedor, e dois pré-requisitos, a publicação da política de privacidade e a exposição pública do servidor de geração 3D, aguardam decisão.

4.6.1. Adequação técnica

A configuração nativa foi declarada nos arquivos de configuração do Expo e em *plugins* próprios, de modo que os projetos Android e iOS são gerados de forma reprodutível. A versão 1.1.0 (código de versão 110) declara apenas as permissões de câmera e de internet e bloqueia no manifesto as permissões acrescentadas por bibliotecas (gravação de áudio, armazenamento e mídia, localização, sobreposição de janelas, identificador de publicidade, vibração e NFC). A galeria usa o seletor de fotos do sistema, que dispensa permissão de mídia. As arquiteturas nativas foram restritas a `armeabi-v7a` e `arm64-v8a`, as únicas distribuídas pelo ViroReact; com as quatro arquiteturas padrão, o aplicativo encerrava ao iniciar em dispositivos x86_64. O tráfego sem criptografia foi proibido, exceto para endereços locais de teste, e os artefatos foram assinados com uma chave de *upload* dedicada (RSA 4096), mantida fora do controle de versão, para uso com o Play App Signing. Após a integração com o servidor real, o AAB e o APK foram recompilados e as verificações foram repetidas, com os resultados a seguir.

Tabela 9. Verificações no AAB e no APK release finais do CERISE AR 1.1.0

Requisito	Resultado medido
Nível de API	targetSdkVersion 36, compileSdk 36, minSdk 24
Páginas de memória de 16 KB [23]	30/30 bibliotecas de 64 bits alinhadas a 16 KB no AAB e no APK; zipalign -c -P 16 bem-sucedido
Permissões no APK	CAMERA, INTERNET, ACCESS_NETWORK_STATE e permissão interna do AndroidX
Componentes exportados	apenas a atividade principal e um receptor do AndroidX protegido por permissão
Assinatura	APK e AAB assinados com o certificado de upload do CERISE/EMC/UFG
Tamanho	AAB de 55,16 MB; download estimado de 35,65 MB (arm64-v8a) e 34,09 MB (armeabi-v7a)

A restrição de arquiteturas, além de eliminar a falha em x86_64, reduziu o AAB de 75,84 MB para 55,16 MB (−27 %) e o APK universal de 116,82 MB para 83,30 MB (−29 %), conforme a figura a seguir, cujos valores estão em MiB.

windows: verificacao do AAB/APK release (16 KB, permissoes, SDK)

```
$ python tools/check-16kb.py build-out/cerise-ar-1.1.0-110.aab build-out/cerise-ar-1.1.0-110.apk
cerise-ar-1.1.0-110.aab: 30/30 .so de 64 bits com PT_LOAD >= 16 KB
cerise-ar-1.1.0-110.apk: 30/30 .so de 64 bits com PT_LOAD >= 16 KB; offset no zip alinhado: 30/30

$ zipalign -c -P 16 -v 4 build-out/cerise-ar-1.1.0-110.apk | grep arm64-v8a
12845056 lib/arm64-v8a/libBulletCollision.so (OK)
13631488 lib/arm64-v8a/libBulletDynamics.so (OK)
14188544 lib/arm64-v8a/libBulletSoftBody.so (OK)
14467072 lib/arm64-v8a/libLinearMath.so (OK)
14614528 lib/arm64-v8a/libappmodules.so (OK)
14991360 lib/arm64-v8a/libarcore_sdk_c.so (OK)
15073280 lib/arm64-v8a/libarcore_sdk_jni.so (OK)
15187968 lib/arm64-v8a/libc++_shared.so (OK)
16482304 lib/arm64-v8a/libexpo-modules-core.so (OK)
17940480 lib/arm64-v8a/libexpo-updates.so (OK)
18055168 lib/arm64-v8a/libfbjni.so (OK)
18235392 lib/arm64-v8a/libfreetype.so (OK)
   ... (60 bibliotecas .so verificadas)
Verification successful

$ aapt2 dump badging build-out/cerise-ar-1.1.0-110.apk
package: name='com.cerisear' versionCode='110' versionName='1.1.0' platformBuildVersionName='16' platformBuildVersionCod
  e='36' compileSdkVersion='36' compileSdkVersionCodename='16'
minSdkVersion:'24'
targetSdkVersion:'36'
uses-permission: name='android.permission.CAMERA'
uses-permission: name='android.permission.CAMERA'
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.ACCESS_NETWORK_STATE'
uses-permission: name='com.cerisear.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION'
application-label:'CERISE AR'
  uses-feature: name='android.hardware.camera'
  uses-feature-not-required: name='android.hardware.camera.autofocus'
  uses-feature-not-required: name='android.hardware.sensor.accelerometer'
  uses-feature-not-required: name='android.hardware.sensor.gyroscope'
  uses-feature-not-required: name='android.hardware.vr.high_performance'
  uses-feature-not-required: name='android.software.vr.mode'
  uses-feature-not-required: name='oculus.software.eye_tracking'
  uses-feature: name='android.hardware.faketouch'
  uses-feature: name='android.hardware.screen.portrait'
native-code: 'arm64-v8a' 'armeabi-v7a'
```

Figura 22. Verificação do AAB e do APK release: alinhamento das bibliotecas nativas a páginas de 16 KB, nível de API alvo 36 e permissões finais

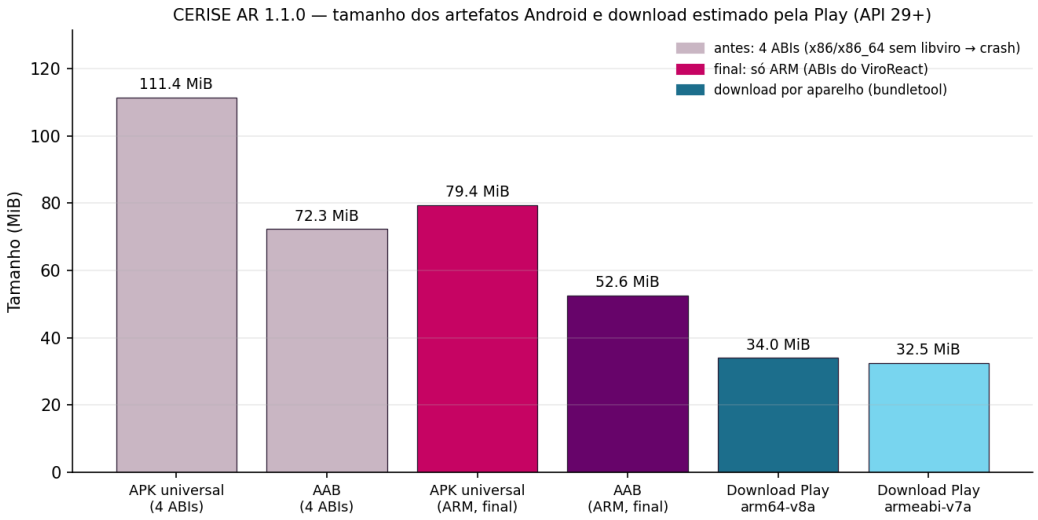


Figura 23. Tamanho dos artefatos Android antes e depois da restrição às arquiteturas ARM e tamanho de download estimado por aparelho, em MiB

4.6.2. Adequação normativa e pacote da loja

Os requisitos normativos e de conteúdo foram tratados da seguinte forma:

- política de privacidade em português, conforme a LGPD [24], descrevendo o uso da câmera e das fotos, o envio ao servidor do CERISE apenas para gerar o modelo, a exclusão em até 7 dias (ou

imediata quando o usuário exclui o modelo), a ausência de anúncios e de rastreamento e os direitos do titular;

- respostas preparadas para o formulário de segurança dos dados, o questionário de classificação etária IARC e a definição de público-alvo (13 anos ou mais);
- remoção do catálogo da versão de loja de modelos de personagens de franquias, em atenção à política de propriedade intelectual, e créditos das licenças dos demais modelos na tela Sobre;
- atualizações remotas restritas ao código JavaScript interpretado, sem *download* de código nativo;
- ficha da loja com limites de texto verificados automaticamente (nome com 26 de 30 caracteres, descrição curta com 69 de 80, descrição completa com 2.251 de 4.000 e notas da versão com 397 de 500), ícone 512×512, recurso gráfico 1024×500 e sete capturas de tela, três delas do teste com o servidor real;
- guia de publicação com as etapas de teste interno, teste fechado com 12 testadores por 14 dias (exigido de contas pessoais novas) e produção.



Figura 24. Pacote gráfico da Play Store: recurso gráfico 1024×500, ícone 512×512 e sete capturas de tela do emulador, das quais as de números 2, 3 e 7 provêm do teste com o servidor 3D real

4.6.3. Situação e pendências

O pacote de publicação está pronto e verificado, mas o aplicativo não foi publicado, razão pela qual a atividade é considerada parcialmente concluída. Permanecem pendentes: a criação do aplicativo no Play Console, o envio do AAB ao teste interno, a condução do teste fechado e o preenchimento dos formulários, que exigem autenticação na conta de desenvolvedor; a publicação da política de privacidade no endereço público previsto, que respondia com erro 404 em 26/10/2026; e a exposição pública do servidor de geração 3D, pois o endereço padrão configurado no aplicativo ainda não responde e a revisão de produção não deve ser solicitada antes disso. Caso a exposição adote chave de API, o aplicativo deverá ser recompilado com a chave. Também devem ser verificadas a disponibilidade do nome de pacote na loja e a cópia de segurança da chave de *upload*. No iOS, a compilação assinada para aparelho e o envio ao TestFlight foram documentados, mas não executados.

Código e evidências: repositório cerise-ar, pull request em rascunho com o pacote da loja (privado)

4.7. Geração e documentação do novo dataset do braço robótico

Esta etapa corresponde à Atividade 7 do Objetivo 3 do plano de trabalho: gerar, com um braço robótico virtual, um conjunto de dados estruturado e documentado para o treinamento e a avaliação do modelo de estimação de pose (Atividades 8 e 9) e para o gêmeo digital do braço (Atividades 11 e 12). Optou-se pelo braço virtual servo6dof, cujo pacote ROS 2 (descrição, controladores, configuração do MoveIt 2 e câmeras) foi fornecido pelo Prof. Dr. Alisson Assis Cardoso e adaptado para um estúdio fotográfico no simulador. O braço possui cinco juntas de revolução (`joint_1` a `joint_5`) e uma junta de garra (`joint_6`).

4.7.1. Metodologia de geração

As poses foram definidas por um plano fixo e reprodutível (`plano_poses.json`, semente 20260909) com 5.000 configurações sorteadas uniformemente em ±90° para as juntas do braço e em [−40°; 30°] para a garra, descontada uma folga de 5 % em cada extremo (faixa efetiva de ±81° nas juntas do braço). A simulação utilizou ROS 2 Humble [25] e Ignition Gazebo Fortress [26] em contêineres Docker na estação NVIDIA DGX Spark (GB10), com renderização ogre2 por GPU em modo *headless* (EGL). Para cada pose, o script de captura comanda a trajetória por `FollowJointTrajectory` (3 s de tempo simulado mais 0,5 s de assentamento), descarta os quadros antigos e grava seis imagens PNG de 1280×720 pixels — RGB, profundidade (inteiro, em milímetros) e mapa de rótulos de segmentação (braço, chão e fundo) — das câmeras frontal e diagonal, além de um JSON com as juntas comandadas e as juntas efetivamente alcançadas (`juntas_reais`, lidas de `/joint_states` no instante da

captura). A geração foi paralelizada em oito instâncias isoladas do Gazebo, com retomada automática a partir do que já existia em disco.

Sobre a saída foram implementadas ferramentas de indexação (`index.csv`, uma linha por pose), verificação de integridade com decodificação integral de cada imagem, estatísticas de distribuição e cobertura do espaço de juntas, comparação entre juntas reais e comandadas e reconstrução da linha do tempo da geração. A documentação segue o formato de *datasheet* proposto por Gebru et al. [27] (`docs/DATASET.md`), com motivação, composição, processo de coleta, formato, *splits* recomendados, usos e limitações; uma amostra de 8 poses (4,2 MB) está versionada no repositório. O datasheet recomenda um *split* determinístico por identificador (`id` mod 10), com 4.000 poses de treino, 500 de validação e 500 de teste; as avaliações das subseções seguintes adotaram, nas mesmas proporções, a divisão aleatória por pose definida na etapa de estimação de pose.

4.7.2. Resultados

A geração foi concluída com as 5.000 poses do plano, totalizando 30.000 imagens e 2,51 GB, conforme a composição apresentada a seguir. As anotações JSON somam 7,7 MB, e o volume médio é de 0,50 MB por pose.

Tabela 10. Composição do dataset por modalidade e câmera (5.000 poses; tamanhos medidos em disco)

Modalidade	Câmera frontal	Câmera diagonal	Total
RGB (PNG 8 bits)	0,60 GB (116 KB/img.)	0,50 GB (98 KB/img.)	1,10 GB
Profundidade (PNG inteiro, mm)	0,24 GB (47 KB/img.)	1,05 GB (206 KB/img.)	1,30 GB
Segmentação (rótulos 0/1/2)	0,05 GB (10 KB/img.)	0,05 GB (11 KB/img.)	0,11 GB

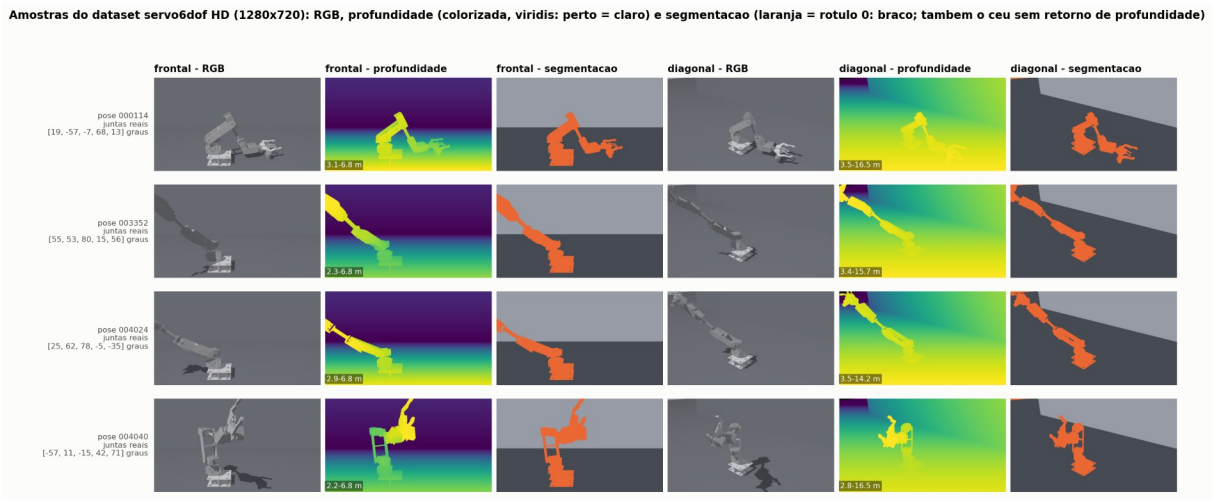


Figura 25. Amostras do dataset nas duas câmeras: imagem RGB, profundidade colorizada e mapa de segmentação, com o rótulo do braço destacado em laranja

A verificação completa decodificou as 30.000 imagens em 86 s (6 processos) sem nenhum problema: todas têm 1280×720 pixels e o modo de cor esperado, as 10.000 máscaras contêm exatamente os rótulos {0, 1, 2}, não há arquivos órfãos nem poses faltantes e não há quadro RGB duplicado entre poses distintas, o que indica ausência de captura de quadros desatualizados. Os principais indicadores estão reunidos na tabela a seguir, e o registro do terminal dessa verificação é mostrado na figura subsequente.

Tabela 11. Indicadores de integridade, distribuição e geração do dataset

Indicador	Valor medido
Imagens verificadas sem problema	30.000 de 30.000
PNGs órfãos / faltantes / RGB duplicados	0 / 0 / 0
Área média ocupada pelo braço (frontal / diagonal)	8,8 % / 6,0 % da imagem
Desvio-padrão dos ângulos comandados (juntas 1 a 5)	45,9° a 46,7° (uniforme: 46,8°)
Cobertura dos pares de juntas (grade 20×20)	100 % nos 10 pares
Maior correlação de Pearson entre juntas (em módulo)	0,024
Poses com juntas reais diferentes das comandadas (> 0,01 rad)	34,1 %
Duração da geração (8 instâncias) e vazão média	3,64 h; 0,38 pose/s

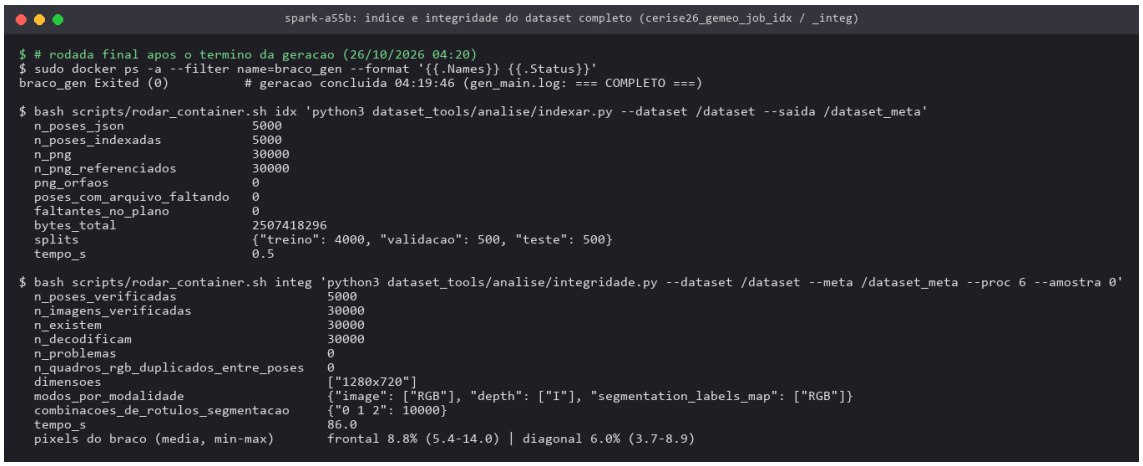


Figura 26. Registro do terminal com a indexação e a verificação de integridade das 30.000 imagens do dataset completo

Os ângulos comandados ocupam exatamente a faixa do plano, com média próxima de zero e dispersão compatível com a distribuição uniforme. A distância média ao vizinho mais próximo no espaço de juntas normalizado (0,1265) é praticamente igual à de uma amostra uniforme de mesmo tamanho (0,1262), o que confirma a cobertura homogênea ilustrada abaixo.

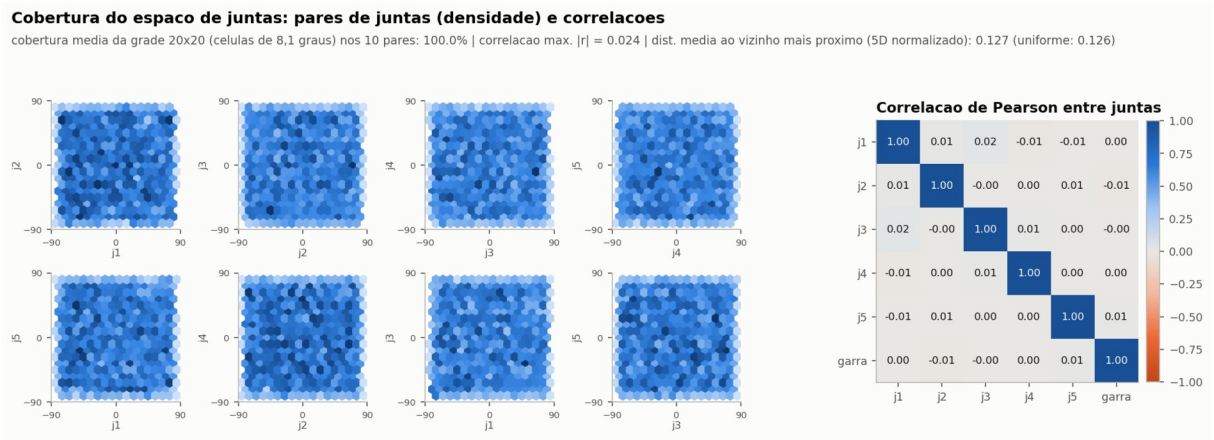


Figura 27. Cobertura do espaço de juntas por pares de juntas e matriz de correlação de Pearson entre as juntas

A comparação entre juntas reais e comandadas mostrou divergência superior a 0,01 rad em 34,1 % das poses, concentrada em `joint_1` a `joint_3` (19,8 %, 26,9 % e 21,9 % das poses). A cinemática direta do modelo digital explica a maior parte dos casos: 19,7 % das configurações comandadas levariam algum elo abaixo do plano do chão, e todas elas divergem, enquanto nenhuma configuração real fica abaixo de −5 cm — o braço é barrado pelo chão e o JSON registra a posição em que ele parou. As imagens confirmam que o rótulo correto é a junta real: nas 670 imagens de poses divergentes avaliadas, o IoU médio entre a silhueta do modelo e a máscara do simulador foi de 0,930 com as juntas reais e de 0,547 com as comandadas.

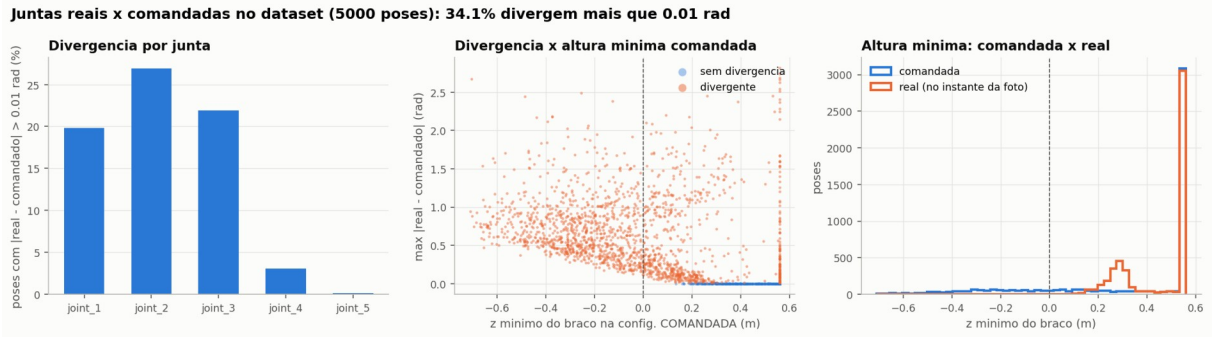


Figura 28. Divergência entre juntas reais e comandadas por junta e em função da altura mínima do braço na configuração comandada

A geração ocorreu em 26/10/2026, da primeira pose às 00h41 até a última às 04h19 (3,64 h), com mediana de 12,8 s por pose em cada instância e pico de 45 poses/min em janelas de 5 minutos; a mediana dessas janelas foi de 17,5 poses/min, pois a vazão caiu quando a estação passou a ser compartilhada com outras tarefas. Uma primeira rodada gravou 4.747 poses, e uma segunda rodada completou as 253 restantes; cada instância terminou com 625 poses.

4.7.3. Discussão e limitações

O dataset é íntegro, uniforme e documentado, mas apresenta limitações que orientam o seu uso. As juntas comandadas não são verdade-base em 34,1 % das poses, de modo que o campo `juntas_reais` deve ser adotado como rótulo; além disso, cerca de 2 % das imagens (estimativa sobre 1.000 poses) apresentam silhueta incompatível com as juntas registradas e devem ser filtradas por IoU. Na segmentação, o braço recebe o rótulo 0, compartilhado com regiões sem objeto, e a máscara do braço deve ser obtida como rótulo 0 com profundidade válida. O Gazebo Fortress não aplica as juntas *mimic* da garra, cuja geometria nas imagens difere parcialmente do modelo cinemático. Por fim, o domínio é sintético e fixo (duas câmeras, iluminação, fundo e materiais constantes), e a transferência para o braço físico exigirá randomização de domínio [28] ou ajuste fino com imagens reais.

Todos os resultados desta subseção e das subseções seguintes sobre o braço robótico referem-se à versão do dataset com o braço em cinza (5.000 poses, 2,51 GB), preservada integralmente na estação DGX Spark. Posteriormente, foi iniciada uma nova renderização do dataset com o braço na cor laranja, ainda em andamento na data deste relatório e sem resultados aqui reportados.

Código e evidências: repositório cerise-2026-servo6dof-gemeo-digital (privado)

4.8. Aplicação do modelo de estimação de pose de 2025 ao novo dataset

Esta etapa corresponde à Atividade 8 do Objetivo 3, que previa obter resultados quantitativos da execução do modelo de detecção de pose do ano anterior sobre o novo conjunto de dados da Atividade 7. Esse conjunto, descrito na subseção anterior, reúne 5.000 poses do braço servo6dof, de cinco juntas rotacionais e garra, observado em simulação por duas câmeras fixas (frontal e diagonal), com imagens RGB 1280×720 e os ângulos reais das juntas. O modelo de 2025 estima a pose do braço a partir de uma única imagem RGB, sem acesso aos estados das juntas, e segue a arquitetura HPE (*Real-time Holistic Robot Pose Estimation with Unknown States*) [29], acrescida de um ramo de profundidade monocular baseado no Depth Anything V2 [30]. No relatório de dezembro de 2025, esse modelo atingiu 75,4 % de acurácia ADD com limiar de 60 mm no robô Panda, no conjunto real DREAM com câmera RealSense [31].

4.8.1. Metodologia e implementação

O experimento final de 2025 foi identificado nos registros de treino (`MyNet/experiments/panda/realsense_full`): os 75,4 % correspondem ao registro da época 355, com AUC-ADD de 0,547. Como o script de 2025 guardava apenas o melhor AUC e o último estado, os pesos dessa época não existem; a

transferência partiu do estado final salvo do mesmo treino (`checkpoint.pk`, 111,2 milhões de parâmetros). A arquitetura combina uma rede de profundidade da raiz (DepthNet, HRNet-W32 [32]) fundida com a profundidade relativa do Depth Anything V2, um ramo integral com mapas de calor 3D (ResNet-50 [33]), a regressão iterativa dos ângulos e da rotação da raiz e uma cinemática direta diferenciável.

O código, escrito para o Google Colab, foi portado para um contêiner derivado da imagem PyTorch da NVIDIA na estação NVIDIA DGX Spark (GB10, aarch64), sem reimplementação e com compatibilidade de pesos. Foram corrigidos dois defeitos do código de 2025: (i) a profundidade da raiz era calculada depois da camada integral, de modo que os keypoints desse ramo não a incluíam, o que explica as métricas zeradas do ramo integral nos registros de 2025; (ii) o laço de treino chamava `optimizer.step()` duas vezes por iteração. A adaptação ao servo6dof envolveu:

- cinemática direta diferenciável da cadeia do servo6dof, obtida do URDF gerado por `xacro`, no lugar da cinemática do Panda;
- 7 keypoints (base, origens das cinco juntas e centro da garra), o mesmo número do Panda, o que permitiu reaproveitar a camada final dos mapas de calor; cinco juntas estimadas (a abertura da garra não é estimada);
- faixa de profundidade dos mapas de calor 3D ampliada de 1,3 m para 3,0 m, pois o braço tem cerca de 2,0 m de alcance, aproximadamente 2,4 vezes o do Panda.

A geometria das câmeras, lida do mundo da simulação, foi verificada projetando sobre as imagens os keypoints calculados por cinemática direta com os ângulos reais, conforme ilustrado abaixo: numa amostra de 300 poses, 99,9 % (frontal) e 99,6 % (diagonal) dos keypoints dentro do quadro recaem sobre a máscara de segmentação do robô.

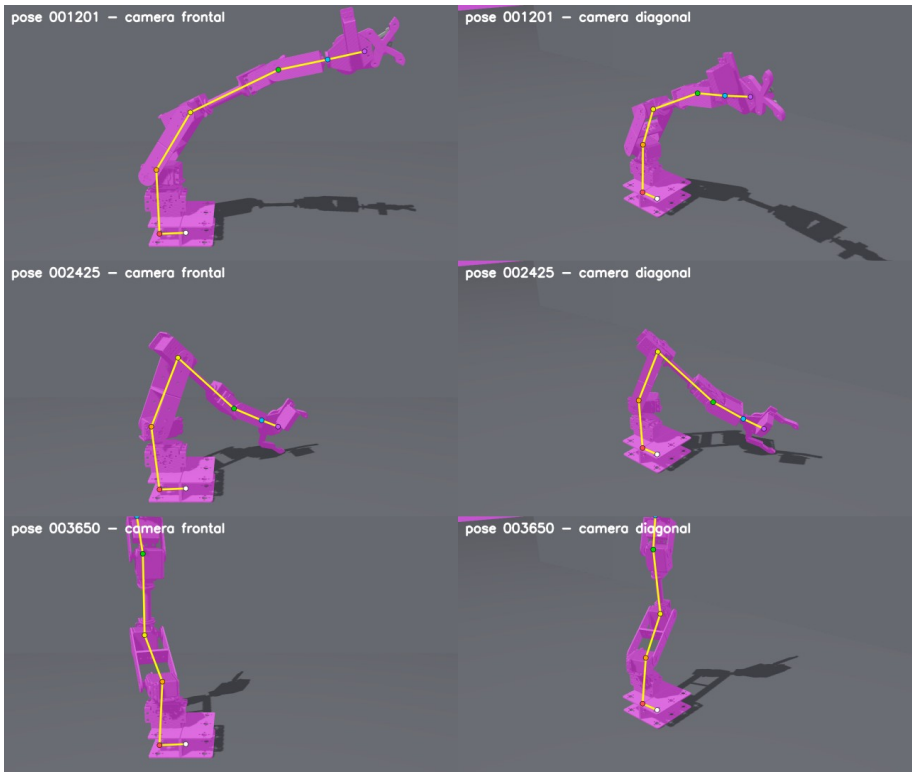


Figura 29. Verificação da geometria: keypoints e esqueleto obtidos por cinemática direta do URDF com os ângulos reais, projetados com as poses e intrínsecos das câmeras sobre imagens do dataset; em magenta, a máscara de segmentação do robô

A divisão treino/validação/teste foi feita por pose, de forma aleatória (semente 2026), na proporção 80/10/10 (4.000/500/500 poses, ou 8.000/1.000/1.000 imagens), mantendo as duas câmeras de uma pose no mesmo subconjunto. Em vez do recorte pela caixa dos keypoints verdadeiros do protocolo DREAM de 2025, adotou-se um recorte fixo por câmera, calculado só com o treino e redimensionado para 256×256, de modo que teste e inferência não dependem do gabarito.

Foram aplicadas as etapas da metodologia de 2025 compatíveis com dados sintéticos rotulados. Na etapa 1, a DepthNet foi treinada por 5 épocas sobre as 3.151 poses de treino já disponíveis naquele momento (subconjunto do split final), atingindo, com os pesos de 2025, erro de profundidade da raiz de 71 mm na validação. Na etapa 2, o modelo completo foi treinado com taxa de aprendizado de 1×10^{-4}, multiplicada por 0,9 a cada época, lote 32 e as augmentations fotométricas de 2025. A etapa 3 de 2025, autossupervisionada com imagens reais, não se aplica, pois ainda não há imagens reais do braço. Foram comparadas duas inicializações: a **transferência**, em que 98,1 % dos parâmetros (109,1 M de 111,2 M) vieram do modelo de 2025 e apenas as camadas do regressor que dependem do número de juntas (8 para 5) foram reinicializadas; e um **baseline** com backbones pré-treinados no ImageNet. A transferência recebeu 20 épocas (cerca de 62 min); o baseline, re-executado

isoladamente depois que uma tentativa em paralelo foi abortada por falta de memória na estação (compartilhada com serviços em produção), parou pelo limite de tempo após 12 épocas (cerca de 52 min).

A figura a seguir mostra as curvas por época. Com os pesos de 2025, o erro 3D na validação caiu de 262 mm (época 1) para 54,1 mm (época 20). Na época 12, a transferência tinha erro 3D de validação de 65,0 mm contra 83,7 mm do baseline e acurácia@60 mm de 59,1 % contra 34,9 %, com MAE das juntas semelhante (13,1° contra 12,8°).

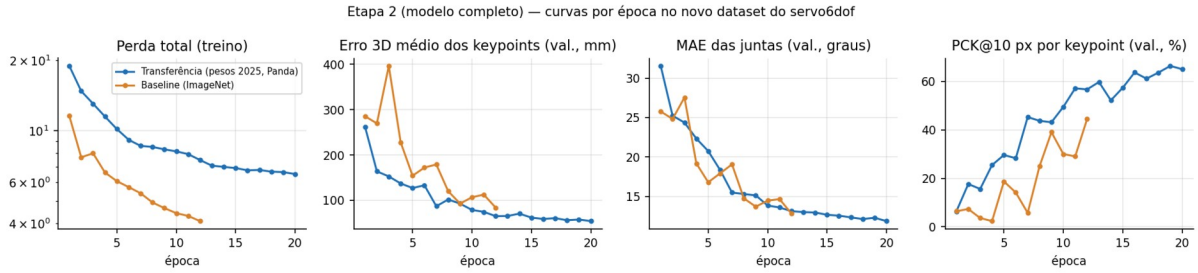


Figura 30. Etapa 2: perda total de treino e métricas de validação por época (erro 3D, MAE das juntas e PCK a 10 px) para a transferência dos pesos de 2025 e para o baseline ImageNet

```
spark-a55b: treino etapa 2 (modelo completo, transferencia dos pesos de 2025)

$ docker run cerise26_pose python scripts/treinar.py --dados /dados/final --etapa full --init 2025 --rootnet /runs/etapa
  1_depthnet_transfer_v2/melhor.pt --epocas 20
=== 2026-10-26 07:25:55  etapa=full init=2025  args={'dados': '/dados/final', 'saida': '/runs/final_full_transfer', 'eta
  pa': 'full', 'init': '2025', 'ckpt_2025': '/workspace/pose/pesos/realsense_full_checkpoint.pk', 'rootnet': '/runs/etap
  a1_depthnet_transfer_v2/melhor.pt', 'epocas': 20, 'bs': 32, 'lr': 0.0001, 'decaimento': 0.9, 'inicio_decaimento': 3, '
  cams_treino': ['frontal', 'diagonal'], 'cams_val': ['frontal', 'diagonal'], 'sem_aug': False, 'sem_amp': False, 'max_i
  ters': 0, 'max_val': 0, 'workers': 4, 'tempo_max_min': 70.0}
treino: 8000 imagens | val: 1000 imagens
[transfer] 2361/2366 tensores (98.1% dos parametros) carregados; forma diferente: ['fc_pose_1.weight', 'decpose.weight',
   'decpose.bias']
[transfer] 1999/2366 tensores (51.3% dos parametros) carregados; forma diferente: []
[rootnet etapa 1] 1999 tensores do DepthNet carregados
precisao mista bf16 + channels_last ativados nos backbones
parametros treinaveis: 111.07 M
...
EPOCA {"epoca": 20, "treino_pose": 0.17048, "treino_rot": 0.01488, "treino_depth": 0.02875, "treino_uv": 0.01016, "train
  o_trans": 0.04713, "treino_kp3d": 0.08054, "treino_kp2d": 0.01816, "treino_kp3d_int": 0.48201, "treino_kp2d_int": 0.01
  753, "treino_align": 0.47722, "treino_total": 6.51264, "val_AUC_ADD": 0.5028, "val_ADD_60mm": 0.734, "val_erro3d_mm":
  54.13748, "val_erro2d_px": 10.85454, "val_MAE_graus": 11.85523, "val_PCK_kp_10px": 0.65106, "lr": 2e-05, "tempo_treino
  _s": 182.0, "tempo_total_s": 187.2}
  -> novo melhor (epoca 20)
=== fim: 08:28:06  total 61.8 min

$ Import-Csv runs/final_full_transfer/curvas.csv | Format-Table  (colunas selecionadas)

epoca perda_treino val_erro3d_mm val_MAE_graus val_ADD60_% val_AUC_ADD val_PCK10px_% lr        t_epoca_s
----- ------------ ------------- ------------- ------------ ----------- ------------- --        ---------
1            18,92        262,10         31,55         0,10        0,00          6,30 1,00E-004 379.3
2            14,76        163,80         25,23         2,70        0,04         17,60 1,00E-004 173.6
3            13,03        152,60         24,36         1,40        0,03         15,60 9,00E-005 137.1
4            11,48        137,20         22,33         4,50        0,07         25,50 8,10E-005 165.9
5            10,15        127,10         20,74         5,70        0,09         29,60 7,29E-005 168.6
6             9,13        133,10         18,34         6,00        0,08         28,30 6,56E-005 160.3
7             8,60         87,00         15,48        35,50        0,28         45,30 5,90E-005 156.7
8             8,51        101,80         15,29        23,30        0,20         43,70 5,31E-005 159.8
9             8,31         92,70         15,12        30,90        0,24         43,20 4,78E-005 159.7
10            8,14         78,80         13,83        43,70        0,33         49,50 4,30E-005 147.0
11            7,93         74,50         13,59        46,80        0,35         57,20 3,87E-005 166.5
12            7,48         65,00         13,11        59,10        0,42         56,70 3,49E-005 180.3
13            7,08         65,40         13,00        58,20        0,42         59,80 3,14E-005 164.9
14            6,99         70,60         12,95        48,60        0,37         52,20 2,82E-005 129.6
15            6,91         61,80         12,66        62,90        0,45         57,40 2,54E-005 203.0
16            6,76         58,90         12,54        65,00        0,46         63,70 2,29E-005 236.2
17            6,78         60,60         12,33        63,80        0,45         61,20 2,06E-005 217.2
18            6,67         56,10         12,11        69,00        0,49         63,70 1,85E-005 199.4
19            6,64         57,60         12,28        65,50        0,47         66,40 1,67E-005 195.2
20            6,51         54,10         11,86        73,40        0,50         65,10 1,50E-005 187.2
```

Figura 31. Saída de terminal do treino da etapa 2 (transferência) na estação NVIDIA DGX Spark, com a fração de parâmetros carregados do modelo de 2025 e as métricas de validação por época

4.8.2. Resultados no conjunto de teste

A tabela a seguir resume as métricas no teste (500 poses × 2 câmeras), não usado no treino nem na escolha do checkpoint, feita pela validação.

Tabela 12. Resultados no conjunto de teste do servo6dof (1.000 imagens)

Métrica	Transferência (pesos 2025)	Baseline (ImageNet)
MAE médio das 5 juntas (°)	12,68	13,51
Erro 3D médio dos keypoints (mm)	58,7	88,3
Erro 3D mediano dos keypoints (mm)	41,1	65,7
AUC-ADD 0–100 mm	0,471	0,265
Acurácia ADD@60 mm (%)	68,0	32,8
Acurácia ADD@100 mm (%)	91,2	69,7
Acurácia ADD@150 mm (%)	96,7	90,2
Erro 2D médio (px, imagem 1280×720)	12,0	17,7
PCK@10 px por keypoint (%)	61,7	43,6
Épocas de treino	20	12

O modelo de 2025 adaptado ao servo6dof atingiu MAE de 12,68° nas cinco juntas, erro 3D médio de 58,7 mm (mediana de 41,1 mm) e acurácia@60 mm de 68,0 %. Em relação ao baseline, o erro 3D médio caiu cerca de 34 % e a acurácia@60 mm mais que dobrou, enquanto o MAE melhorou menos (13,51° para 12,68°). Como o baseline teve menos épocas, a comparação de mesmo orçamento é a da validação na época 12, que confirma a vantagem da transferência na precisão 3D. Exemplos de predições no teste são apresentados abaixo.

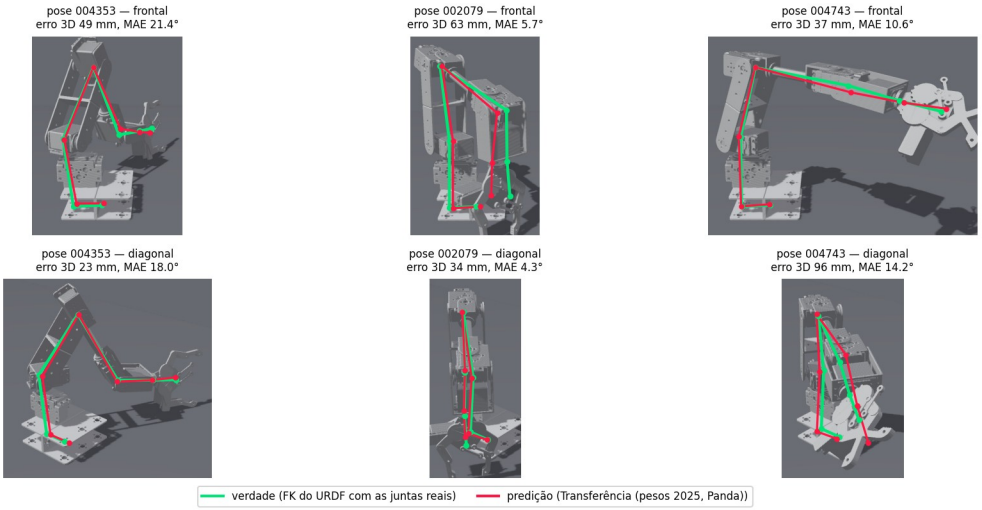


Figura 32. Predições do modelo de transferência (vermelho) e verdade obtida por cinemática direta com as juntas reais (verde) sobre imagens do teste nas duas câmeras, para poses nos percentis 25, 50 e 75 do erro 3D

A tabela seguinte coloca lado a lado os resultados de 2025 e de 2026, que **não são diretamente comparáveis**: robôs, escala, câmeras e dados

diferem. Como o servo6dof tem cerca de 2,4 vezes o alcance do Panda, 60 mm correspondem a aproximadamente 3 % do alcance, contra cerca de 7 % no Panda; as câmeras ficam a cerca de 3,6 m (frontal) e 4,4 m (diagonal) da base; os dados de 2026 são sintéticos, o recorte não usa o gabarito, não houve etapa autossupervisionada e o MAE se refere a conjuntos de juntas distintos. Com limiar proporcional ao tamanho do robô (cerca de 140 mm), a acurácia@150 mm é de 96,7 %.

Tabela 13. Acurácia@60 mm em 2025 (Panda) e 2026 (servo6dof); robôs, escalas, câmeras e dados diferentes

Modelo	Robô / dados	ADD@60 mm (%)	AUC-ADD	MAE (°)
RoboPose [34]	Panda / DREAM real	74,3	—	—
HPRE	Panda / DREAM real	75,2	—	—
Modelo de 2025 (HPE + DA-V2)	Panda / DREAM real	75,4	0,547	11,58
Modelo de 2025 com transferência	servo6dof / sintético	68,0	0,471	12,68
Mesma arquitetura, ImageNet	servo6dof / sintético	32,8	0,265	13,51

Os valores de RoboPose e HPRE foram transcritos do relatório de 2025 e não puderam ser reconferidos em arquivo de resultados nesta etapa; os do modelo de 2025 foram conferidos no arquivo de resultados do treino de 2025. As principais limitações desta etapa são: dados exclusivamente sintéticos; uso do estado final do treino de 2025 em vez dos pesos da época de 75,4 %; baseline com menos épocas que a transferência; etapa 1 treinada num subconjunto (o modelo final e as avaliações usaram o dataset completo); e orçamento de treino de cerca de uma hora em GPU compartilhada.

Código e evidências: repositório cerise-2026-pose-servo6dof (privado)

4.9. Avaliação de precisão e robustez do modelo de pose

Esta etapa corresponde à Atividade 9 do Objetivo 3, que previa avaliar o modelo de detecção de pose quanto à precisão e à robustez no novo cenário. Avaliou-se o modelo de 2025 adaptado ao servo6dof por transferência (Atividade 8), comparando-o, quando pertinente, ao baseline com inicialização ImageNet. Todas as medições foram feitas sobre os dados sintéticos do servo6dof.

4.9.1. Protocolo de avaliação

A avaliação usou o conjunto de teste (1.000 imagens), nunca visto no treino nem usado na seleção do checkpoint, e compreendeu:

- erro absoluto médio (MAE) por junta, em graus, entre os ângulos estimados e os reais;
- erro 3D dos keypoints no referencial da câmera (média e mediana) e acurácia ADD, isto é, a fração de imagens cujo erro 3D médio dos 7 keypoints é menor ou igual a um limiar, com AUC-ADD entre 0 e 100 mm calculado com a mesma implementação do código de 2025;
- PCK 2D a 5, 10 e 20 px na imagem original 1280×720;
- generalização entre câmeras, com um modelo treinado apenas com a câmera frontal e testado nas duas;
- robustez a sete perturbações da imagem de entrada, em três ou quatro severidades cada, num subconjunto fixo de 400 imagens do teste, com a profundidade do Depth Anything V2 recalculada sobre a imagem perturbada (pipeline completo); exemplos das perturbações são mostrados na figura a seguir;
- latência de inferência com lote 1 na GPU GB10, compartilhada com outros serviços.

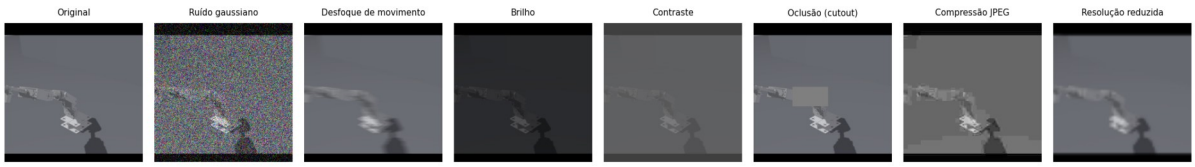


Figura 33. Exemplos das perturbações aplicadas ao recorte de entrada de 256×256 px nos testes de robustez: ruído gaussiano, desfoque de movimento, brilho, contraste, oclusão, compressão JPEG e resolução reduzida

4.9.2. Precisão por junta, por keypoint e por câmera

Tabela 14. Erro por junta no conjunto de teste

Junta	Transferência: MAE (°)	Transferência: mediana (°)	Baseline: MAE (°)
joint_1 (base)	4,19	2,94	7,20
joint_2 (ombro)	3,04	1,89	3,87
joint_3 (cotovelo)	3,91	2,54	5,41
joint_4 (punho)	10,52	6,64	9,25
joint_5 (rotação da garra)	41,73	42,49	41,83

As juntas 1 a 3 (base, ombro e cotovelo) são estimadas com MAE entre 3,0° e 4,2° e a junta 4 (punho) com 10,5°. A junta 5, rotação da garra em torno do eixo do antebraço, apresenta MAE de 41,7° nos dois modelos, valor compatível com um preditor que não extrai informação dessa junta: o keypoint da garra está praticamente sobre o eixo de rotação dela, de modo que sua variação quase não altera os keypoints e as perdas geométricas não a supervisionam. Excluída a junta 5, o MAE médio das juntas 1 a 4 é de 5,4°. O

erro 3D por keypoint é mínimo no keypoint de referência (cotovelo, 27 mm) e cresce em direção à extremidade da cadeia, atingindo 110 mm na garra, conforme ilustrado abaixo. No plano da imagem, 30,1 %, 61,7 % e 87,1 % dos keypoints ficam a menos de 5, 10 e 20 px da posição verdadeira. Com as duas câmeras no treino, o desempenho é equivalente nas duas vistas: erro 3D de 57,4 mm e acurácia@60 mm de 69,0 % na frontal, contra 60,0 mm e 67,0 % na diagonal, com MAE de 12,70° e 12,65°.

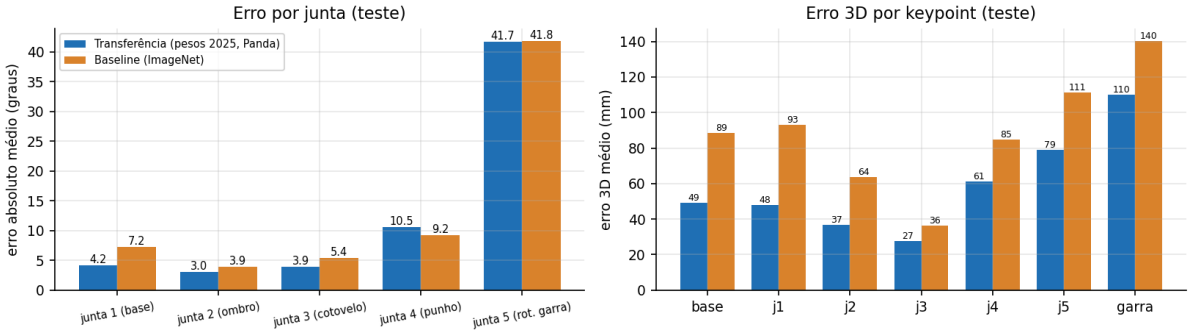


Figura 34. Erro absoluto médio por junta e erro 3D médio por keypoint no conjunto de teste, para a transferência dos pesos de 2025 e para o baseline ImageNet

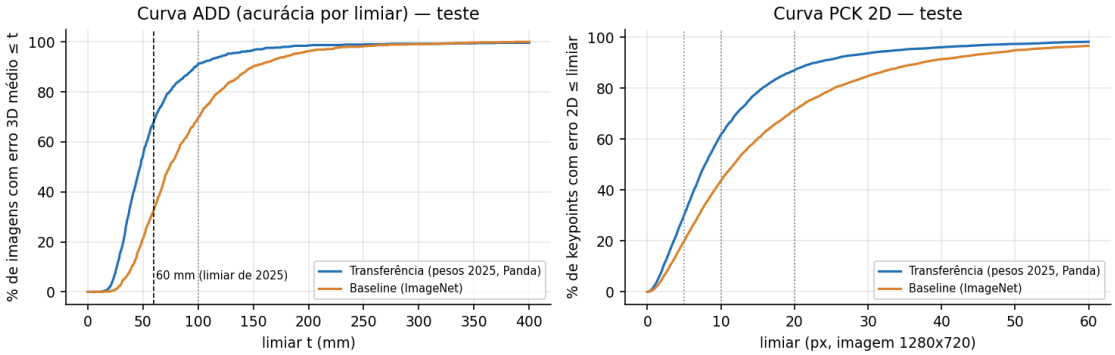


Figura 35. Curvas ADD (porcentagem de imagens com erro 3D médio até o limiar, com o limiar de 60 mm de 2025 destacado) e PCK 2D (porcentagem de keypoints com erro até o limiar, em pixels da imagem 1280×720) no conjunto de teste

4.9.3. Generalização entre câmeras

O modelo treinado apenas com a câmera frontal (experimento reduzido: 3.151 poses de treino, 10 épocas, melhor checkpoint na época 6) degrada fortemente na câmera não vista: o erro 3D passa de 140,4 mm na frontal para 418,5 mm na diagonal, a acurácia@60 mm cai de 8,6 % para 0,0 % e o MAE sobe de 23,1° para 32,6°. Como esse experimento usou menos dados e épocas que o modelo final, a comparação absoluta com o modelo treinado nas duas câmeras é apenas indicativa; a evidência relevante é a degradação de cerca de três vezes no erro 3D entre a vista conhecida e a desconhecida. O modelo aprende implicitamente a relação entre câmera e robô da vista de treino; generalizar para câmeras novas exige variar a pose da câmera nos dados de

treino ou fornecer os parâmetros extrínsecos como entrada.

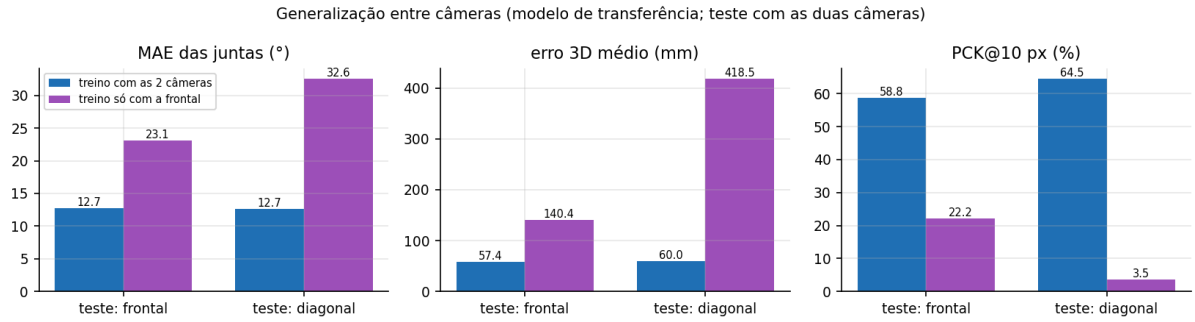


Figura 36. Generalização entre câmeras: modelo treinado com as duas câmeras e modelo treinado apenas com a câmera frontal, avaliados separadamente em cada câmera do teste

4.9.4. Robustez a perturbações da imagem

A tabela a seguir apresenta uma seleção das condições avaliadas; as curvas completas são mostradas na figura subsequente.

Tabela 15. Robustez em subconjunto fixo de 400 imagens do teste, com profundidade recalculada sobre a imagem perturbada

Perturbação	Severidade	Transf.: erro 3D (mm)	Transf.: ADD@60 (%)	Transf.: MAE (°)	Baseline: erro 3D (mm)
Sem perturbação	—	57,4	68,0	12,41	88,2
Ruído gaussiano	σ = 0,05	99,0	24,5	15,64	109,7
Ruído gaussiano	σ = 0,10	213,2	2,0	23,57	169,7
Desfoque de movimento	7 px	96,0	23,5	15,21	113,0
Desfoque de movimento	15 px	206,6	1,0	20,44	162,1
Brilho	×0,4	60,5	64,2	12,68	93,7
Contraste	×0,3	67,2	52,0	13,24	93,1
Oclusão	30 % da caixa	94,3	33,2	15,59	114,5
JPEG	qualidade 30	83,5	37,0	14,35	132,0
JPEG	qualidade 15	113,8	14,2	16,19	214,6
Resolução reduzida	escala 0,25	109,2	14,8	15,49	130,0

O modelo mostrou-se robusto a variações de brilho e contraste (erro 3D de no máximo 68 mm mesmo com brilho ×0,4 ou contraste ×0,3), o que é coerente com as augmentations fotométricas herdadas do treino de 2025. É, porém, sensível a perturbações de alta frequência: ruído gaussiano com σ = 0,05 eleva o erro 3D de 57 para 99 mm, o desfoque de 7 px para 96 mm e a compressão JPEG com qualidade 30 para 84 mm; a oclusão de 30 % da caixa do robô leva a 94 mm e a redução da resolução para 25 % a 109 mm. Em todas as perturbações o MAE das juntas degrada mais lentamente que o erro 3D, que acumula também os erros de profundidade e de rotação da raiz. O baseline

ImageNet parte de um erro maior, mas degrada menos sob ruído e desfoque fortes (σ = 0,10: 170 mm contra 213 mm; desfoque de 15 px: 162 mm contra 207 mm) e mais sob compressão JPEG (qualidade 15: 215 mm contra 114 mm); sob brilho, contraste, oclusão, JPEG e resolução reduzida a transferência permanece melhor em todas as severidades.

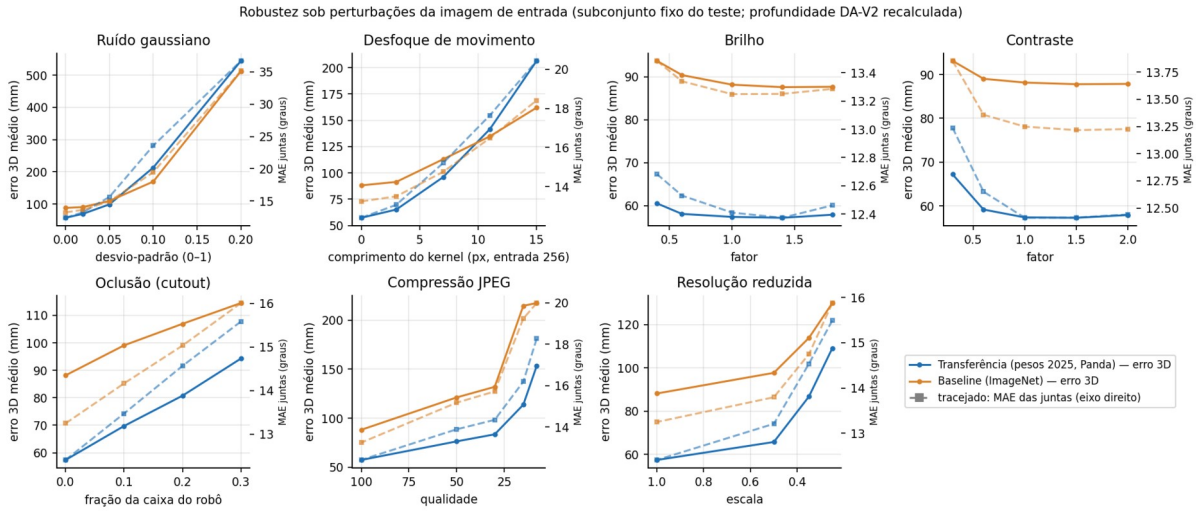


Figura 37. Robustez: erro 3D médio (linha cheia) e MAE das juntas (tracejado) em função da severidade de cada perturbação, para a transferência e para o baseline ImageNet, em subconjunto fixo de 400 imagens do teste

4.9.5. Latência e limitações

O modelo final foi empacotado num módulo de inferência (`PosePredictor`) destinado ao gêmeo digital do Objetivo 4, que recebe uma imagem RGB 1280×720 e devolve os ângulos das cinco juntas e os keypoints 2D e 3D. Em 100 chamadas com lote 1, a latência ponta a ponta foi de 162,5 ms em média (mediana de 170,1 ms e percentil 95 de 174,1 ms), ou 6,15 quadros por segundo, com pico de 2,94 GB de memória de GPU. Os valores são pessimistas, pois a GPU estava compartilhada com serviços em produção e com outras frentes. Numa medição separada, feita com um treino rodando em paralelo, o Depth Anything V2 (ViT-L) respondeu por cerca de 65 % do tempo total (138,9 ms de 213,8 ms), enquanto a rede HPE isolada levou 74,9 ms (13,3 quadros por segundo); versões menores do Depth Anything V2 ou a otimização com TensorRT são as alternativas para operação em tempo real.

As principais limitações da avaliação são: medições apenas em dados sintéticos e com duas câmeras fixas, sem verificação em imagens reais do braço físico; a junta 5 não é observável com os keypoints escolhidos e a abertura da garra não é estimada; o experimento entre câmeras usou um subconjunto dos dados; a robustez foi medida num subconjunto de 400 imagens; e a latência foi medida em GPU compartilhada. Como próximos passos, propõem-se adicionar keypoints nas pontas das pinças, randomizar pose de câmera, iluminação e

texturas na geração de dados, incluir ruído, desfoque e compressão nas augmentations, capturar imagens reais para aplicar a etapa autossupervisionada de 2025 e reduzir a latência do ramo de profundidade.

Código e evidências: repositório cerise-2026-pose-servo6dof (privado)

4.10. Modelo digital do braço robótico

Esta etapa corresponde à Atividade 11 do Objetivo 4: modelar o braço robótico em ambiente virtual, com suas características físicas e cinemáticas. O modelo digital é a base comum do dataset, do planejamento de movimento e do gêmeo digital, e por isso foi caracterizado e validado de forma independente.

4.10.1. Metodologia

O URDF foi gerado a partir do xacro do pacote de descrição, com os mesmos parâmetros utilizados pelo Gazebo na geração do dataset. A partir dele foram extraídas as tabelas de elos e juntas e a árvore cinemática, e foi implementada uma cinemática direta própria em Python e NumPy (`modelo_digital/cinematica.py`), que compõe as transformações das origens (convenção RPY) com rotações de eixo-ângulo (fórmula de Rodrigues), resolve as juntas *mimic* e opera em lotes vetorizados. Essa implementação foi validada contra o TF publicado pelo `robot_state_publisher` do ROS 2 para o mesmo URDF, em 300 configurações aleatórias e 11 elos. O espaço de trabalho foi estimado por Monte Carlo com 100.000 configurações uniformes nos limites das juntas. Também foi implementado o modelo *pinhole* das câmeras do estúdio (pose do SDF, campo de visão horizontal de 60°, 1280×720 pixels), que permite projetar as malhas STL posicionadas pela cinemática direta sobre as imagens do dataset e medir o IoU entre a silhueta do modelo e a máscara de segmentação do simulador. Por fim, o MoveIt 2 [35] foi executado sem interface gráfica, com `move_group`, planejador OMPL [36] RRTConnect [37], cinemática inversa KDL e `ros2_control` com hardware simulado (`mock_components/GenericSystem`); um cliente da ação de planejamento solicitou cinco objetivos, com dez pedidos de planejamento por objetivo.

4.10.2. Resultados

O URDF possui 13 elos e 12 juntas: 11 de revolução, das quais 6 são ativas (`joint_1` a `joint_6`) e 5 são *mimic* de `joint_6`, e uma junta fixa entre o referencial do mundo e a base. O braço tem 5 graus de liberdade e a garra, 1. A tabela a seguir resume a cadeia ativa; as distâncias entre origens consecutivas da cadeia principal são 0,23, 0,57, 0,63, 0,78 e 0,36 m, e as malhas STL são aplicadas com fator de escala 6. As massas somam 1,51 kg, e todas as inércias têm valor de preenchimento (1 kg·m²), de modo que o modelo é cinematicamente completo, mas não dinamicamente calibrado.

Tabela 16. Juntas ativas do modelo digital extraídas do URDF

Junta	Elo pai → elo filho	Origem xyz (m)	Eixo	Limites (°)
joint_1	base_link → primeiro_elo	(0; −0,23; −0,01)	z	−90 a 90
joint_2	primeiro_elo → segundo_elo	(−0,05; −0,03; 0,57)	x	−90 a 90
joint_3	segundo_elo → terceiro_elo	(0,06; 0; 0,63)	x	−90 a 90
joint_4	terceiro_elo → quarto_elo	(0; 0,78; 0)	x	−90 a 90
joint_5	quarto_elo → base_garra	(0,02; 0,36; 0)	y	−90 a 90
joint_6 (garra)	base_garra → garra_elo1	(−0,072; 0,12; 0,01)	z	−40 a 30

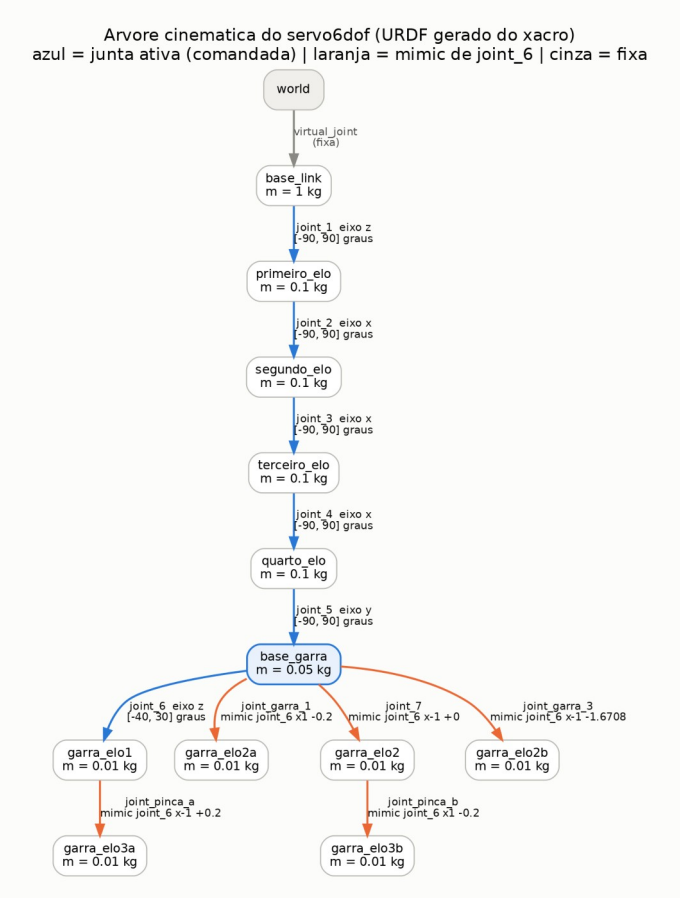


Figura 38. Árvore cinemática do braço servo6dof extraída do URDF, com as juntas ativas, seus eixos e limites e as juntas mimic da garra

As verificações do modelo estão reunidas na tabela seguinte. A cinemática direta própria coincidiu com o TF do ROS 2 no nível do arredondamento numérico, inclusive nos elos *mimic* da garra, e a cinemática do visualizador web, usada no gêmeo digital, coincidiu com a do servidor dentro da precisão de ponto flutuante de 32 bits.

Tabela 17. Verificações do modelo digital

Verificação	Amostra	Resultado medido
Cinemática direta própria × TF do ROS 2	300 configurações × 11 elos	erro máx. $1{,}9\times10^{-15}$ m e $4{,}2\times10^{-8}$ rad
Cinemática do front-end (three.js) × própria	capturas do visualizador	até $7{,}6\times10^{-7}$ m
Espaço de trabalho da ponta da garra	100.000 configurações	alcance horizontal 2,05 m; z de −0,82 a 2,58 m
Silhueta do modelo × máscara (câmera frontal)	1.000 imagens	IoU médio 0,950 (mediana 0,965)
Silhueta do modelo × máscara (câmera diagonal)	1.000 imagens	IoU médio 0,938 (mediana 0,951)
MoveIt 2 headless	5 objetivos × 10 pedidos	50/50 planos; 12,9 a 17,0 ms em média

O fecho convexo do espaço de trabalho tem 27,8 m³, e o volume efetivamente ocupado, medido em voxels de 5 cm, é de 7,9 m³. Cerca de 20 % das configurações dentro dos limites levariam a ponta da garra abaixo do plano do chão, propriedade que explica parte das divergências entre juntas comandadas e reais observadas no dataset.

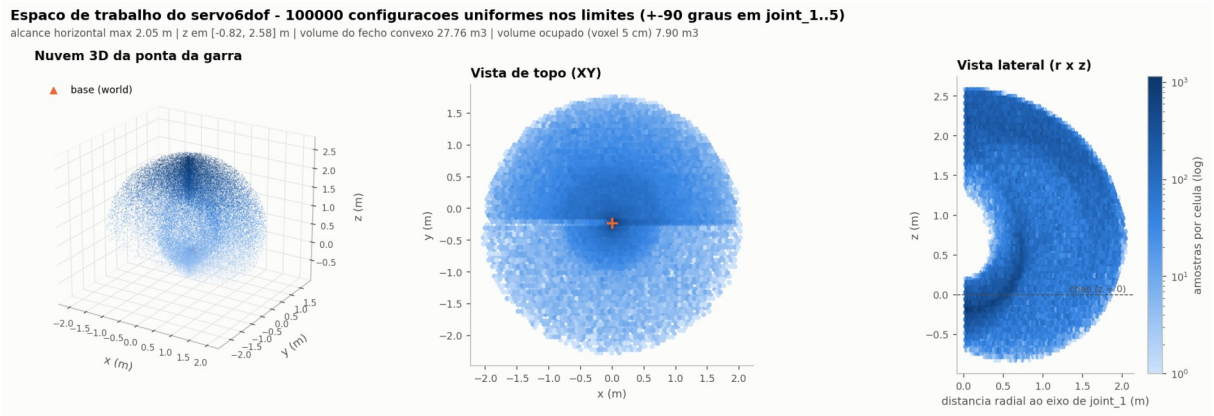


Figura 39. Espaço de trabalho da ponta da garra estimado com 100.000 configurações: nuvem 3D, vista de topo e vista lateral radial com o plano do chão

A projeção das malhas posicionadas pelas juntas registradas sobre as imagens do simulador coincide visualmente com o braço renderizado, o que valida simultaneamente o modelo cinemático, o modelo de câmera e os rótulos do dataset. Em 2,05 % das imagens avaliadas (41 de 2.000), o IoU ficou abaixo de 0,8, casos em que o braço aparece numa configuração diferente da registrada e que devem ser filtrados para treino.

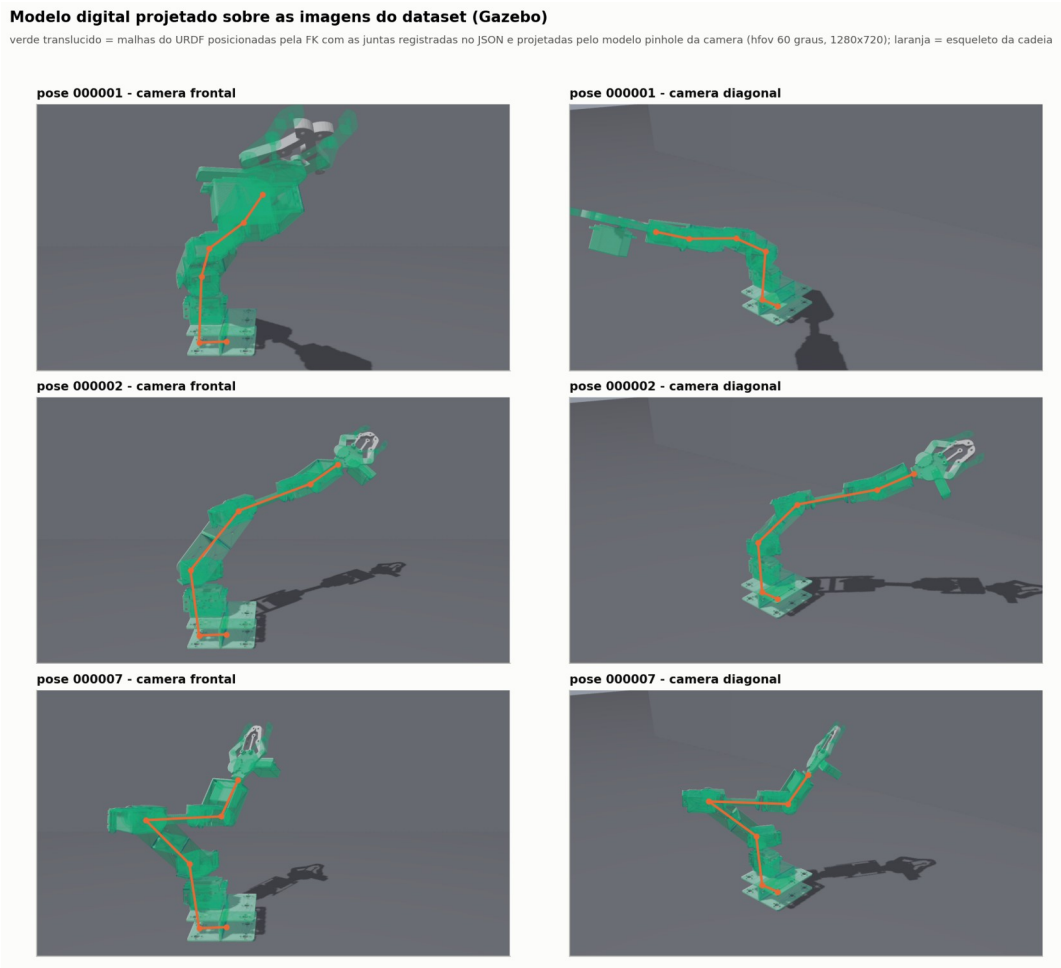


Figura 40. Malhas do modelo digital (verde translúcido) e cadeia cinemática (laranja) projetadas sobre imagens do dataset nas câmeras frontal e diagonal

No MoveIt 2, os cinco objetivos — três em espaço de juntas, um alvo cartesiano de posição e o retorno à posição inicial — foram planejados e executados com sucesso, com trajetórias de 17 a 28 pontos e duração de 1,56 a 2,63 s. No alvo cartesiano, a posição final de `base_garra` ficou a 1,6 cm do alvo, dentro da tolerância de 2 cm especificada no pedido.

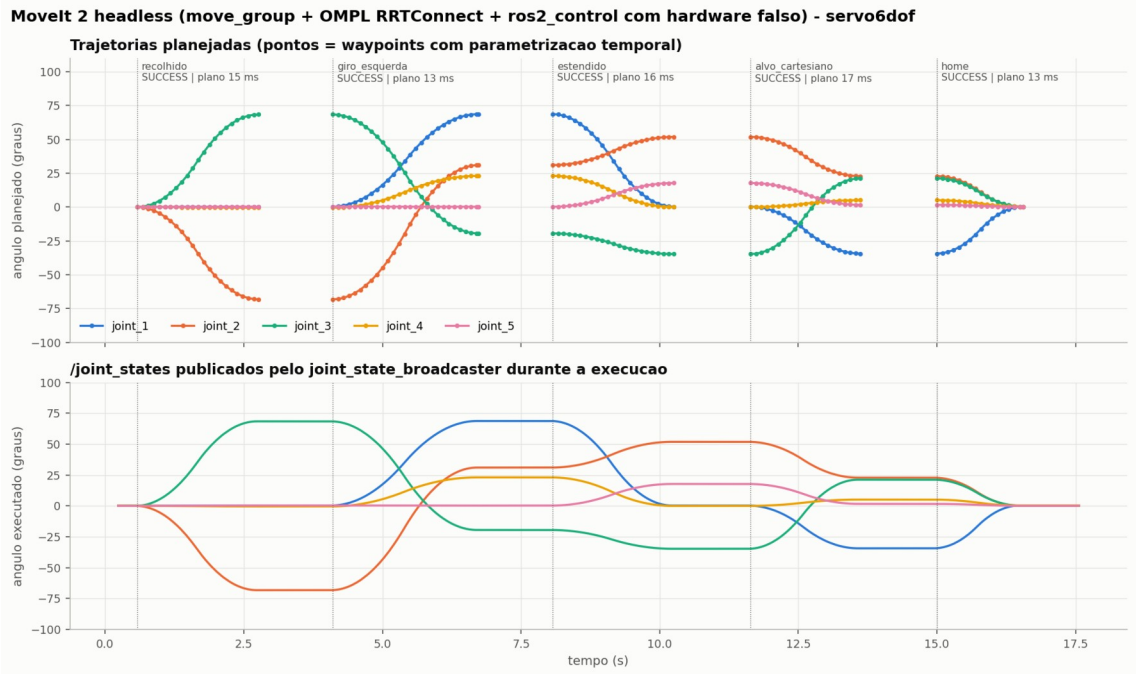


Figura 41. Trajetórias de juntas planejadas pelo MoveIt 2 com OMPL RRTConnect e estados de junta publicados durante a execução com hardware simulado

4.10.3. Discussão e limitações

O modelo digital é cinematicamente consistente com o simulador e com o ROS 2, o que o qualifica como referência para o gêmeo digital. As limitações são de natureza dinâmica e de fidelidade da garra: as massas são simbólicas e as inércias são valores de preenchimento, inadequados para simulação dinâmica fiel; o Gazebo Fortress não implementa as juntas *mimic*, de modo que a geometria da garra nas imagens difere da do modelo e responde por parte do erro residual de silhueta; e o planejamento não inclui o chão como objeto de colisão. O MoveIt 2 foi exercitado apenas com hardware simulado, e a validação com o braço físico, assim como a calibração de massas e inércias a partir do CAD ou de medições, permanece como trabalho futuro.

Código e evidências: repositório cerise-2026-servo6dof-gemeo-digital (privado)

4.11. Gêmeo digital do braço robótico

Esta etapa corresponde à Atividade 12 do Objetivo 4: integrar o modelo de detecção de pose ao ambiente virtual do braço robótico, com o resultado esperado de um gêmeo digital capaz de reproduzir o comportamento do braço a partir dos dados capturados [38]. No sistema desenvolvido, o braço de referência segue a verdade-base do simulador (`juntas_reais`), enquanto o gêmeo digital segue apenas as juntas estimadas pelo modelo de pose a partir de uma única imagem RGB, sem acesso a qualquer anotação; a comparação quadro a quadro

entre os dois mede a fidelidade do gêmeo.

4.11.1. Arquitetura e implementação

O servidor do gêmeo (FastAPI com WebSocket) executa em contêiner na estação DGX Spark e é publicado apenas na interface local da máquina, com acesso por túnel SSH local. A estimação é feita por um componente plugável (`Predictor`), com implementações de oráculo (verdade-base), de ruído sintético (apenas para demonstração do painel), de chamada HTTP e de adaptação ao modelo da frente de estimação de pose (Atividades 8 e 9), que combina profundidade monocular Depth Anything V2 [30] com um estimador de pose e devolve os ângulos `joint_1` a `joint_5`; a garra não é estimada. O front-end, em three.js r160 com urdf-loader 0.13.1, carrega o mesmo URDF e as malhas STL duas vezes (referência sólida e gêmeo verde translúcido), lado a lado ou sobrepostos na vista da câmera do dataset, e exibe a imagem com o esqueleto 2D previsto, os ângulos por junta e o gráfico de erro.

Antes da integração, a infraestrutura foi validada com o oráculo (12 malhas carregadas por braço, sem erros, e cinemática do navegador coincidente com a do servidor). A avaliação usa o mesmo conjunto de teste da frente de pose (500 poses × 2 câmeras = 1.000 imagens, nenhuma usada no treino). Para medir latência, o WebSocket opera em malha fechada: cada quadro só é enviado após o front-end confirmar que o anterior foi desenhado no canvas, em Chromium *headless* (Playwright) com WebGL acelerado pela GPU da própria estação. No ROS 2, o nó `gemeo_twin` publica `/twin/joint_states` a partir das imagens, e um segundo `robot_state_publisher` (prefixo `twin/`) alimenta o RViz 2 em tela virtual.

4.11.2. Resultados

A tabela a seguir apresenta o erro absoluto por junta nos 1.000 quadros. A média das cinco juntas no gêmeo (12,68°) é igual à da avaliação *offline* da frente de pose nas mesmas imagens, o que indica que a integração (leitura da imagem, recorte, inferência, adaptação e conversão de unidades) não acrescenta erro. O viés médio é inferior a 1° em módulo nas juntas 1 a 4.

Tabela 18. Erro absoluto por junta no gêmeo digital, em graus (conjunto de teste, apenas imagem)

Junta	Frontal (média)	Diagonal (média)	Total (média)	Total (mediana)	Total (p95)
joint_1 (base)	4,08	4,30	4,19	2,92	11,9
joint_2 (ombro)	3,12	2,96	3,04	1,89	9,0
joint_3 (cotovelo)	4,09	3,73	3,91	2,54	11,6
joint_4 (punho)	10,46	10,57	10,52	6,63	36,1
joint_5	41,75	41,71	41,73	42,49	78,0

(rotação da garra)					
Média das juntas 1 a 4	5,44	5,39	5,41	3,83	15,4

O erro de posição do efetuador, calculado pela cinemática direta com as juntas previstas e verdadeiras, teve mediana de 76,4 mm, média de 107,1 mm, p95 de 275,0 mm e máximo de 991,4 mm; 64,5 % dos quadros ficaram abaixo de 100 mm e 27,2 % abaixo de 50 mm. Como a ponta da garra alcança até 2,09 m de distância do ombro, 1° de erro no ombro desloca a garra em até cerca de 37 mm.

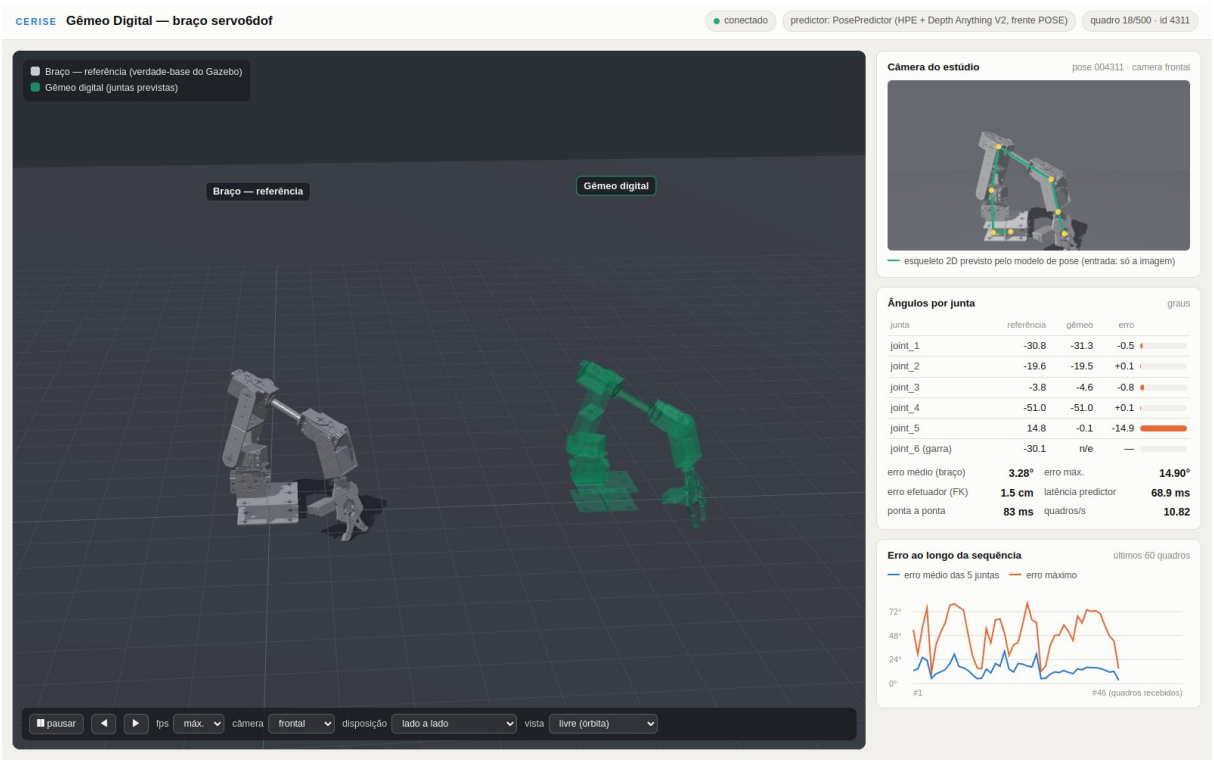


Figura 42. Gêmeo digital com o modelo de pose em um quadro de baixo erro: braço de referência e gêmeo lado a lado, esqueleto 2D previsto sobre a imagem, ângulos por junta e erro do efetuador de 1,5 cm

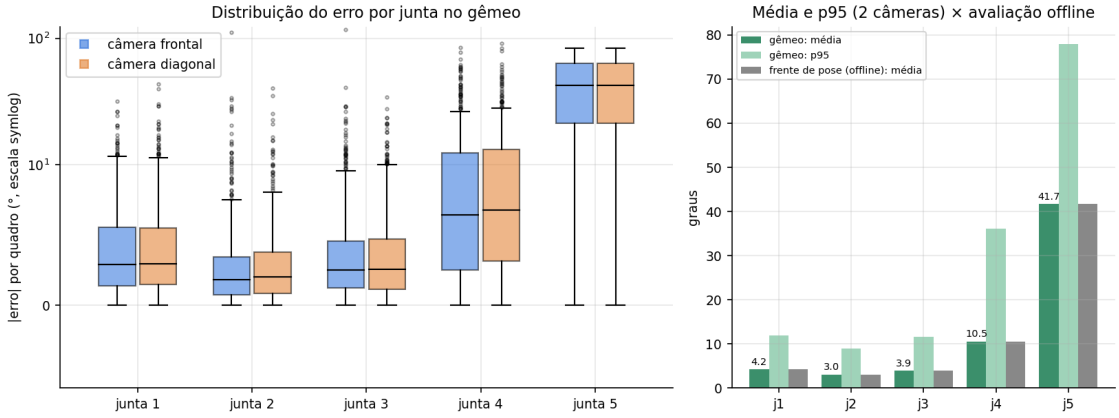


Figura 43. Distribuição do erro por junta no gêmeo por câmera e comparação entre média e p95 do gêmeo e a avaliação offline da frente de pose

A latência entre a leitura da imagem e o gêmeo desenhado está decomposta na tabela seguinte. O modelo de pose domina o tempo: a mediana ponta a ponta supera a do modelo em cerca de 22 ms, parcela atribuível às demais etapas da integração.

Tabela 19. Latência por etapa, em milissegundos (1.000 quadros, malha fechada, WebGL na GPU)

Etapa	Mediana	Média	p95
Leitura do PNG 1280×720	8,4	10,0	18,2
Modelo de pose (recorte, profundidade e pose)	77,7	103,1	180,4
Cinemática direta e JPEG da câmera	3,6	4,4	8,6
Serialização e WebSocket	1,3	1,4	2,2
Atualização no three.js até o canvas	14,1	15,0	18,2
Ponta a ponta medido	99,6	134,5	216,9

Em malha fechada, a sessão da câmera frontal sustentou 9,76 quadros/s e a da diagonal, 5,93 quadros/s; com taxas-alvo fixas, foram medidos 2,00, 5,00 e 9,97 quadros/s para alvos de 2, 5 e 10. A diferença entre câmeras decorre da carga da GPU, compartilhada com serviços de produção e outras tarefas: num controle com 120 chamadas alternando as câmeras, a latência do modelo foi de 65,2 ms (frontal) e 64,6 ms (diagonal). O pico de memória de GPU do modelo foi de 2,94 GB alocados. Com rasterização por software (SwiftShader), o navegador caiu para 0,42 quadro/s, o que mostra a necessidade de WebGL acelerado.

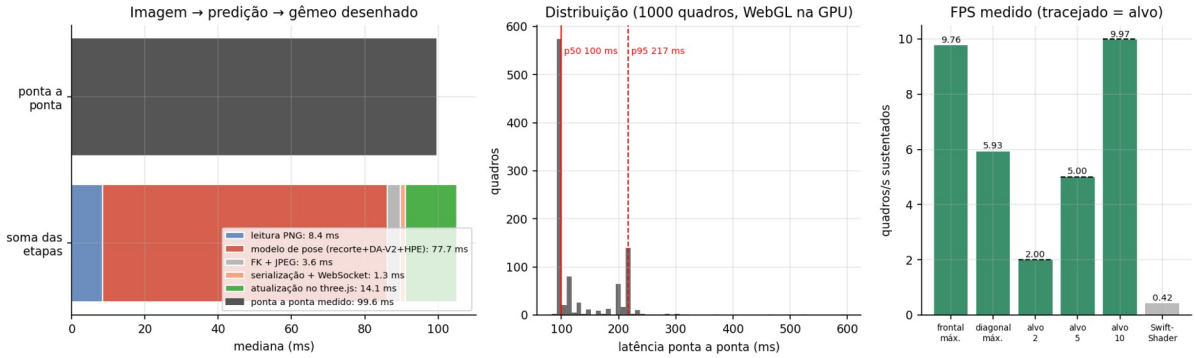


Figura 44. Latência imagem → predição → gêmeo desenhado por etapa, distribuição da latência em 1.000 quadros e taxa de quadros sustentada por sessão

Na análise de falhas, o principal modo identificado foi o braço cortado pelo enquadramento da câmera: em uma amostra de 200 quadros, isso ocorreu em 22,5 % deles, com erro mediano do efetuador de 117 mm contra 70 mm nos demais, e esteve presente em 45 % dos 20 piores quadros. O pior caso (991 mm) é uma ambiguidade de configuração do tipo "cotovelo invertido", com erros de +111° no ombro e −118° no cotovelo e silhueta 2D semelhante à verdadeira. O erro também cresce com o giro da base (correlação de Spearman de 0,36 com o módulo de `joint_1`), situação de escorço em relação às câmeras.

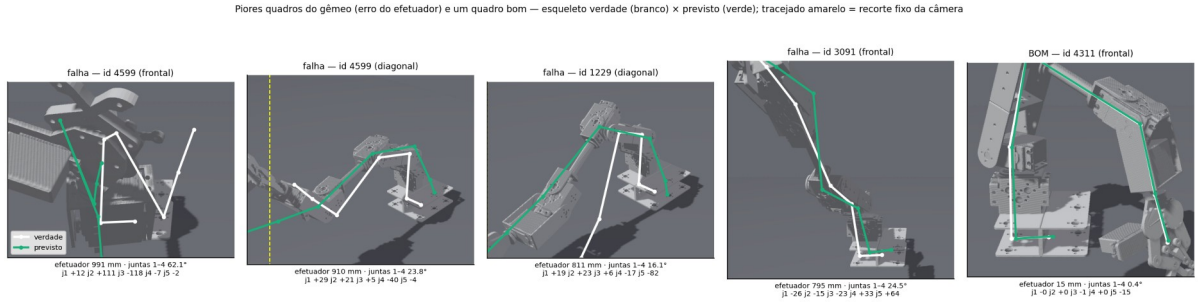


Figura 45. Piores quadros do gêmeo pelo erro do efetuador e um quadro de baixo erro, com esqueleto verdadeiro em branco e previsto em verde

No teste em ROS 2 (40 s, imagens da câmera frontal do conjunto de teste publicadas a 2 Hz), o tópico `/twin/joint_states` foi publicado a 2,00 Hz, com 79 pares casados com a referência pelo carimbo de tempo. O atraso entre a imagem e a mensagem do gêmeo teve mediana de 255 ms (p95 de 282 ms), dos quais 87 ms correspondem ao modelo via HTTP e o restante ao transporte da imagem bruta; os erros médios por junta foram de 4,2, 5,3, 4,6, 11,0 e 37,6°, e o erro mediano do efetuador, de 77 mm, coerentes com a avaliação pelo navegador.

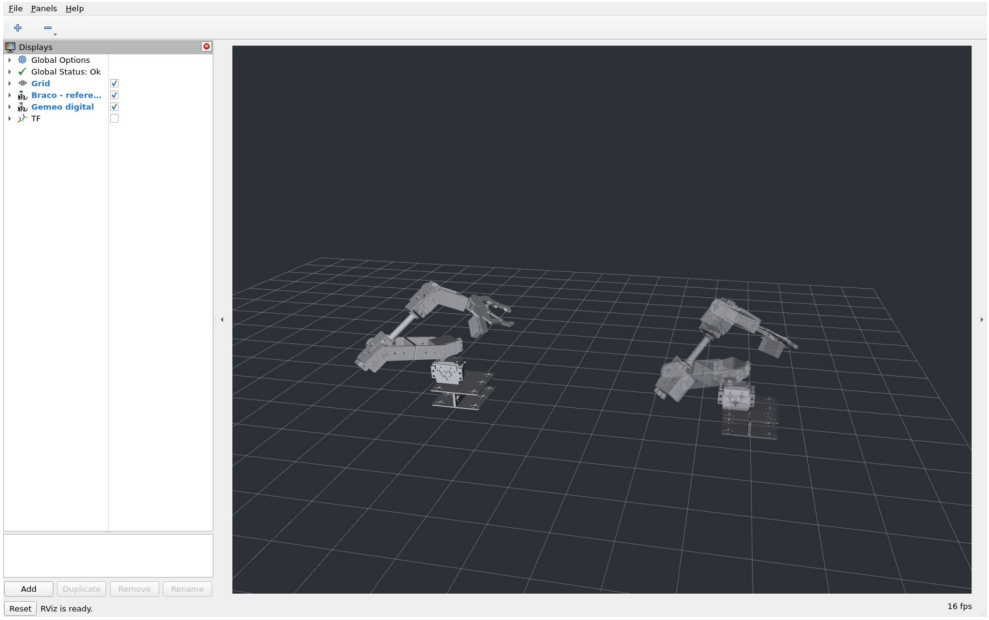


Figura 46. RViz 2 em tela virtual com o braço de referência e o gêmeo digital, deslocado lateralmente, movido por /twin/joint_states a partir das imagens

4.11.3. Discussão e limitações

O gêmeo reproduz o comportamento do braço nas juntas que o modelo observa bem: nas juntas 1 a 4, o erro mediano é de 3,8°, e 64,5 % dos quadros têm o efetuador a menos de 10 cm da posição real, com cerca de 10 quadros/s e latência mediana de 0,1 s, adequados ao acompanhamento visual. O resultado depende essencialmente do modelo de pose, pois a integração não acrescenta erro e acrescenta pouco tempo. Como limitações, a validação foi feita apenas em simulação, com imagens sintéticas e as mesmas duas câmeras fixas do treino; a rotação da garra (`joint_5`, 41,7°) não é estimada de forma útil e a abertura da garra não é estimada (no ROS 2 ela é fixada em 0 rad apenas para completar a árvore TF); a latência foi medida com a GPU compartilhada e o navegador no mesmo host, sem garantia de tempo real; e cada quadro é tratado de forma independente, sem filtro temporal, sobre uma sequência de poses independentes, não um movimento contínuo. O visualizador não foi exposto publicamente, o que depende de decisão do bolsista. Como próximos passos, propõem-se a avaliação com o braço físico e câmera calibrada, a fusão das duas câmeras com filtragem temporal e a inclusão de pontos-chave nas pinças da garra.

Código e evidências: repositório cerise-2026-servo6dof-gemeo-digital (privado)

4.12. Protótipo de comunicação LoRa entre dois dispositivos

Esta etapa corresponde à Atividade 10 do plano de trabalho, que prevê o desenvolvimento de um protótipo de comunicação LoRa capaz de enviar e

receber dados entre dois dispositivos. O protótipo foi concebido como o enlace entre um nó embarcado no drone (papel *node*) e uma estação de solo (papel *gateway*). Foram desenvolvidos o protocolo de mensagens, o firmware dos dois papéis, testes automatizados e ferramentas de análise e de registro. A gravação nas placas e o ensaio físico de envio e recebimento não foram realizados, porque os módulos de rádio ainda não foram adquiridos; toda a verificação descrita a seguir foi feita sem hardware de rádio, por compilação, testes, validação cruzada e simulação.

4.12.1. Protocolo e firmware

O protocolo `cerise_proto` v1 foi implementado em C99 como biblioteca compartilhada pelo firmware e pelos testes. Cada quadro possui cabeçalho de 8 bytes (versão e tipo, flags, identificador de rede, origem, destino, número de sequência de 16 bits e tamanho do payload), payload de layout fixo por tipo e CRC-16/CCITT-FALSE, conforme ilustrado abaixo. Os tipos de mensagem foram alinhados com a frente de segmentação de áreas entre drones: `HEARTBEAT`, `POS` (telemetria de posição, bateria e contagem), `AREA_ASSIGN` (área atribuída pela estação de solo), `COVERAGE_DONE`, `COUNT_REPORT` e `ACK`, que devolve ao nó o RSSI e o SNR medidos no gateway. Três tipos propostos por aquela frente (`AREA_POLY`, `TASK_BID` e `TASK_AWARD`) foram incorporados como extensão aditiva, sem mudança de versão, e um teste automatizado confirma a compatibilidade byte a byte com os quadros de exemplo daquela frente. O payload do `POS` foi dimensionado em 14 bytes para que o quadro completo, de 24 B, permaneça abaixo de 400 ms no ar até SF10.

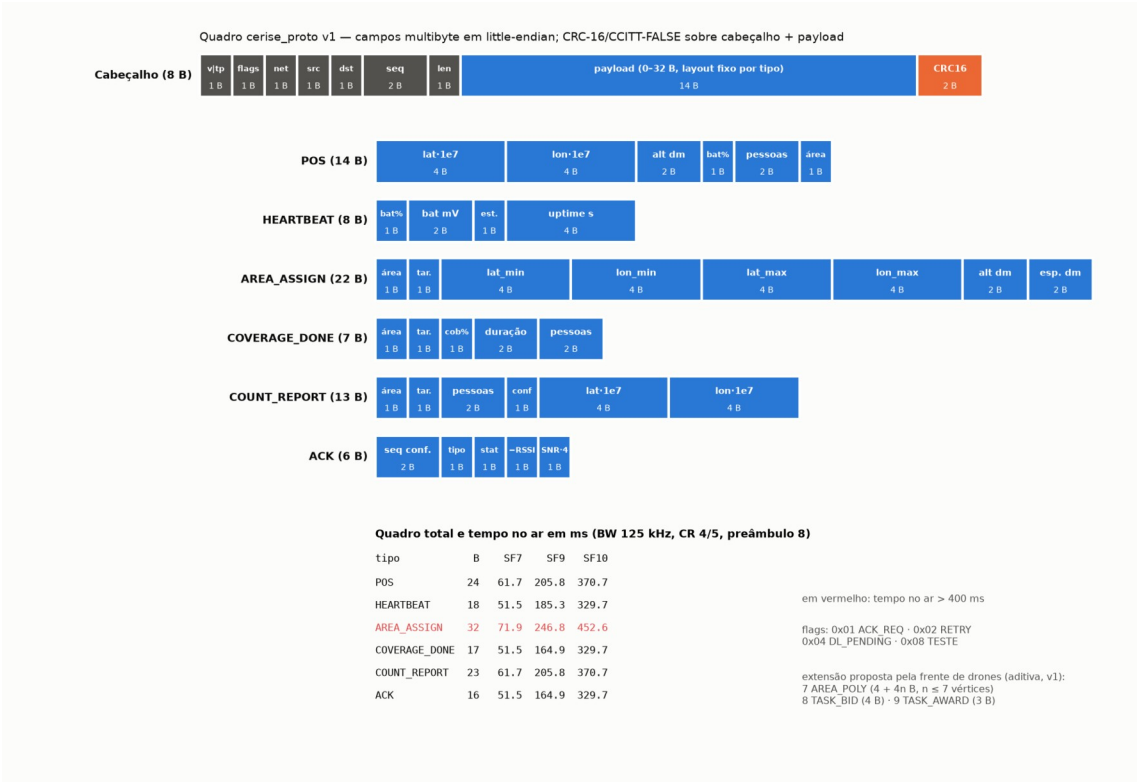


Figura 47. Formato do quadro cerise_proto v1, layout do payload de cada tipo de mensagem e tempo no ar por tipo em SF7, SF9 e SF10 (BW 125 kHz, CR 4/5)

O firmware foi escrito com PlatformIO (framework Arduino), RadioLib 7.8.1 [39] e U8g2, tendo a Heltec WiFi LoRa 32 V3 (ESP32-S3 e SX1262) como placa principal e a TTGO LoRa32 v2.1 (ESP32 e SX1276) como alternativa. O papel *node* gera um POS a cada 5 s com variação aleatória de ±10 %, aguarda o ACK por ToA(ACK) + 160 ms e repete a transmissão até três vezes com recuo aleatório exponencial; antes de transmitir, executa detecção de atividade no canal (CAD) e bloqueia quadros com mais de 400 ms no ar. O papel *gateway* valida os CRCs, descarta duplicatas e tráfego de outras redes, responde com ACK, estima a taxa de entrega de pacotes (PDR) por nó pelas lacunas de sequência, mantém uma fila de downlink e emite uma linha JSON por evento na serial. A troca de mensagens prevista é representada na figura a seguir.

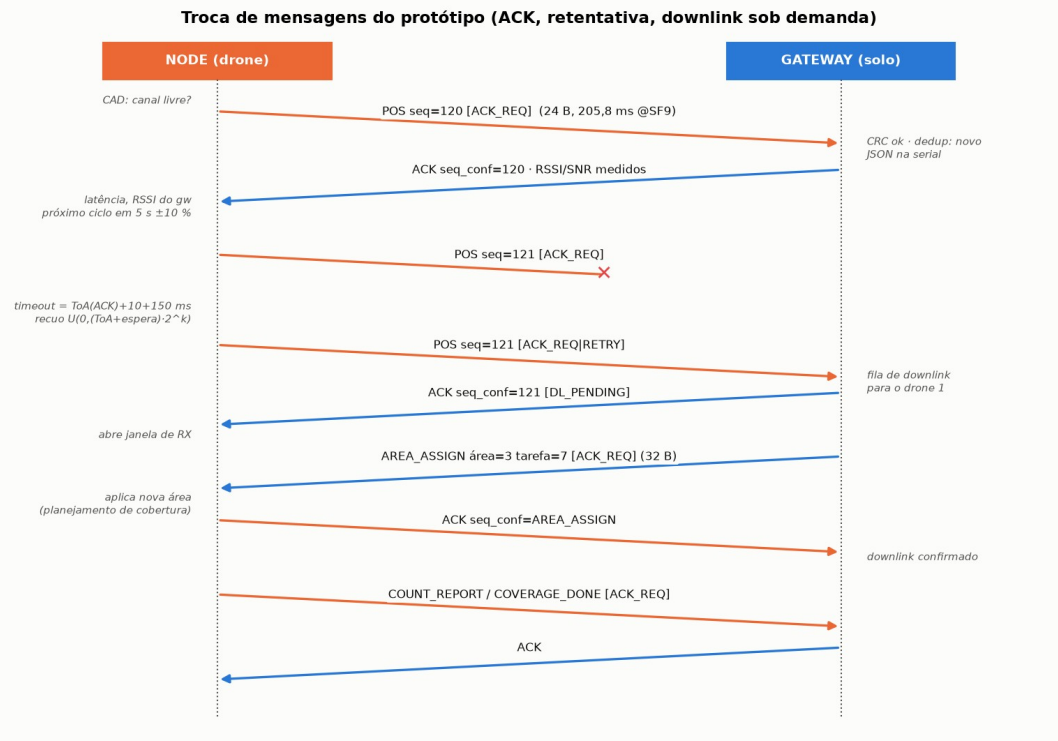


Figura 48. Sequência de mensagens entre node e gateway, com confirmação (ACK), retentativa após perda e downlink AREA_ASSIGN entregue logo após o ACK

Os parâmetros de rádio seguiram o enquadramento brasileiro: plano AU915-928 [40], [41], canal único de 916,8 MHz para a bancada, 14 dBm e limite de 400 ms no ar por quadro, compatível com o tempo de ocupação do item 10.2.5 do Ato ANATEL nº 14.448/2017 [42].

4.12.2. Verificação e análise

A verificação foi organizada em quatro partes: (i) 30 testes unitários nativos (Unity, executados no PC); (ii) um espelho do protocolo em Python, comparado byte a byte com a implementação em C por meio de vetores aleatórios gerados por um programa em C; (iii) uma calculadora de tempo no ar baseada na formulação da Semtech [43]; e (iv) uma simulação de eventos discretos de N drones compartilhando o canal, validada contra a expressão analítica do ALOHA puro [44]. Os testes unitários cobrem, entre outros casos, a detecção de todos os erros de 1 bit (192), de 2 bits (18.336) e de rajadas de até 16 bits (2.952) em um quadro de 24 B, conforme a saída de terminal abaixo. Os cinco ambientes de firmware compilaram sem avisos; o *node* da Heltec V3 ocupa 22.400 B de RAM (6,8 %) e 376.241 B de Flash (11,3 %), e o *gateway*, 30.552 B (9,3 %) e 355.821 B (10,6 %).

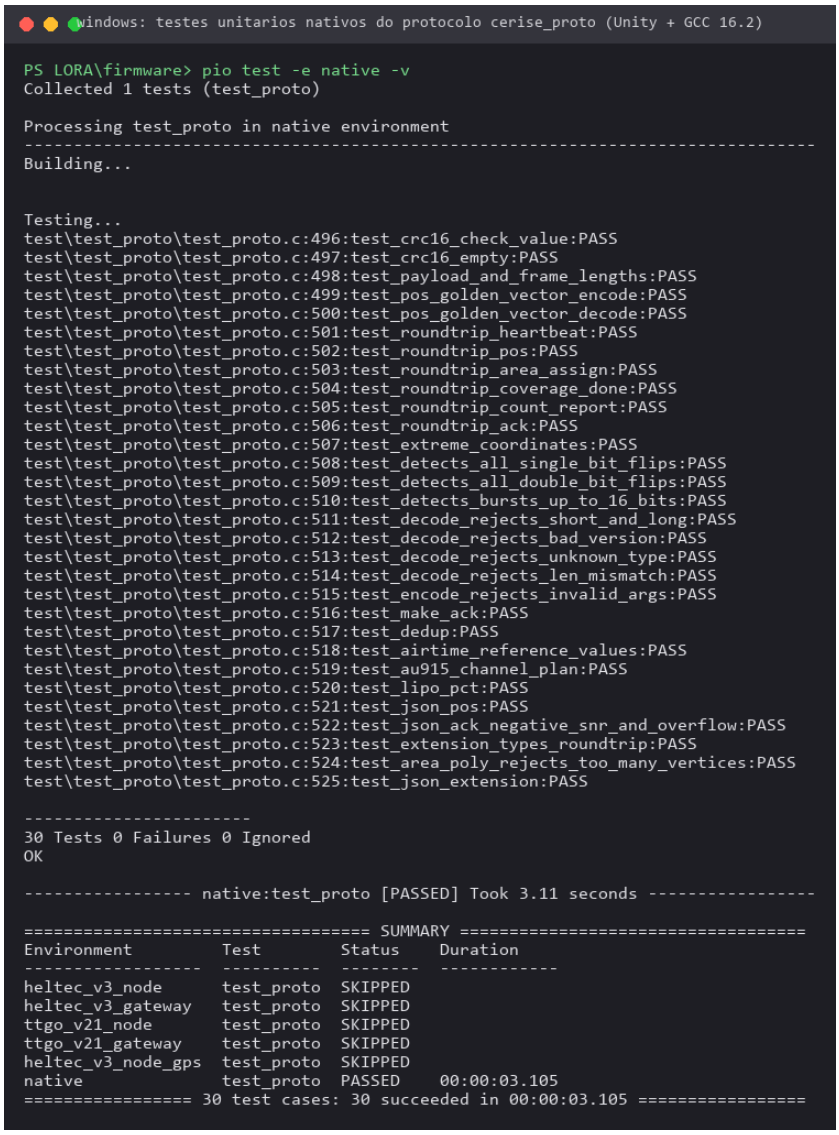


Figura 49. Saída do terminal com a execução dos 30 testes unitários nativos do protocolo (pio test -e native), todos aprovados

Tabela 20. Verificação do software do protótipo, realizada sem hardware de rádio

Item verificado	Resultado
Testes unitários nativos (Unity)	30/30 aprovados
Testes em Python (pytest)	21/21 aprovados
Quadros válidos idênticos entre C e Python	3.000/3.000
Linhas JSON idênticas entre C e Python	3.000/3.000
Quadros corrompidos rejeitados com o mesmo código	3.000/3.000
Combinações de tempo no ar idênticas	9.360/9.360
Ambientes de firmware compilados	5/5, sem avisos

O tempo no ar do quadro `POS` resultou em 61,7 ms, 205,8 ms e 370,7 ms em SF7, SF9 e SF10. O `AREA_ASSIGN` (32 B) ocupa 452,6 ms em SF10 e, portanto, deve ser enviado com SF ≤ 9; SF11 e SF12 não comportam nenhum quadro dentro do limite de 400 ms.

A simulação totalizou 1.130 execuções (5 sementes de 1.800 s simulados por configuração). O simulador reproduziu a teoria do ALOHA puro com erro absoluto médio de 0,56 p.p. e máximo de 1,47 p.p. entre N = 2 e N = 20. A tabela a seguir e a figura subsequente apresentam a PDR média para SF9 e POS a cada 5 s.

Tabela 21. PDR simulada (%) em função do número de drones N, em SF9, com POS de 24 B a cada 5 s

N	ALOHA puro	ALOHA + ACK	CSMA/CAD + ACK	Slotted ALOHA	TDMA + ACK
2	91,1	99,7	100,0	93,0	100,0
5	71,4	67,6	100,0	83,7	100,0
8	54,5	26,6	97,9	72,8	100,0
10	46,0	15,1	62,6	67,2	100,0
12	38,4	8,1	44,0	61,0	100,0
15	30,1	3,2	26,5	54,2	83,5
20	19,2	0,7	11,2	43,7	62,6

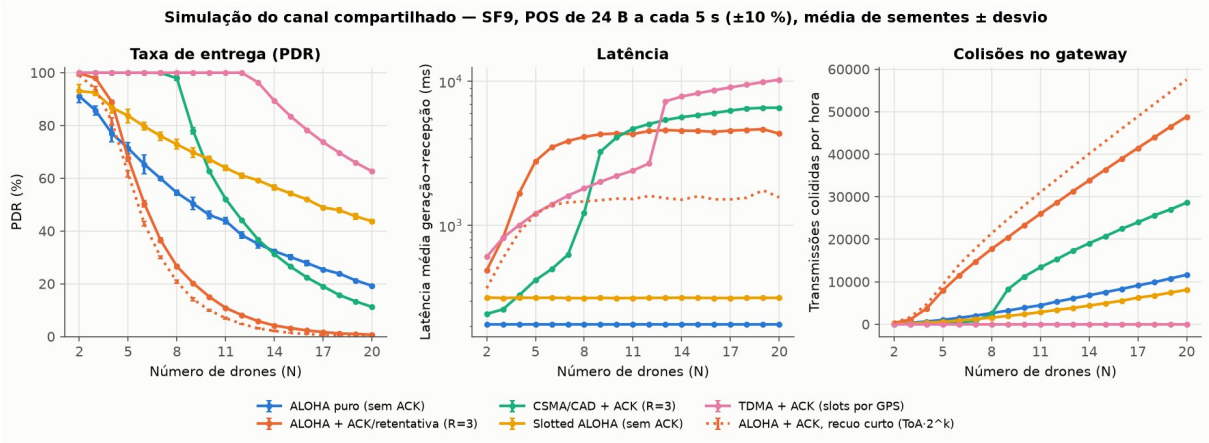


Figura 50. Resultados da simulação de eventos discretos em SF9: PDR, latência média e colisões no gateway em função do número de drones para cinco esquemas de acesso ao canal

Três conclusões orientam o projeto. Primeiro, o ACK com retransmissão no mesmo canal piora o desempenho sob carga: com 10 drones, a ocupação somada do canal chega a 145 % (transmissões sobrepostas) e a PDR cai para 15,1 %, contra 46,0 % sem ACK; esse resultado motivou a alteração do recuo de retransmissão no firmware para uma janela proporcional a (ToA + espera do ACK)·2^k. Segundo, a escuta prévia do canal (CAD) mantém PDR ≥ 97,9 % até 8 drones, mas também satura acima disso. Terceiro, o TDMA com slots sincronizados por GPS (slot de 401 ms em SF9) entrega 100 % até 12 drones, ao custo de maior latência (2.214 ms em média com 10 drones, contra 206 ms no ALOHA puro). Com efeito captura de 6 dB, a PDR do ALOHA puro com 10 drones sobe para 62,6 %, o que indica que a tabela é conservadora.

Por fim, o leitor serial do gateway (dashboard) foi exercitado com 600 s simulados de três drones a 1,0 km, 4,6 km e 6,4 km da estação, usando o

caminho real de codificação e decodificação dos quadros sobre um canal sintético (log-distância com n = 3, sombreamento de 4 dB, colisões e 10 % de perda de ACK). Foram lidas 593 linhas sem nenhuma inválida, e a PDR calculada pelo dashboard coincidiu com a reportada pelo gateway simulado: 98,6 % (137/139), 97,8 % (135/138) e 94,8 % (129/136), conforme o painel abaixo.

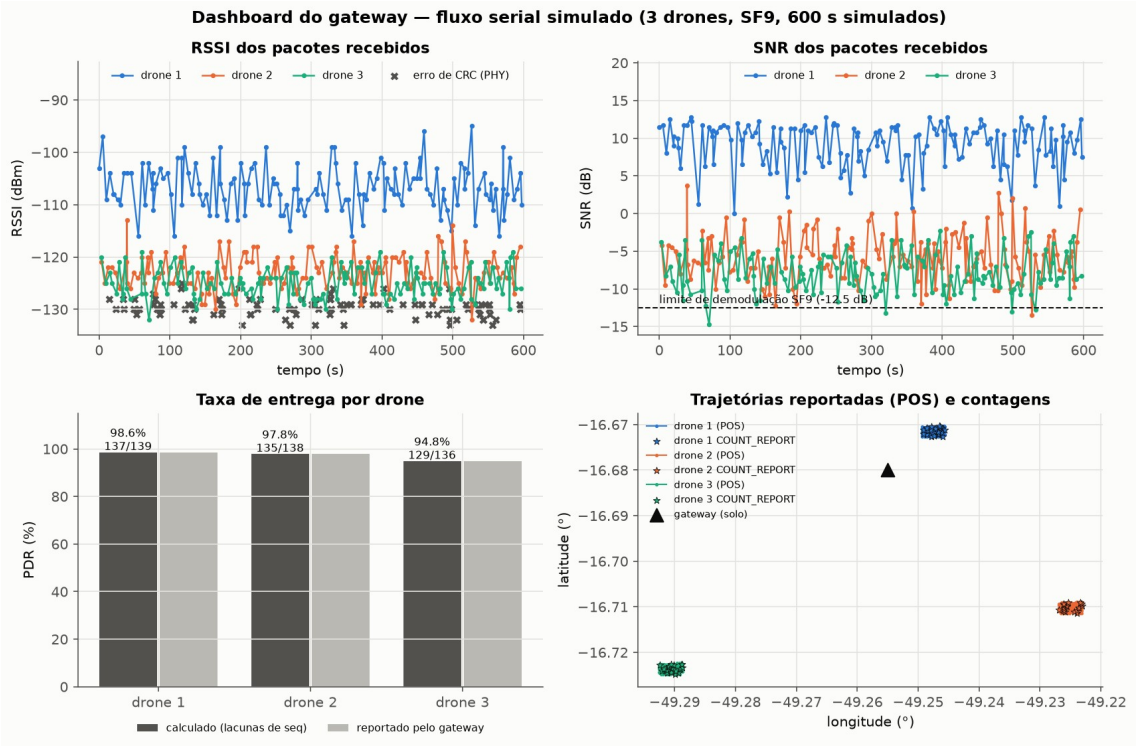


Figura 51. Painel gerado pelo dashboard do gateway a partir de fluxo serial simulado com três drones em SF9: RSSI, SNR, PDR por drone e trajetórias reportadas

4.12.3. Discussão e limitações

O software do protótipo está completo e verificado, mas a atividade permanece parcial: nenhum ensaio físico foi realizado, de modo que RSSI, SNR, PDR real, alcance e temporização do ACK não foram medidos. Os resultados do dashboard validam o software de registro, não o enlace, pois provêm de um canal sintético; a simulação adota sincronismo ideal no TDMA. O protótipo opera em canal único: a conformidade plena ao Ato nº 14.448/2017 requer salto de frequência com no mínimo 35 canais ou o enquadramento como modulação digital, além da homologação (alterações posteriores do Ato não foram verificadas). O protocolo também não possui cifragem nem autenticação. O próximo passo é adquirir duas placas Heltec V3 e antenas, gravar o firmware e executar o ensaio de alcance e PDR descrito na subseção seguinte.

Código e evidências: repositório cerise-2026-lora-prototipo (privado)

4.13. Projeto dos módulos de comunicação para os drones

Esta etapa corresponde aos itens 3 e 4 do Objetivo 5 do plano de trabalho, que preveem a construção dos módulos de comunicação definitivos e a adequação dos drones para recebê-los. Foi executada apenas a parte de projeto: especificação do hardware, lista de materiais, diagrama de ligação, gabinete paramétrico, requisitos de integração, estimativas de enlace e de energia e procedimento de ensaio. A fabricação dos módulos, a instalação nos drones e as medições dependem da aquisição das placas de rádio e permanecem pendentes.

4.13.1. Arquitetura e requisitos

O módulo definitivo foi especificado como um modem LoRa transparente para o computador de bordo do drone, o mesmo que executa o planejamento de cobertura e a contagem de pessoas. O computador de bordo envia comandos de linha pela UART (3,3 V, GPIO47/48, 115200 bps) e recebe o log em linhas JSON, inclusive os downlinks `AREA_ASSIGN`; assim, o rádio não depende da controladora de voo, e a integração com MAVLink ou ROS 2 permanece no computador de bordo. A alimentação de 5 V vem de um BEC dedicado ou da placa de distribuição do frame, e a antena dipolo de 915 MHz fica na vertical, voltada para baixo, a pelo menos 10 cm do receptor GNSS e dos cabos dos ESCs. A lista de materiais prevê duas placas Heltec V3, antenas de 3 dBi (solo) e 2 dBi (drone), *pigtails* U.FL–SMA, baterias LiPo 1S, GPS opcional para o ensaio de alcance, BEC de 5 V e chicote JST-GH. A figura a seguir apresenta a ligação da bancada e a do módulo no drone.

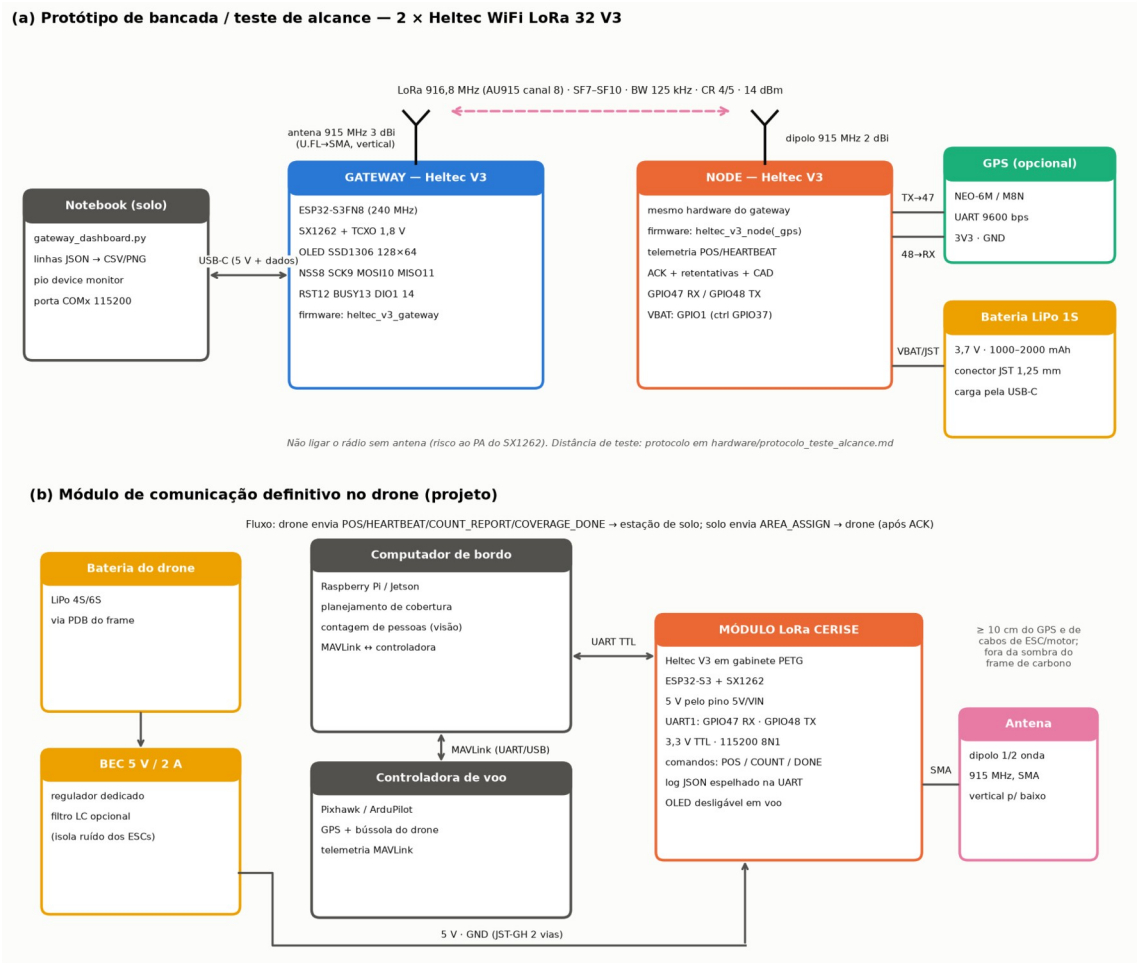


Figura 52. Diagrama de ligação: (a) protótipo de bancada com duas placas Heltec V3 e notebook de solo; (b) módulo definitivo no drone, com BEC de 5 V, UART para o computador de bordo e antena dipolo

Foram documentados quinze requisitos (R1 a R15), que cobrem mensagens, taxa de telemetria, ocupação do canal, alcance, entrega, acesso múltiplo, massa, dimensões, alimentação, interface, fixação, antena, regulamentação, robustez e segurança da informação. A tabela a seguir resume os principais e indica a natureza de cada valor.

Tabela 22. Principais requisitos do módulo de comunicação definitivo e natureza dos valores de projeto

Requisito	Valor de projeto	Natureza
Ocupação por quadro	≤ 400 ms (POS até SF10; AREA_ASSIGN até SF9)	calculado
Alcance	≥ 3 km com margem de 10 dB, drone a 60 m	estimativa
Acesso múltiplo	até 12 drones por canal em SF9 (TDMA)	simulação
Massa	≤ 40 g; gabinete de 12,9 g	meta; estimativa
Alimentação	5 V; 0,23 a 0,34 W médios	estimativa
Interface	UART 3,3 V, 115200 8N1, JST-GH 4 vias	especificação
Regulamentação	salto de frequência (≥ 35	pendente

	canais) e homologação	
Segurança	autenticação e cifragem AES-128-CCM	não implementado

4.13.2. Gabinete

O gabinete foi modelado de forma paramétrica em Python (trimesh e manifold3d), com base e tampa exportadas em STL. A base tem trilhos de apoio da placa, recortes para USB-C, conector SMA e passa-cabo, e quatro orelhas para parafusos M3 ou abraçadeiras, com furos espaçados de 30,5 mm no sentido do comprimento; a tampa tem janela para o display OLED e furos de ventilação sobre o amplificador de potência. As dimensões externas resultaram em 55,0 × 44,3 × 17,2 mm (internas de 51,4 × 26,7 × 14,0 mm). As malhas da base (7,68 cm³) e da tampa (2,45 cm³) são estanques, o que corresponde a 12,9 g de PETG com preenchimento total, valor tomado como limite superior. As dimensões da placa usadas no modelo (50,2 × 25,5 mm) ainda devem ser conferidas com paquímetro antes da impressão.

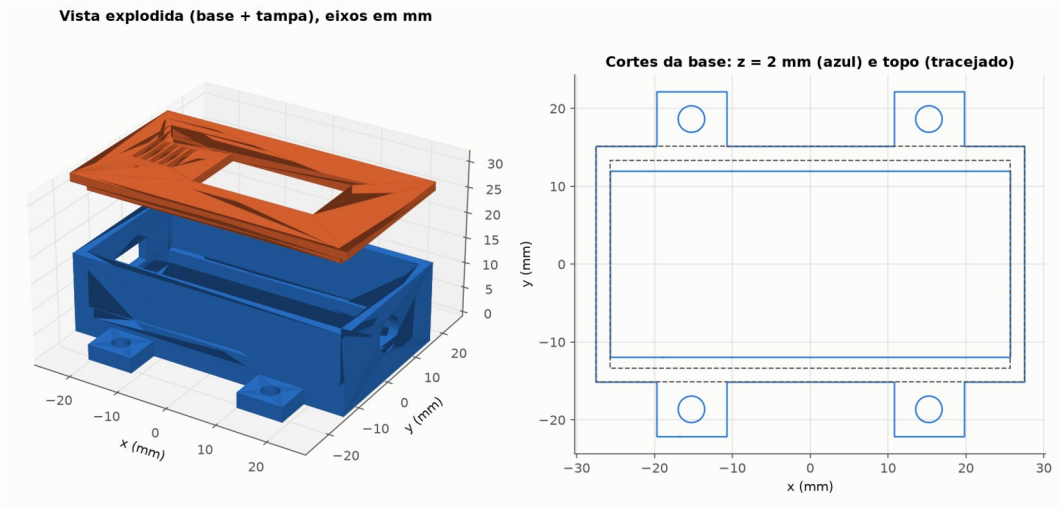


Figura 53. Gabinete paramétrico do módulo: vista explodida da base e da tampa e cortes da base com as orelhas de fixação M3

4.13.3. Enlace e energia estimados

O orçamento de enlace considerou 14 dBm, antenas de 2 dBi (drone) e 3 dBi (solo), 1,5 dB de perdas em cabos e conectores, margem de 10 dB, sensibilidade teórica do receptor e antena de solo a 2 m. Os alcances para o drone a 60 m foram estimados pelos modelos log-distância e Okumura–Hata [45], este com o drone tratado como antena elevada.

Tabela 23. Alcance estimado ar–solo por fator de espalhamento (14 dBm, drone a 60 m, margem de 10 dB)

SF	Sensibilidade (dBm)	Perda máx. (dB)	Log-dist. n = 2,7 (km)	Hata suburbano (km)	Hata urbano (km)

7	−124,5	132,0	5,21	4,24	2,12
8	−127,0	134,5	6,44	5,04	2,52
9	−129,5	137,0	7,97	5,99	3,00
10	−132,0	139,5	9,87	7,12	3,57

A altitude do drone tem efeito marcante: no modelo suburbano em SF9, o alcance estimado passa de 4,13 km a 30 m para 5,99 km a 60 m e 9,11 km a 120 m, conforme ilustrado abaixo. Esses valores são estimativas de projeto e precisam ser confirmados em campo.

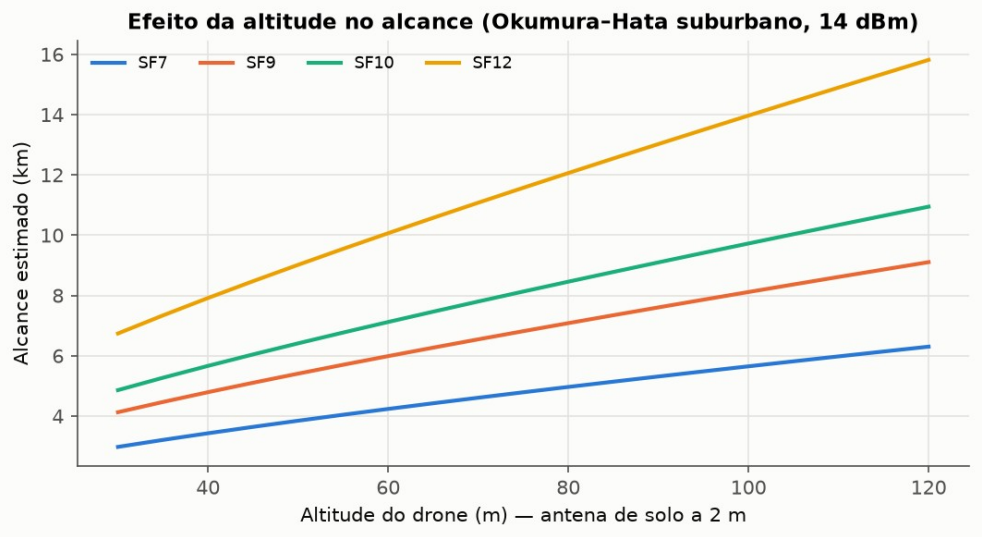


Figura 54. Efeito da altitude do drone no alcance estimado pelo modelo de Okumura–Hata suburbano, com 14 dBm e antena de solo a 2 m, para SF7, SF9, SF10 e SF12

O consumo foi estimado com correntes típicas de datasheet do SX1262 (recepção de 4,6 mA; transmissão de 45 mA a 14 dBm e de 118 mA a 22 dBm) e com a hipótese de 40 mA para o ESP32-S3 com Wi-Fi e Bluetooth desligados. O consumo médio resultou entre 0,23 W e 0,34 W na entrada de 5 V (SF7 a SF10, com e sem OLED), o que corresponde a 0,11 % a 0,17 % de uma potência média de voo de referência de 200 W.

4.13.4. Adequação dos drones e ensaio previsto

A adequação dos drones foi definida em nível de projeto: fixação do gabinete pelas orelhas M3 ou por abraçadeiras, com fita dupla-face de espuma para amortecimento; derivação de 5 V do BEC; chicote JST-GH de quatro vias (5 V, GND, TX e RX) até o computador de bordo; e posição da antena fora da sombra do frame de carbono. Para a validação, foi redigido um protocolo de ensaio de alcance e PDR com 200 quadros por ponto, distâncias de 50 m a 3 km e SF7, SF9 e SF10; com 200 quadros e PDR próxima de 0,9, a meia-largura do intervalo de confiança de 95 % é de aproximadamente 4,2 p.p. Para o firmware do módulo definitivo, foram previstos TDMA sincronizado por GPS, ACK apenas para mensagens críticas, salto de frequência sobre os canais AU915, ajuste de

SF e potência a partir do RSSI e do SNR devolvidos no ACK, e cifragem com autenticação.

4.13.5. Discussão e limitações

O projeto reúne os elementos necessários para a construção dos módulos assim que as placas forem adquiridas, e a interface com o computador de bordo já é compatível com o formato adotado pela frente de drones. Entretanto, nenhum módulo foi fabricado nem instalado, e os valores de alcance, massa e consumo são estimativas por modelo e por dados típicos de datasheet. A operação em voo também depende do salto de frequência, da homologação do equipamento e da implementação de autenticação, ainda não realizados.

Código e evidências: repositório cerise-2026-lora-prototipo (privado)

4.14. Segmentação da área de atuação entre drones

Esta etapa corresponde à Atividade 13 do Objetivo 5 do plano de trabalho, que prevê o desenvolvimento de um software funcional de segmentação da área de atuação entre os drones disponíveis. O problema tratado consiste em dividir uma área poligonal georreferenciada entre uma frota possivelmente heterogênea, de modo que as aeronaves concluam a varredura em tempos próximos, e em gerar, para cada drone, uma missão de cobertura executável por um piloto automático. A validação foi exclusivamente algorítmica e em planta (2D), com um modelo cinemático simplificado e com o polígono real de uma área do Campus Samambaia; não foram realizadas simulações 3D de voo (SITL com Gazebo ou Isaac) nem voos reais nesta etapa.

4.14.1. Metodologia e implementação

Foi desenvolvido o pacote Python `cerise_drones`, que recebe o polígono da área em GeoJSON, zonas proibidas com margem de segurança (10 m nos experimentos) e a descrição da frota (velocidade, autonomia, reserva de bateria de 20 %, campo de visão da câmera, altitude de 40 m e base). O planejamento é feito em coordenadas UTM, e o espaçamento s entre linhas de varredura corresponde a 80 % da largura da pegada da câmera em nádir (sobreposição lateral de 20 % e frontal de 70 %). Foram definidos três tipos de multirrotor com parâmetros representativos, sem correspondência com modelos comerciais: A (10 m/s, 20 min, 70° × 55°), B (6 m/s, 30 min, 70° × 55°) e C (14 m/s, 25 min, 82° × 52°).

Foram comparados quatro métodos de partição:

- **faixas ponderadas**: faixas paralelas à direção de varredura, com área proporcional ao peso de cada drone;
- **k-means + Voronoi**: células de Voronoi dos centroides de um k-means

sobre pontos da área livre, atribuídas aos drones pelo método húngaro [46]; trata-se da linha de base clássica, sem ponderação;

- **DARP em grade** [47]: atribuição das células de uma grade por distância geodésica a sementes, com balanceamento iterativo até erro de área inferior a 1 % do alvo; além da pesagem multiplicativa original, foi implementada uma pesagem aditiva que gera regiões conexas por construção, com retificação das fronteiras pelo algoritmo de Douglas–Peucker [48];
- **DARP + refinamento por tempo**, proposto neste trabalho: sementes nos centros de um k-means ponderado, pesos iniciais proporcionais à taxa de cobertura `v·s` e quatro iterações de reajuste `w ← w·(T̄/T)^0,8`, em que `T` é o tempo de missão estimado de cada drone, mantendo-se a solução de menor makespan (maior tempo de missão da frota).

Cada subárea recebe uma rota em boustrophedon [49] com a orientação de menor tempo estimado (subida, percurso, 4 s por curva e descida) e trânsito por grafo de visibilidade em torno das zonas proibidas. Uma checagem de bateria a cada 25 m insere retorno à base, troca de bateria (5 min) e retomada quando a autonomia útil não permite prosseguir. Para cada drone são exportados um plano `.plan` do QGroundControl (PX4, com waypoints, disparo da câmera por distância, retorno à base e geocerca), arquivos KML e GeoJSON e um mapa interativo. Os planos foram aprovados por um validador estrutural do formato, mas ainda não foram abertos no QGroundControl nem executados em SITL.

O benchmark utilizou o campo aberto do Campus Samambaia (polígono côncavo de 34,23 ha e 49 vértices, obtido do OpenStreetMap [50]) e três cenários sintéticos: convexo (96 ha), côncavo em L com uma zona proibida (43 ha) e côncavo em U com duas zonas proibidas (50 ha). Com N = 2 a 6 drones e frotas homogênea (somente tipo A) e heterogênea (tipos A, B e C alternados), foram realizadas 160 execuções, além de 70 execuções de ablação. A suíte de 16 testes automatizados (partições, balanceamento, cobertura, trocas de bateria, validade dos `.plan`, protocolo LoRa e georreferenciamento) foi aprovada integralmente.

4.14.2. Resultados

A tabela a seguir resume a partição do campo aberto do Campus Samambaia entre quatro drones heterogêneos (dois do tipo A, um B e um C).

Tabela 24. Partição do campo aberto do Campus Samambaia (34,2 ha) entre 4 drones heterogêneos pelos quatro métodos

Método	Makespan (min)	CV do tempo	Distância (km)	Curvas	Cobertura (%)
Faixas ponderadas	8,63	0,120	13,36	56	100,0
k-means +	10,63	0,385	10,65	70	100,0

Voronoi					
DARP (sementes nas bases)	8,18	0,152	11,66	77	100,0
DARP + refinamento	6,57	0,002	11,37	73	100,0

No plano final (DARP + refinamento), o drone do tipo C recebeu 13,46 ha, os dois do tipo A receberam 7,07 ha e 9,20 ha e o do tipo B, 4,50 ha. Os quatro concluem a missão entre 6,54 e 6,57 min, com uso de 22 % a 33 % da bateria, e a sobreposição entre drones foi de 7,0 % das células rasterizadas. As figuras a seguir mostram a partição pelos quatro métodos e o plano final sobre imagem de satélite.

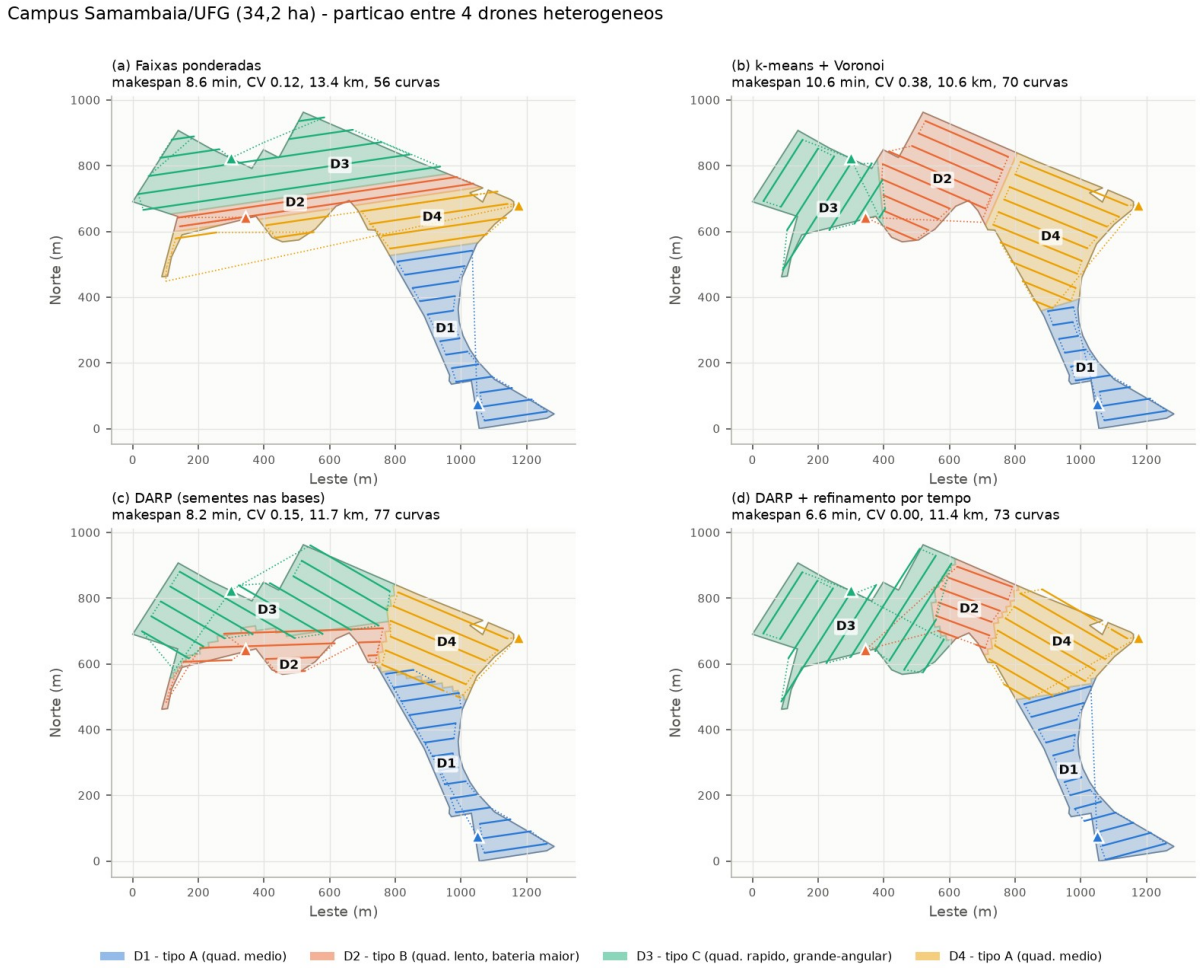


Figura 55. Partição do campo aberto do Campus Samambaia entre 4 drones heterogêneos pelos quatro métodos, com as rotas em boustrophedon (linhas cheias: câmera ligada; pontilhadas: trânsito e curvas) e as bases indicadas por triângulos

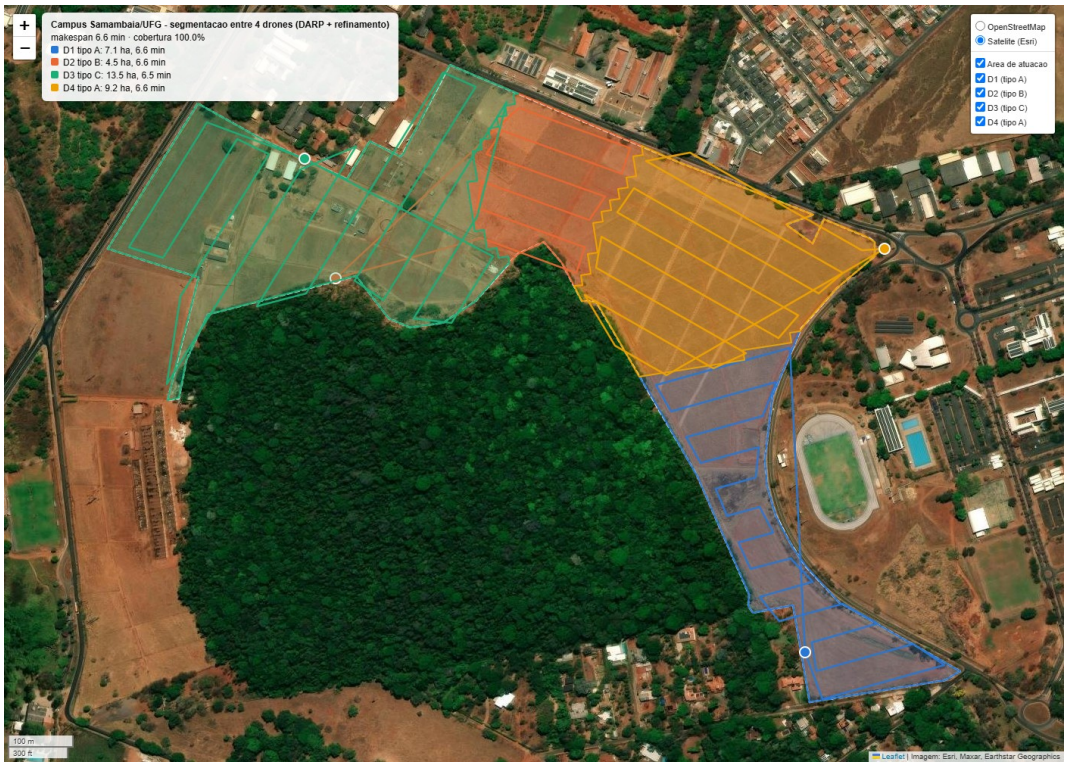


Figura 56. Mapa interativo do plano final (DARP + refinamento) sobre imagem de satélite, com subárea, rota e base de cada drone; legenda com área e tempo de missão por drone

Tabela 25. Médias do benchmark sobre os 4 cenários e N = 2 a 6 (20 casos por linha; vitórias = casos com o menor makespan)

Frota	Método	Makespan médio (min)	Makespan / melhor do caso	CV do tempo	Vitórias
Homogênea	Faixas	11,66	1,153	0,097	2
Homogênea	k-means	11,47	1,100	0,117	2
Homogênea	DARP	11,37	1,107	0,055	1
Homogênea	DARP + ref.	10,64	1,008	0,023	15
Heterogênea	Faixas	12,68	1,189	0,123	3
Heterogênea	k-means	15,90	1,504	0,308	0
Heterogênea	DARP	12,04	1,119	0,075	1
Heterogênea	DARP + ref.	10,96	1,007	0,028	16

O DARP com refinamento por tempo obteve o menor makespan em 31 dos 40 casos, ficou em média a 0,7–0,8 % do melhor resultado de cada caso e reduziu o coeficiente de variação (CV) dos tempos de missão para 0,023–0,028, contra 0,055–0,308 dos demais métodos. Com frota heterogênea, o k-means/Voronoi, por não ser ponderado, ficou em média 50 % acima do melhor makespan. As faixas ponderadas fizeram menos curvas (49 a 50 em média, contra 71 a 78 dos demais métodos), mas perderam em trânsito nos polígonos côncavos. A cobertura rasterizada ficou entre 99,65 % e 100 % nas 160 execuções. As medianas de tempo de execução foram de 0,4 a 1,2 s para os métodos de passo único e de 5,8 a 7,1 s para o DARP + refinamento, que

executa cinco partições completas; o máximo observado (29,7 s) inclui interferência de outras tarefas no notebook compartilhado.

A ablação (cenários UFG e U, frota heterogênea, N = 2 a 6) mostrou que a pesagem multiplicativa original não convergiu no polígono real da UFG (erro máximo de área de 24,9 %, contra 0,6 % a 1,2 % da pesagem aditiva); que sementes em centros de k-means reduziram o makespan médio em relação a sementes nas bases (8,61 contra 8,67 min na UFG; 11,44 contra 12,38 min no U); e que o refinamento por tempo foi o fator de maior ganho (8,13 min na UFG e 9,89 min no U, com CV de 0,011 e 0,041).

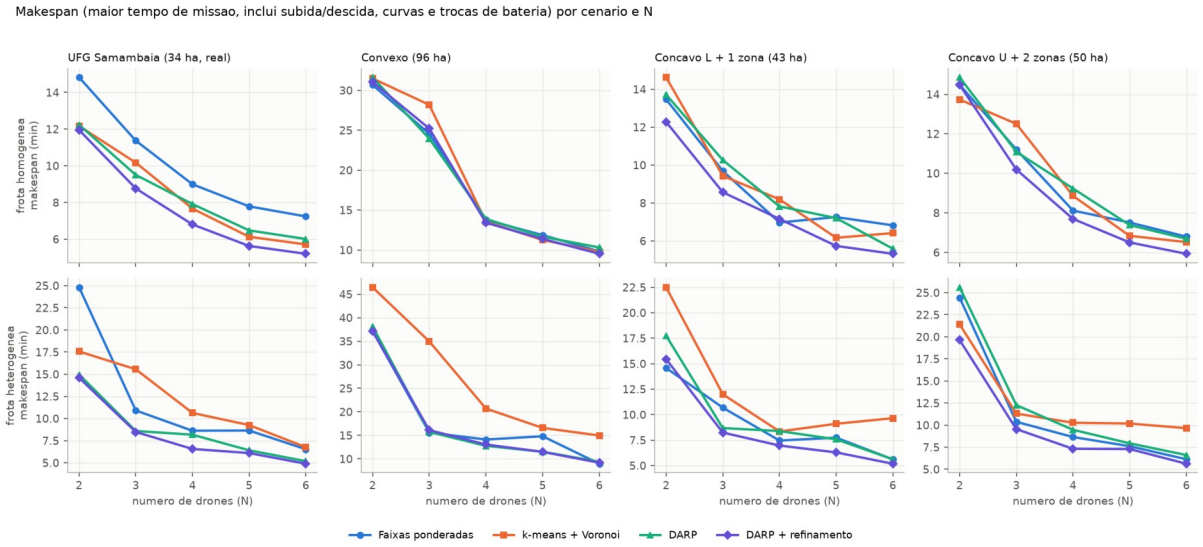


Figura 57. Makespan em função do número de drones para os quatro métodos, por cenário (colunas) e tipo de frota (linhas)

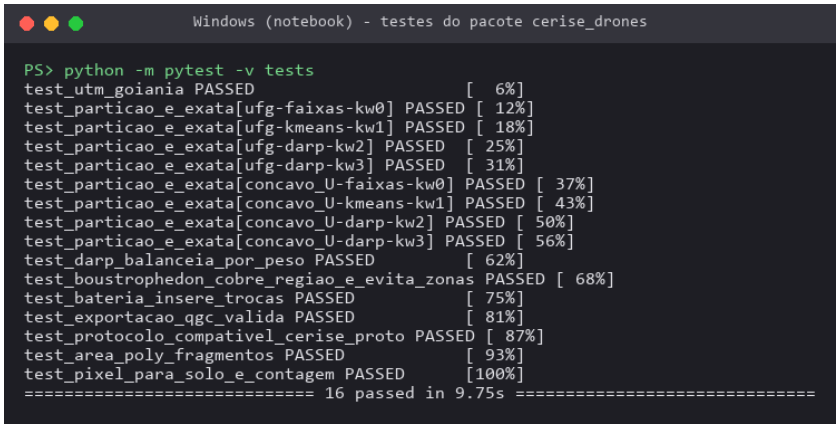


Figura 58. Saída de terminal da suíte de testes do pacote cerise_drones, com os 16 testes aprovados

4.14.3. Discussão e limitações

Os resultados indicam que, em frotas heterogêneas, a ponderação pela capacidade de cobertura e, sobretudo, o ajuste dos pesos pelo tempo estimado de missão são determinantes para o balanceamento. As principais limitações

são: (i) os tempos de missão derivam de um modelo cinemático simples (velocidade constante, penalidade fixa por curva, energia proporcional ao tempo e ausência de vento), não calibrado com voos; (ii) a validação foi feita apenas em planta, sem SITL nem voo real; e (iii) o k-means/Voronoi foi usado sem ponderação, e uma versão ponderada (diagrama de potência) reduziria a diferença em frotas heterogêneas.

Código e evidências: repositório cerise-2026-drones-segmentacao (privado)

4.15. Colaboração entre drones e realocação dinâmica

Esta etapa trata dos algoritmos de colaboração entre drones previstos no plano de trabalho e complementa a Atividade 13 do Objetivo 5. O objetivo foi tratar o abandono da missão por um drone, seja por falha, seja por retorno antecipado devido a bateria baixa, redistribuindo a área ainda não varrida entre os sobreviventes, e quantificar o custo de comunicação dessa coordenação no enlace LoRa desenvolvido no projeto. Toda a avaliação foi feita por simulação 2D, com o mesmo modelo cinemático da etapa de segmentação.

4.15.1. Metodologia e implementação

O cenário de referência é o plano final da UFG com quatro drones heterogêneos (DARP + refinamento), cujo makespan sem falha é de 394 s (6,57 min). A falha é detectada pela ausência de mensagens HEARTBEAT, com atraso de 3 s. Foram comparadas três estratégias:

- **sem realocação**: a parte não varrida da subárea do drone perdido é abandonada;
- **repartição**: a área ainda não coberta por nenhum drone é repartida entre os sobreviventes a partir de suas posições atuais; cada componente conexa recebe um subconjunto de drones, escolhido por enumeração das alocações, e passa pelo DARP com refinamento pelo instante de conclusão, considerando a bateria já consumida; as novas subáreas são difundidas por mensagens AREA_ASSIGN e AREA_POLY;
- **leilão**: a área órfã é dividida em tarefas (células de 2s × 2s) distribuídas por leilão sequencial de item único com objetivo minimax [51], por meio de mensagens TASK_BID e TASK_AWARD; as tarefas ganhas são varridas após a própria subárea.

Foram simuladas 12 falhas: cada um dos quatro drones abandonando a missão a 25 %, 50 % e 75 % do seu voo.

As mensagens foram alinhadas à biblioteca `cerise_proto` v1 do protótipo LoRa apresentado anteriormente. O módulo `cerise_drones.protocolo` codifica

e decodifica os mesmos quadros byte a byte (cabeçalho de 8 B, payload de até 32 B e CRC-16/CCITT-FALSE, cujo valor de verificação 0x29B1 é conferido nos testes). Como o AREA_ASSIGN v1 descreve apenas um retângulo envolvente, insuficiente para as subáreas poligonais do DARP, foi proposta uma extensão com os tipos 7 (AREA_POLY, vértices em deslocamentos inteiros de metros, até 7 por quadro, com fragmentação), 8 (TASK_BID) e 9 (TASK_AWARD), incorporada à biblioteca do protótipo LoRa, mas ainda não exercitada com rádios. Os tempos no ar foram calculados pela formulação da Semtech [43] (BW 125 kHz, CR 4/5, preâmbulo de 8 símbolos).

4.15.2. Resultados

Tabela 26. Efeito do abandono de um drone no plano da UFG com 4 drones heterogêneos: área perdida sem realocação, aumento do makespan por estratégia e custo LoRa sem ACKs (4 falhas por instante)

Instante da falha	Perda sem realocação (%)	Makespan, repartição (%)	Makespan, leilão (%)	Quadros LoRa (rep./leilão)	No ar em SF9, s (rep./leilão)
25 % do voo	16,7	+33,3	+49,2	20,0 / 59,8	5,5 / 10,5
50 % do voo	9,5	+18,3	+32,2	15,0 / 38,5	4,0 / 6,8
75 % do voo	1,4	+17,8	+12,2	8,0 / 12,8	2,0 / 2,3
Média (12 casos)	9,2 (máx. 26,4)	+23,1	+31,2	14,3 / 37,0	3,8 / 6,5

Sem realocação, o makespan não se altera, mas em média 9,2 % da área permanece descoberta, chegando a 26,4 % quando o drone rápido do tipo C abandona a missão a 25 % do voo. As duas estratégias colaborativas recuperaram pelo menos 99,7 % de cobertura em todos os casos. A repartição teve menor aumento de makespan em 9 dos 12 casos; o leilão foi melhor em três casos, dois deles com falha a 75 % do voo, quando a área órfã é pequena e os sobreviventes não abandonam as próprias rotas, e um terceiro praticamente empatado (+75,8 % nas duas estratégias). Uma política híbrida que escolhesse a melhor das duas teria aumento médio de 20,6 %. O replanejamento levou em média 1,95 s na repartição e 0,08 s no leilão.

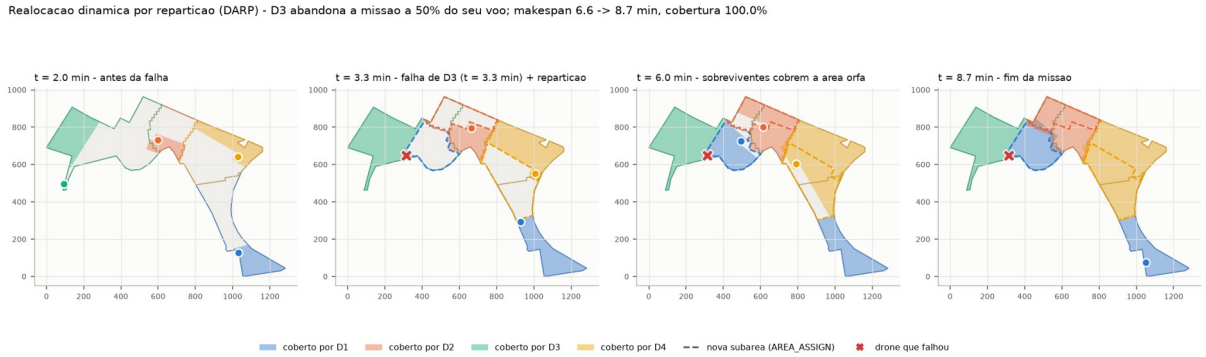


Figura 59. Linha do tempo da realocação por repartição quando o drone D3 abandona a missão a 50 % do voo: áreas já cobertas coloridas por drone e novas subáreas tracejadas; makespan de 6,6 para 8,7 min, com 100 % de cobertura

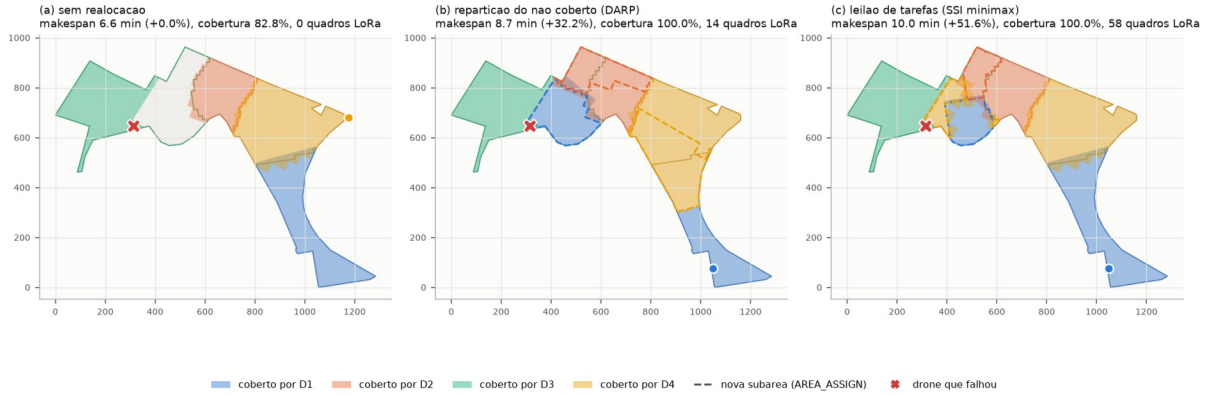


Figura 60. Estado final das três estratégias para a falha de D3 a 50 % do voo: sem realocação (82,8 % de cobertura), repartição (+32,2 % de makespan, 14 quadros LoRa) e leilão (+51,6 %, 58 quadros)

Tabela 27. Tamanho e tempo no ar das mensagens de coordenação (BW 125 kHz, CR 4/5)

Mensagem	Quadro (B)	SF7 (ms)	SF9 (ms)	SF10 (ms)	Origem
HEARTBEAT	18	51,5	185,3	329,7	cerise_proto v1
POS	24	61,7	205,8	370,7	cerise_proto v1
AREA_ASSIGN	32	71,9	246,8	452,6	cerise_proto v1
COVERAGE_DONE	17	51,5	164,9	329,7	cerise_proto v1
COUNT_REPORT	23	61,7	205,8	370,7	cerise_proto v1
ACK	16	51,5	164,9	329,7	cerise_proto v1
AREA_POLY (7 vértices)	42	87,3	287,7	534,5	extensão proposta
TASK_BID	14	46,3	164,9	288,8	extensão proposta
TASK_AWARD	13	46,3	164,9	288,8	extensão proposta

Em SF9, configuração padrão do firmware do protótipo LoRa, todos os

quadros ficam abaixo do limite de 400 ms de ocupação adotado no protótipo; em SF10, AREA_ASSIGN e AREA_POLY o ultrapassam. Em média, uma repartição custou 14,3 quadros (537 B e 3,8 s no ar em SF9) e um leilão, 37,0 quadros (608 B e 6,5 s), sem contar confirmações, conforme ilustrado abaixo.

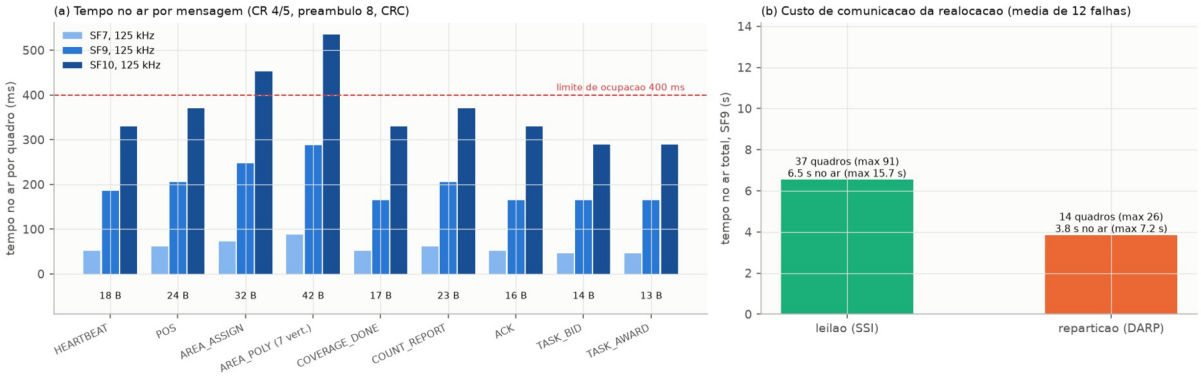


Figura 61. Tempo no ar de cada mensagem em SF7, SF9 e SF10, com o limite de 400 ms, e custo médio de comunicação das estratégias de realocação em SF9 nas 12 falhas simuladas

4.15.3. Discussão e limitações

A repartição é mais vantajosa quando a falha ocorre cedo e a área órfã é grande, pois redistribui o trabalho de forma equilibrada e exige menos mensagens; o leilão é competitivo para áreas órfãs pequenas, o que sugere uma política híbrida. A simulação assume detecção da falha em 3 s e canal sem perdas, e o custo LoRa foi contado sem ACKs nem retransmissões. A realocação não foi testada de ponta a ponta com os módulos de rádio nem em voo, e a política híbrida não foi implementada.

Código e evidências: repositório cerise-2026-drones-segmentacao (privado)

4.16. Detecção e contagem de pessoas em imagens aéreas

Esta etapa corresponde ao item 1 do Objetivo 5 do plano de trabalho, que trata do planejamento e da adequação de um projeto com drones para detecção e contagem de pessoas. Foram validados os pipelines de detecção e de contagem em imagens aéreas reais do conjunto público VisDrone2019-DET [52], desenvolvido o método de agregação das contagens entre drones e levantados os requisitos de adequação das aeronaves. Não foram coletadas imagens próprias do Campus Samambaia, e a construção e a instalação do hardware embarcado permanecem pendentes.

4.16.1. Dados, ambiente e treinamento

As categorias *pedestrian* e *people* do VisDrone2019-DET foram unidas em uma única classe *pessoa*. O conjunto de treino tem 6.471 imagens e 106.396 pessoas; o de validação, 548 imagens e 13.969 pessoas; e o test-dev, 1.610

imagens e 27.382 pessoas. As imagens de treino foram reduzidas para 960 px no lado maior, enquanto as de validação e de test-dev foram mantidas na resolução original.

O treinamento e a avaliação foram executados na estação NVIDIA DGX Spark (GB10), em um contêiner derivado da imagem NGC PyTorch 26.05 com a biblioteca Ultralytics 8.4.189 [53], com GPU compartilhada com outros processos. O uso de memória de GPU foi limitado a 7,5 GB, e o pico medido foi de 6,5 GB. No GB10, o *autocast* em FP16 produziu valores NaN e caixas espúrias, razão pela qual treino e inferência foram realizados em FP32. O ajuste fino partiu do YOLO11s com pesos COCO, com imagens de 960 px, lote 4, otimizador AdamW (taxa de aprendizado 0,002) e limite de tempo de 0,8 h, que resultou em 5 épocas em 48,7 min. A figura a seguir mostra as perdas e o mAP de pessoa na validação ao longo das épocas.

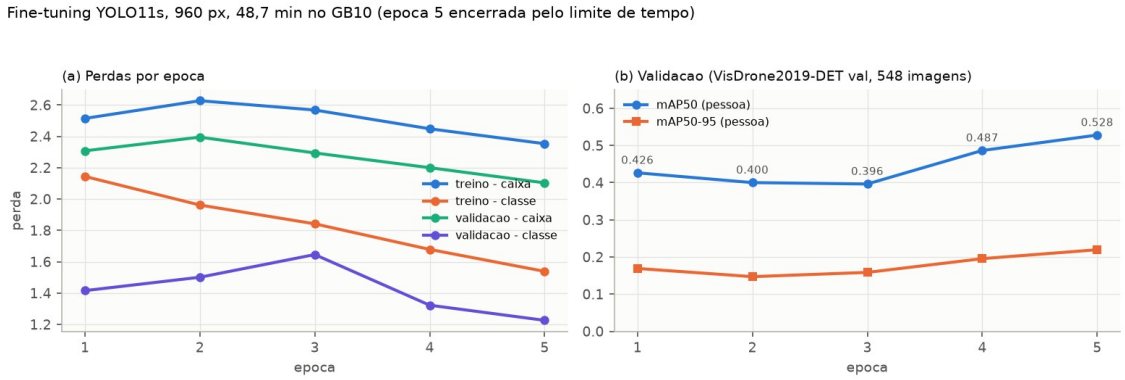


Figura 62. Curvas do ajuste fino do YOLO11s a 960 px no GB10: perdas de caixa e de classe por época e mAP50 e mAP50-95 de pessoa no conjunto de validação

4.16.2. Resultados de detecção e contagem

Tabela 28. Classe pessoa no VisDrone2019-DET (validador Ultralytics, FP32, conf 0,001, IoU 0,7)

Modelo	Resolução (px)	Conjunto	mAP50	mAP50-95	Revocação
YOLO11n COCO	960	val	0,274	0,111	0,285
YOLO11m COCO	960	val	0,433	0,194	0,401
YOLO11s COCO	1280	val	0,443	0,201	0,417
YOLO11s COCO	960	val	0,384	0,163	0,363
YOLO11s ajustado	960	val	0,528	0,220	0,473
YOLO11s COCO	960	test-dev	0,183	0,077	0,192
YOLO11s ajustado	960	test-dev	0,316	0,121	0,312

As linhas do YOLO11s a 960 px foram avaliadas com até 1.000 detecções por imagem; as demais, com o padrão de 300 (nessa condição, o YOLO11s COCO a 960 px obtém mAP50 de 0,372 no val). O ajuste fino elevou o mAP50 de pessoa de 0,384 para 0,528 no val (+37 %) e de 0,183 para 0,316 no test-dev (+73 %), superando o YOLO11m COCO a 960 px e o YOLO11s COCO a 1280 px.

Para a contagem, considera-se o número de detecções com confiança acima de um limiar escolhido no val (mínimo erro absoluto médio, MAE) e aplicado ao test-dev. Detecções com centro nas regiões ignoradas da anotação do VisDrone foram descartadas, e o erro percentual absoluto médio (MAPE) foi calculado apenas sobre as imagens com pelo menos uma pessoa.

Tabela 29. Erro de contagem de pessoas por imagem, com limiar escolhido no val e aplicado ao test-dev

Modelo	Limiar	MAE val	MAPE val (%)	MAE test-dev	MAPE test-dev (%)
YOLO11s COCO	0,05	9,91	44,8	11,44	81,7
YOLO11s ajustado	0,175	6,91	31,3	8,76	61,2

O modelo ajustado reduziu o MAE em cerca de 30 % no val e 23 % no test-dev. O viés é de subcontagem nos dois conjuntos: o erro do total de pessoas foi de −13,8 % no val e −42,8 % no test-dev, que concentra pessoas muito pequenas e imagens noturnas. Para o modelo COCO, o mínimo do MAE ocorreu no limite inferior da grade de limiares (0,05).

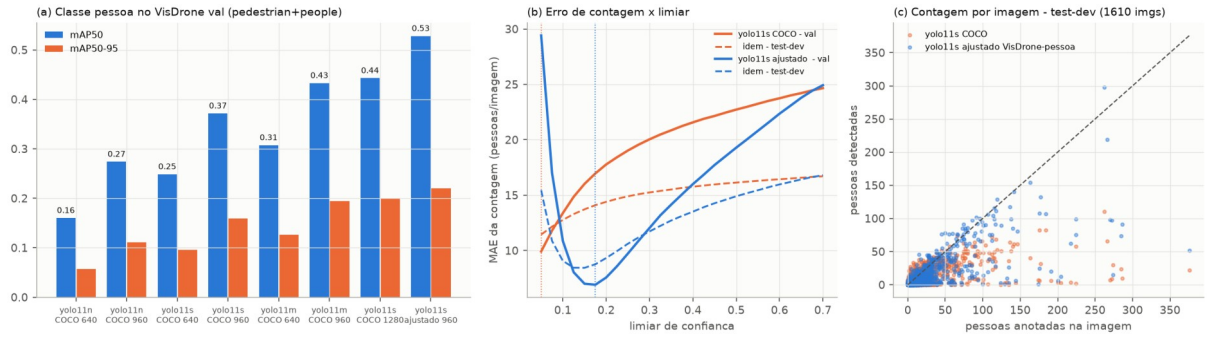


Figura 63. Métricas de detecção e contagem: mAP de pessoa no val para os modelos COCO (com o limite padrão de 300 detecções) e o ajustado; MAE da contagem em função do limiar (val e test-dev); e pessoas detectadas versus anotadas por imagem no test-dev



Figura 64. Exemplos do VisDrone val com as detecções do modelo ajustado (limiar 0,175) e as contagens real e detectada por imagem

Com lote 1, FP32 e 150 imagens pré-carregadas, o YOLO11s ajustado atingiu 25,5 imagens/s de ponta a ponta a 960 px (39,2 ms por imagem, dos quais 18,2 ms de inferência) e 31,0 imagens/s a 640 px; a 960 px, o YOLO11n atingiu 28,8 imagens/s e o YOLO11m, 12,9. CPU e GPU estavam compartilhadas com outros processos. Nos planos exportados para a UFG, um drone do tipo A dispara uma foto a cada 12,5 m a 10 m/s (0,8 foto/s), e a soma para os quatro drones é de cerca de 3,3 fotos/s; a vazão medida é cerca de 8 vezes superior, o que permite processar as imagens de vários drones em uma única estação de solo.

```
spark-a55b (DGX Spark GB10): avaliacao deteccao/contagem de pessoas

$ bash deteccao/run.sh avaliacao 06_avaliar_tudo.py
===== 03_avaliar_coco.py
yolo11n.pt                          640 val   mAP50=0.160 mAP50-95=0.057 P=0.438 R=0.181
yolo11n.pt                          960 val   mAP50=0.274 mAP50-95=0.111 P=0.488 R=0.285
yolo11s.pt                          640 val   mAP50=0.249 mAP50-95=0.096 P=0.484 R=0.269
yolo11s.pt                          960 val   mAP50=0.372 mAP50-95=0.159 P=0.567 R=0.363
yolo11m.pt                          640 val   mAP50=0.307 mAP50-95=0.126 P=0.575 R=0.300
yolo11m.pt                          960 val   mAP50=0.433 mAP50-95=0.194 P=0.632 R=0.401
yolo11s.pt                         1280 val   mAP50=0.443 mAP50-95=0.201 P=0.612 R=0.417
===== 05_avaliar_ft.py
yolo11s COCO (pre-treinado)         960 val   mAP50=0.384 mAP50-95=0.163 P=0.567 R=0.363
yolo11s COCO (pre-treinado)         960 test  mAP50=0.183 mAP50-95=0.077 P=0.447 R=0.192
yolo11s ajustado VisDrone-pessoa    960 val   mAP50=0.528 mAP50-95=0.220 P=0.638 R=0.473
yolo11s ajustado VisDrone-pessoa    960 test  mAP50=0.316 mAP50-95=0.121 P=0.506 R=0.312
CONTAGEM yolo11s COCO (pre-treinado): limiar(val)=0.05  val MAE=9.91 MAPE=44.8%  | test-dev MAE=11.44 MAPE=81.7%
CONTAGEM yolo11s ajustado VisDrone-pessoa: limiar(val)=0.175  val MAE=6.91 MAPE=31.3%  | test-dev MAE=8.76 MAPE=61.2%
FPS yolo11s ajustado   imgsz= 640 lote=1 FP32:  31.0 FPS (32.3 ms/img; inferencia 11.2 ms)
FPS yolo11s ajustado   imgsz= 960 lote=1 FP32:  25.5 FPS (39.2 ms/img; inferencia 18.2 ms)
FPS yolo11n COCO       imgsz= 640 lote=1 FP32:  34.9 FPS (28.7 ms/img; inferencia 6.0 ms)
FPS yolo11n COCO       imgsz= 960 lote=1 FP32:  28.8 FPS (34.7 ms/img; inferencia 11.9 ms)
FPS yolo11m COCO       imgsz= 640 lote=1 FP32:  23.4 FPS (42.8 ms/img; inferencia 19.6 ms)
FPS yolo11m COCO       imgsz= 960 lote=1 FP32:  12.9 FPS (77.5 ms/img; inferencia 47.9 ms)
FIM
```

Figura 65. Saída de terminal da avaliação no DGX Spark: mAP dos modelos, erro de contagem e vazão em imagens por segundo

4.16.3. Agregação das contagens entre drones

Como as subáreas são disjuntas, a contagem total é a soma das contagens de cada subárea, desde que cada pessoa seja contada uma única vez. O método implementado (`cerise_drones.agregacao`) georreferencia cada detecção pela posição, altitude e rumo do drone no instante da foto (câmera em nádir e terreno plano); atribui a detecção apenas ao drone dono da subárea em que o ponto cai (regra de posse); agrupa detecções a menos de um raio `r` para eliminar repetições causadas pela sobreposição entre fotos; e, em uma faixa de fronteira de largura `max(3 m, 2r)`, faz o agrupamento entre drones, que trocariam as detecções de borda por mensagens COUNT_REPORT.

A lógica foi validada apenas por simulação, com 348 pessoas sintéticas estáticas (300 distribuídas uniformemente e 6 grupos de 8 sobre as fronteiras) no plano da UFG, com 619 fotos e ruído de georreferenciamento σ. Com detector ideal, σ = 0,8 m e r = 3 m, a soma ingênua superestimou a contagem em 300,5 %, a deduplicação sem regra de posse em 6,8 ± 3,4 %, e o método proposto errou −1,1 ± 1,8 %. A regra de posse deslocou a estimativa para baixo em 7 a 20 pontos percentuais em todas as configurações simuladas, o que reduz o erro quando há sobrecontagem, mas acentua a subcontagem quando o raio é grande em relação a σ (com σ = 0,3 m e r = 2 m, de +4,0 % para −6,8 %). Com a revocação e a precisão medidas (0,473 e 0,638) e falsos positivos independentes entre fotos, hipótese pessimista, o erro foi de +102,7 %, dominado pelos falsos positivos.

4.16.4. Requisitos para a adequação dos drones

A adequação física dos drones não foi executada por depender de hardware. Além do módulo LoRa descrito anteriormente, foram levantados como requisitos: câmera em nádir com pelo menos 12 MP (resolução no solo inferior a 1,5 cm/pixel a 40 m) e disparo por distância; processamento das imagens na estação de solo após o pouso, cenário validado nesta etapa, ou em computador embarcado, o que exige revisar massa e autonomia; e GNSS com correção RTK ou PPK, pois um erro de georreferenciamento menor permite raios de deduplicação de 1 a 2 m.

4.16.5. Discussão e limitações

O ajuste fino foi curto (5 épocas) e o VisDrone contém majoritariamente imagens oblíquas de cenas urbanas, distintas das imagens em nádir a 40 m previstas para o campus; o mAP50 de 0,316 e a subcontagem de 42,8 % no test-dev indicam limites de generalização. O FP16 não pôde ser usado no GB10 com a versão de PyTorch empregada. A agregação foi validada apenas com pessoas sintéticas e estáticas, e o resultado com o detector medido indica a necessidade de confirmação multivista ou de calibração da contagem antes do uso

operacional. Os próximos passos incluem coletar e anotar imagens próprias do campus e treinar por mais épocas.

Código e evidências: repositório cerise-2026-drones-segmentacao (privado)

5. COMPARAÇÃO ENTRE O PLANO ORIGINAL E O EXECUTADO

A tabela a seguir compara as 14 atividades do plano de trabalho com o que foi efetivamente entregue. A situação de cada atividade foi atribuída a partir dos resultados medidos e das evidências guardadas nos repositórios: considera-se concluída a atividade cuja entrega prevista foi produzida e verificada, ainda que com ressalvas, e parcial aquela em que parte essencial da entrega não pôde ser realizada.

Tabela 30. Situação das atividades do plano de trabalho 2026

Atividade	Entrega prevista	Situação	Observação
1. Análise comparativa de modelos de geração 3D	Comparação de redes de geração 3D com textura e seleção do modelo	Concluída	10 modelos levantados e 6 medidos; 4 bloqueados por falta de versão aarch64 ou acesso restrito
2. Aplicativo com geração 3D	Versão funcional do aplicativo com objetos 3D texturizados	Concluída	Validada no emulador Android contra o servidor real; sem aparelho físico
3. Aplicativo otimizado e relatório de desempenho	Otimização e relatório de desempenho	Parcial	Tempos medidos; tamanhos de GLB a remedir após a correção da textura
4. Validação	Relatório de validação e resultados experimentais	Concluída	58/58 testes; sem fotos tiradas pela câmera do celular
5. Integração e servidor	Análise de gargalos e servidor para múltiplas requisições	Concluída	Erro de 90,9 % para 0 %; servidor não exposto publicamente
6. Play Store	Publicação do aplicativo	Parcial	Pacote pronto e verificado; publicação não realizada
7. Dataset	Dataset estruturado e documentado	Concluída	Braço virtual; 5.000 poses e 30.000 imagens
8. Resultados do modelo	Resultados do modelo de pose de 2025 no novo dataset	Concluída	ADD@60 mm de 68,0 %; dados sintéticos
9. Avaliação	Avaliação de precisão e robustez	Concluída	Somente em simulação, sem imagens reais
10. Protótipo LoRa	Envio e recebimento entre dois dispositivos	Parcial	Software verificado sem rádio; ensaio físico pendente
11. Modelo digital	Modelo físico e	Concluída	Massas e inércias não

	cinemático do braço		calibradas
12. Gêmeo digital	Gêmeo com o modelo de pose integrado	Concluída	Validado somente em simulação
13. Segmentação de áreas	Software funcional de segmentação entre drones	Concluída	Validação 2D; sem SITL nem voo
14. Relatório e artigo	Relatório técnico-científico	Concluída (este relatório)	Resultados rastreáveis aos repositórios

Das 14 atividades, 11 foram concluídas e 3 ficaram parciais. Todas as conclusões, porém, têm ressalvas que precisam ser explicitadas. As atividades com robótica foram realizadas inteiramente em simulação, com o braço virtual servo6dof, e a validação com o braço físico e com imagens reais não foi feita. A segmentação de áreas e a colaboração entre drones foram validadas apenas em nível algorítmico, em planta, sem simulação de voo nem voo real. Os tempos de geração 3D, de servidor e de inferência foram medidos numa estação compartilhada com serviços de IA em produção e devem ser lidos como limites superiores.

As pendências decorrem de quatro causas principais. A primeira é a falta de hardware: as placas LoRa e as antenas não foram adquiridas, de modo que o protótipo LoRa (Atividade 10) não passou pelo ensaio físico entre dois dispositivos e, no Objetivo 5, a construção dos módulos de comunicação definitivos e a adequação dos drones ficaram apenas em projeto, assim como a adequação física dos drones para a detecção e a contagem de pessoas, cujo pipeline foi validado com o conjunto público VisDrone. A segunda são as ações que exigem autenticação do bolsista: a publicação na Play Store (Atividade 6) depende do Play Console, e a conclusão do benchmark de geração 3D depende do aceite de licenças no Hugging Face. A terceira é a decisão, ainda não tomada, sobre a exposição pública do servidor de geração 3D e do visualizador do gêmeo digital: nada foi exposto à internet, e o aplicativo só funcionará fora da rede interna depois dessa exposição; a publicação da política de privacidade no endereço público previsto também aguarda decisão. A quarta é um defeito encontrado no fim do período: a recodificação da textura em JPEG não ocorria no pipeline empacotado, o que invalidou os tamanhos de arquivo medidos na rodada de *presets* (Atividade 3). O defeito foi corrigido no servidor e no repositório, mas a rodada ainda precisa ser refeita.

6. PRÓXIMOS PASSOS

Os próximos passos foram priorizados pelo quanto destravam as entregas pendentes:

- **Decisões e ações do bolsista**: decidir sobre a exposição pública do servidor de geração 3D e do visualizador do gêmeo digital; publicar a

política de privacidade no endereço previsto; criar o aplicativo no Play Console e conduzir o teste interno, o teste fechado e o pedido de produção; aceitar no Hugging Face as licenças do Stable Fast 3D, do SPAR3D e do DINOv3 para completar o benchmark; e definir o acesso dos avaliadores aos repositórios privados.

- **Aquisição de hardware e ensaios de rádio**: adquirir as placas Heltec WiFi LoRa 32 V3 e as antenas da lista de materiais, gravar o firmware, executar o ensaio de alcance e de taxa de entrega e medir o consumo e a massa reais do módulo; em seguida, implementar o TDMA sincronizado por GPS, o salto de frequência e a cifragem com autenticação, encaminhar a homologação, testar a realocação entre drones com os rádios e instalar os módulos nos drones.
- **Remedições e consolidação**: reconstruir a imagem Docker do pipeline com a correção da textura e refazer, numa janela ociosa da GPU, a rodada de *presets*, os tamanhos de GLB e a suíte de carga do servidor; exigir textura JPEG nos testes automatizados; recalibrar a estimativa de tempo do *preset* alto; e ligar o monitoramento das métricas do servidor.
- **Validação em condições reais**: testar o aplicativo em aparelhos físicos Android e iOS, com fotografias da câmera, rede móvel e RA dos modelos gerados; capturar imagens do braço servo6dof físico para aplicar a etapa autossupervisionada de 2025 e validar o modelo de pose e o gêmeo digital fora da simulação, incluindo pontos-chave nas pinças para estimar a rotação e a abertura da garra; executar as missões de drones no QGroundControl e em SITL antes de voos no Campus Samambaia; e coletar imagens próprias em nádir para ajustar o detector de pessoas.

7. REFERÊNCIAS BIBLIOGRÁFICAS

[1] AZUMA, R. T. A survey of augmented reality. Presence: Teleoperators and Virtual Environments, v. 6, n. 4, 1997.

[2] BILLINGHURST, M.; CLARK, A.; LEE, G. A survey of augmented reality. Foundations and Trends in Human-Computer Interaction, v. 8, n. 2-3, 2015.

[3] CHEN, P.; LIU, X.; CHENG, W.; HUANG, R. A review of using augmented reality in education from 2011 to 2016. In: Innovations in Smart Learning. Singapore: Springer, 2017.

[4] BARSOM, E. Z.; GRAAFLAND, M.; SCHIJVEN, M. P. Systematic review on the effectiveness of augmented reality applications in medical training. Surgical Endoscopy, v. 30, n. 10, 2016.

[5] DOWNS, L. et al. Google Scanned Objects: a high-quality dataset of 3D scanned

household items. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION (ICRA), 2022, Filadélfia. Proceedings [...]. IEEE, 2022. arXiv:2204.11918.

[6] LAINE, S. et al. Modular primitives for high-performance differentiable rendering. ACM Transactions on Graphics, v. 39, n. 6, 2020.

[7] UMEYAMA, S. Least-squares estimation of transformation parameters between two point patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 13, n. 4, p. 376-380, 1991.

[8] RADFORD, A. et al. Learning transferable visual models from natural language supervision. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML), 38., 2021. Proceedings [...]. PMLR, 2021. arXiv:2103.00020.

[9] ZHAO, Z. et al. Hunyuan3D 2.0: scaling diffusion models for high resolution textured 3D assets generation. arXiv:2501.12202, 2025.

[10] LAI, Z. et al. Unleashing vecset diffusion model for fast shape generation. arXiv:2503.16302, 2025.

[11] HUNYUAN3D TEAM. Hunyuan3D 2.1: from images to high-fidelity 3D assets with production-ready PBR material. arXiv:2506.15442, 2025.

[12] XIANG, J. et al. Structured 3D latents for scalable and versatile 3D generation. arXiv:2412.01506, 2024.

[13] TOCHILKIN, D. et al. TripoSR: fast 3D object reconstruction from a single image. arXiv:2403.02151, 2024.

[14] BOSS, M. et al. SF3D: stable fast 3D mesh reconstruction with UV-unwrapping and illumination disentanglement. arXiv:2408.00653, 2024.

[15] HUANG, Z. et al. SPAR3D: stable point-aware reconstruction of 3D objects from single images. arXiv:2501.04689, 2025.

[16] XU, J. et al. InstantMesh: efficient 3D mesh generation from a single image with sparse-view large reconstruction models. arXiv:2404.07191, 2024.

[17] RAMÍREZ, S. FastAPI. Disponível em: https://fastapi.tiangolo.com.

[18] SQLITE. Write-Ahead Logging. Disponível em: https://www.sqlite.org/wal.html.

[19] EXPO. Expo documentation. Disponível em: https://docs.expo.dev.

[20] GOOGLE. model-viewer: easily display interactive 3D models on the web and in AR. Disponível em: https://modelviewer.dev.

[21] REACTVISION. ViroReact: AR and VR for React Native. Disponível em: https://github.com/ReactVision/viro.

[22] KHRONOS GROUP. glTF 2.0 Specification. Beaverton: Khronos Group, 2021. Disponível em: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html.

[23] GOOGLE. Support 16 KB page sizes. Android Developers. Disponível em: https://developer.android.com/guide/practices/page-sizes.

[24] BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados

Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018.

[25] MACENSKI, S. et al. Robot Operating System 2: design, architecture, and uses in the wild. Science Robotics, v. 7, n. 66, eabm6074, 2022.

[26] KOENIG, N.; HOWARD, A. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS (IROS), 2004, Sendai. Proceedings [...]. IEEE, 2004. p. 2149-2154.

[27] GEBRU, T. et al. Datasheets for datasets. Communications of the ACM, v. 64, n. 12, p. 86-92, 2021.

[28] TOBIN, J. et al. Domain randomization for transferring deep neural networks from simulation to the real world. In: IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS (IROS), 2017, Vancouver. Proceedings [...]. IEEE, 2017. p. 23-30.

[29] BAN, S.; FAN, J.; MA, X.; ZHU, W.; QIAO, Y.; WANG, Y. Real-time holistic robot pose estimation with unknown states. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 2024. Anais [...]. Cham: Springer, 2024.

[30] YANG, L. et al. Depth Anything V2. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 37., 2024. Proceedings [...]. 2024.

[31] LEE, T. E.; TREMBLAY, J.; TO, T.; CHENG, J.; MOSIER, T.; KROEMER, O.; FOX, D.; BIRCHFIELD, S. Camera-to-robot pose estimation from a single image. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION (ICRA), 2020. Anais [...]. IEEE, 2020. p. 9426-9432.

[32] SUN, K.; XIAO, B.; LIU, D.; WANG, J. Deep high-resolution representation learning for human pose estimation. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2019. Anais [...]. IEEE, 2019. p. 5693-5703.

[33] HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2016. Anais [...]. IEEE, 2016. p. 770-778.

[34] LABBÉ, Y.; CARPENTIER, J.; AUBRY, M.; SIVIC, J. Single-view robot pose and joint angle estimation via render & compare. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2021. Anais [...]. IEEE, 2021. p. 1654-1663.

[35] COLEMAN, D. et al. Reducing the barrier to entry of complex robotic software: a MoveIt! case study. Journal of Software Engineering for Robotics, v. 5, n. 1, p. 3-16, 2014.

[36] ŞUCAN, I. A.; MOLL, M.; KAVRAKI, L. E. The Open Motion Planning Library. IEEE Robotics & Automation Magazine, v. 19, n. 4, p. 72-82, 2012.

[37] KUFFNER, J. J.; LAVALLE, S. M. RRT-Connect: an efficient approach to single-query path planning. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION (ICRA), 2000, San Francisco. Proceedings [...]. IEEE, 2000.

p. 995-1001.

[38] GRIEVES, M.; VICKERS, J. Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In: KAHLEN, F.-J.; FLUMERFELT, S.; ALVES, A. (ed.). Transdisciplinary perspectives on complex systems. Cham: Springer, 2017. p. 85-113.

[39] GROMEŠ, J. RadioLib: universal wireless communication library for embedded devices. Versão 7.8.1. Disponível em: https://github.com/jgromes/RadioLib.

[40] THE THINGS NETWORK. Frequency plans by country. Disponível em: https://www.thethingsnetwork.org/docs/lorawan/frequencies-by-country/. Acesso em: out. 2026.

[41] AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES (ANATEL). Resolução nº 680, de 27 de junho de 2017. Aprova o Regulamento sobre Equipamentos de Radiocomunicação de Radiação Restrita. Brasília: Anatel, 2017.

[42] AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES (ANATEL). Ato nº 14.448, de 4 de dezembro de 2017. Aprova os requisitos técnicos para a avaliação da conformidade de equipamentos de radiocomunicação de radiação restrita. Brasília: Anatel, 2017.

[43] SEMTECH CORPORATION. AN1200.13: LoRa Modem Designer's Guide. Camarillo: Semtech, 2013.

[44] ABRAMSON, N. The ALOHA System: another alternative for computer communications. In: FALL JOINT COMPUTER CONFERENCE, 1970, Houston. Proceedings [...]. New York: ACM, 1970. p. 281-285.

[45] HATA, M. Empirical formula for propagation loss in land mobile radio services. IEEE Transactions on Vehicular Technology, v. 29, n. 3, p. 317-325, 1980.

[46] KUHN, H. W. The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, v. 2, n. 1-2, p. 83-97, 1955.

[47] KAPOUTSIS, A. C.; CHATZICHRISTOFIS, S. A.; KOSMATOPOULOS, E. B. DARP: divide areas algorithm for optimal multi-robot coverage path planning. Journal of Intelligent & Robotic Systems, v. 86, n. 3-4, p. 663-680, 2017.

[48] DOUGLAS, D. H.; PEUCKER, T. K. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. Cartographica, v. 10, n. 2, p. 112-122, 1973.

[49] CHOSET, H. Coverage of known spaces: the boustrophedon cellular decomposition. Autonomous Robots, v. 9, n. 3, p. 247-253, 2000.

[50] OPENSTREETMAP CONTRIBUTORS. OpenStreetMap. Dados disponibilizados sob a licença Open Database License (ODbL). Disponível em: https://www.openstreetmap.org. Acesso em: 26 out. 2026.

[51] KOENIG, S. et al. The power of sequential single-item auctions for agent coordination. In: NATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE (AAAI-06), 21., 2006, Boston. Proceedings [...]. Menlo Park: AAAI Press, 2006. p. 1625-1629.

[52] ZHU, P. et al. Detection and tracking meet drones challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 44, n. 11, p. 7380-7399, 2022.

[53] JOCHER, G.; QIU, J. Ultralytics YOLO11. Versão 11.0.0, 2024. Disponível em: https://github.com/ultralytics/ultralytics.

8. ANEXOS

O código-fonte, os resultados medidos (arquivos JSON e CSV), os registros de execução e as evidências citadas neste relatório (gráficos, capturas de tela e saídas de terminal) estão nos repositórios listados a seguir. Os repositórios são privados, e o acesso pode ser concedido mediante solicitação ao bolsista. Dados volumosos, como o dataset completo do braço robótico e os pesos dos modelos treinados, não foram versionados e permanecem na estação DGX Spark.

Tabela 31. Repositórios com o código, os resultados medidos e as evidências das atividades

Repositório	Conteúdo
cerise-2026-gen3d	Análise comparativa, otimização e validação da geração 3D; servidor de geração 3D (pasta server)
cerise-ar, pull request #1	Aplicativo CERISE AR 1.1.0: integração com o servidor e pacote da Play Store (branch feat/2026-gen3d-playstore)
cerise-2026-servo6dof-gemeo-digital	Geração do dataset do braço (datasheet e amostra), modelo digital e gêmeo digital
cerise-2026-pose-servo6dof	Porte, treino e avaliação do modelo de estimação de pose e módulo de inferência
cerise-2026-lora-prototipo	Protocolo, firmware, testes, simulação e projeto do módulo de comunicação LoRa
cerise-2026-drones-segmentacao	Segmentação de áreas, colaboração entre drones e detecção e contagem de pessoas